

SoC 를 위한 *Peripheral* 설계

Reference : MicroBlaze.v15 [IHIL]

2025-06-24

Table of Contents

Microblaze
BlazeTop.xpr

➤ SoC를 위한 Peripheral 설계

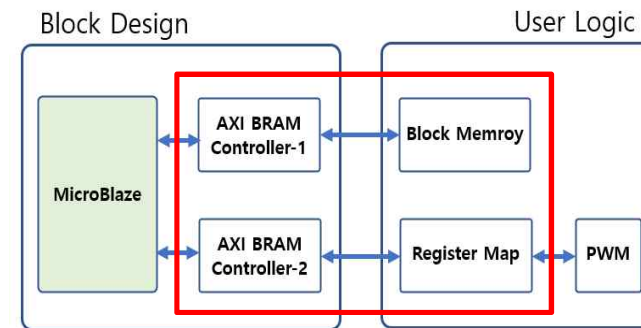
1. Xilinx IP
2. Create and Package New IP
3. SPI
 - 1) SPI Master
 - 2) SPI Slave
 - 3) SPI Controller
4. UART
5. AMBA
6. MicroBlaze_Hello World
7. **MicroBlaze_Hello World & LED_Counter**
 - Seral Flash Program Download
 - Vitis_elf_Vivado_Serial Flash Program Download
8. MicroBlaze_Peripheral Implementation
9. MicroBlaze_User Logic Interface

10. SPI_Master_IP(MicroBlaze_User Logic Interface)

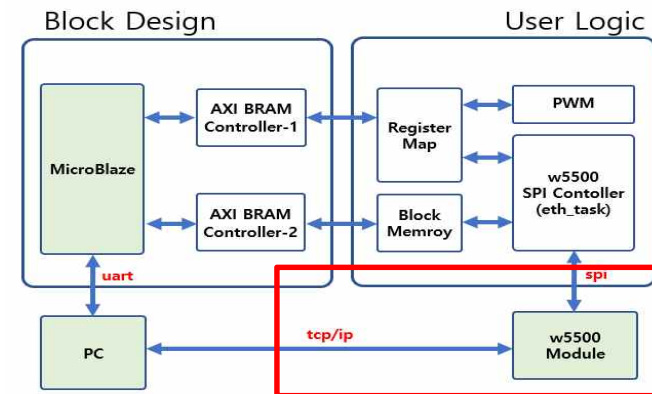
11. TCP_IP Implementation Using W5500

12. MicroBlaze_Block Memory Interface-1

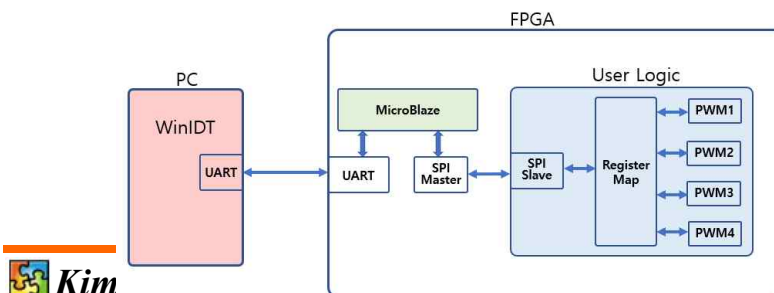
13. MicroBlaze_Block Memory Interface-2



14. w5500 Interface Implementation



➤ SoC Peripheral RTC Design Project



LED_Counter Implementation

Microblaze
BlazeTop.xpr

- MicroBlaze를 이용하여 *Hello World* 출력하고, 간단한 사용자 로직(*User Logic*)으로 LED를 On/Off하는 것을 구현
 - MicroBlaze는 *Processor Core*와 *Peripheral* 이 분리
 - 원하는 *Peripheral* 추가 및 개수도 필요한 만큼 추가하여 사용 가능
 - MicroBlaze *Processor Core*와 *Peripheral* 중에 *UART* 를 추가하여 *UART* 포트를 통하여 *Hello World* 를 출력 기능 구현
- *FPGA*에서 MicroBlaze를 사용하는 목적
 - 사용자가 만든 *User Logic*을 제어하고, *Logic* 으로 구현할 수 없는 부분을 MicroBlaze 로 구현
 - *FPGA*에서 *IP* 와 *User Logic* 을 생성해서 MicroBlaze사용하는 궁극적인 목적은 HW로 구현하는 부분과 SW로 구현하는 부분을 나누어서 구현하고 서로 간에 인터페이스를 구현하여 외부에 추가적인 부품을 사용하지 않는 SoC (System on Chip)를 구현

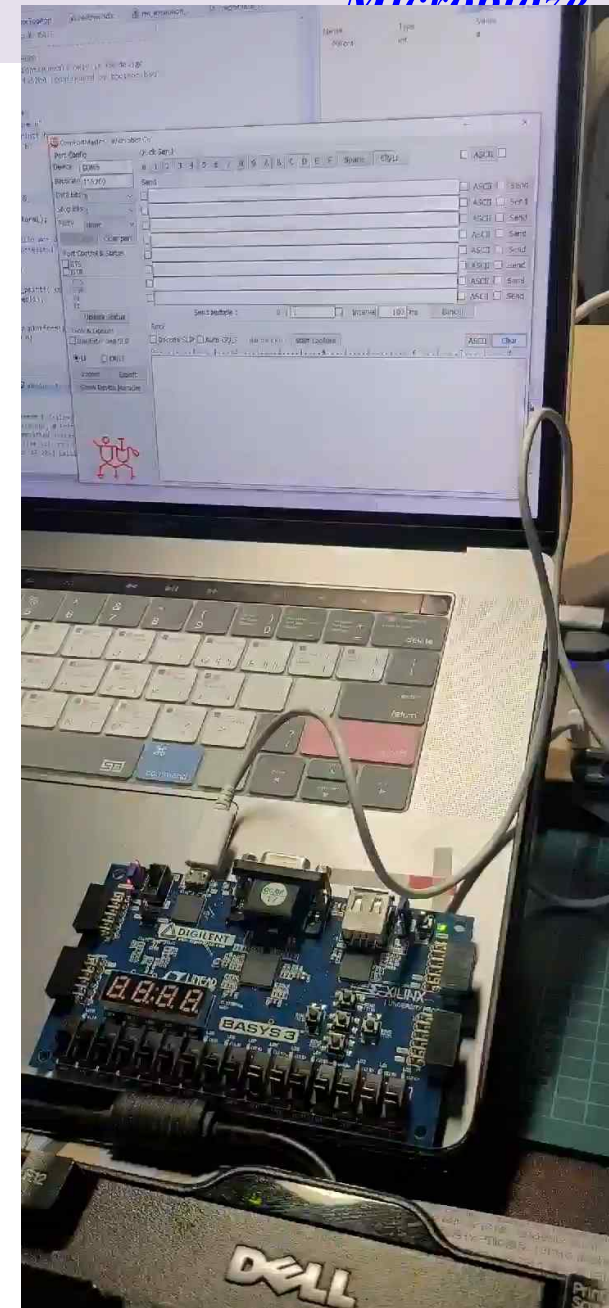
LED_Counter Implementation

Microblaze

➤ The Goal → LED_Counter

➤ Check The Result

- LD0 ~ LD3 이 0.5초 간격으로 On/Off
- 기본적인 동작이 완료 → [LED_Counter Module]에서 구현한 LED on/off 동작과 SW에서 구현한 1초마다 메시지를 표시



- 1. Create Project*
- 2. Create Block Design*
- 3. Add IP → MicroBlaze (Processor Core & Memory & Clock & Reset & ...)*
- 4. Add IP → AXI Uartlite (Uart IP and Baudrate)*
- 5. Change Port Name and Interrupt Pin Connection*
- 6. Validate Design and Regenerate Layout*
- 7. Create HDL Wrapper*
- 8. User Logic Implementation(1) → Led_On/Off_Logic → led_control*
- 9. User Logic Implementation(2) → Led_On/Off_Logic → BlazeTop*
- 10. XDC Implementation*
- 11. Generate Bitstream*
- 12. Export Hardware (XSA File Extraction)*
- 13. SW Implementation(1) → Launch Vitis IDE & Create Application Project*
- 14. SW Implementation(2) → [helloworld.c] Code Implementation*
- 15. SW Implementation(3) → Build Project , Debug and Program Device*
- 16. Check The Result*

LED_Counter Implementation

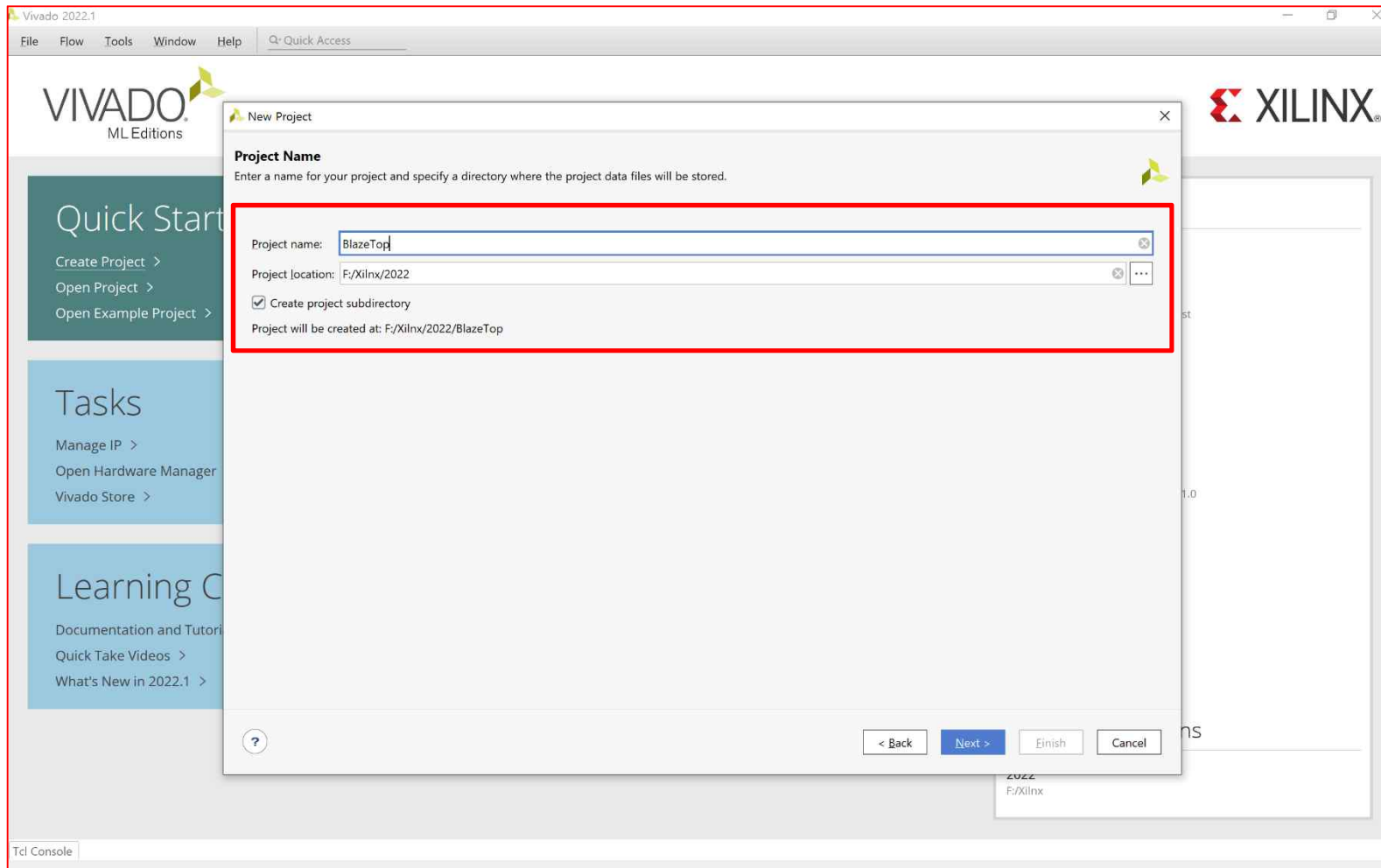
➤ *Create Project*

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation → Basic Template Implementation

- Vivado 실행 → [Create Project] 클릭 → [Next]
- [Project name] 설정 → [**BlazeTop**]

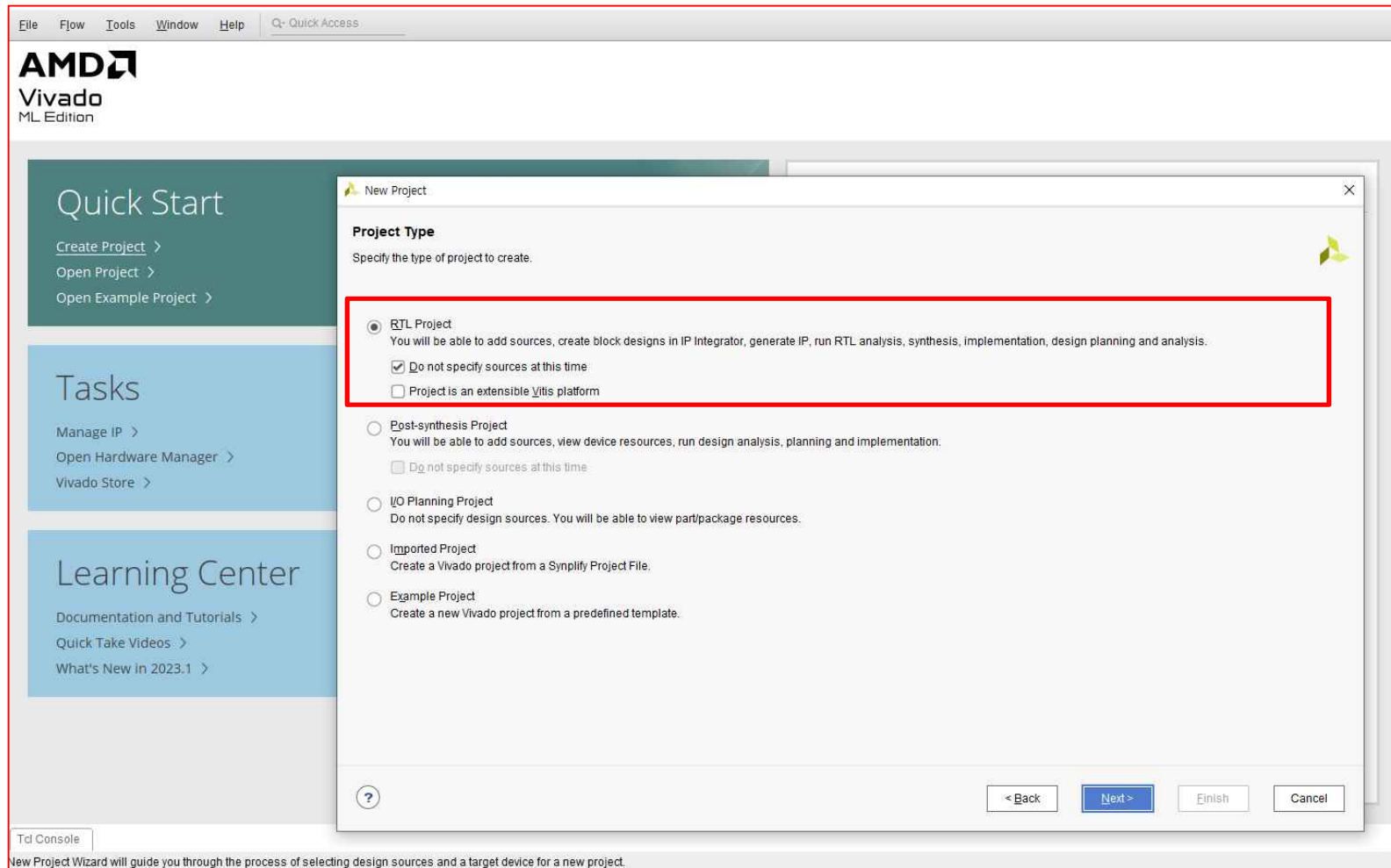


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ **LED_Counter Implementation → Basic Template Implementation**

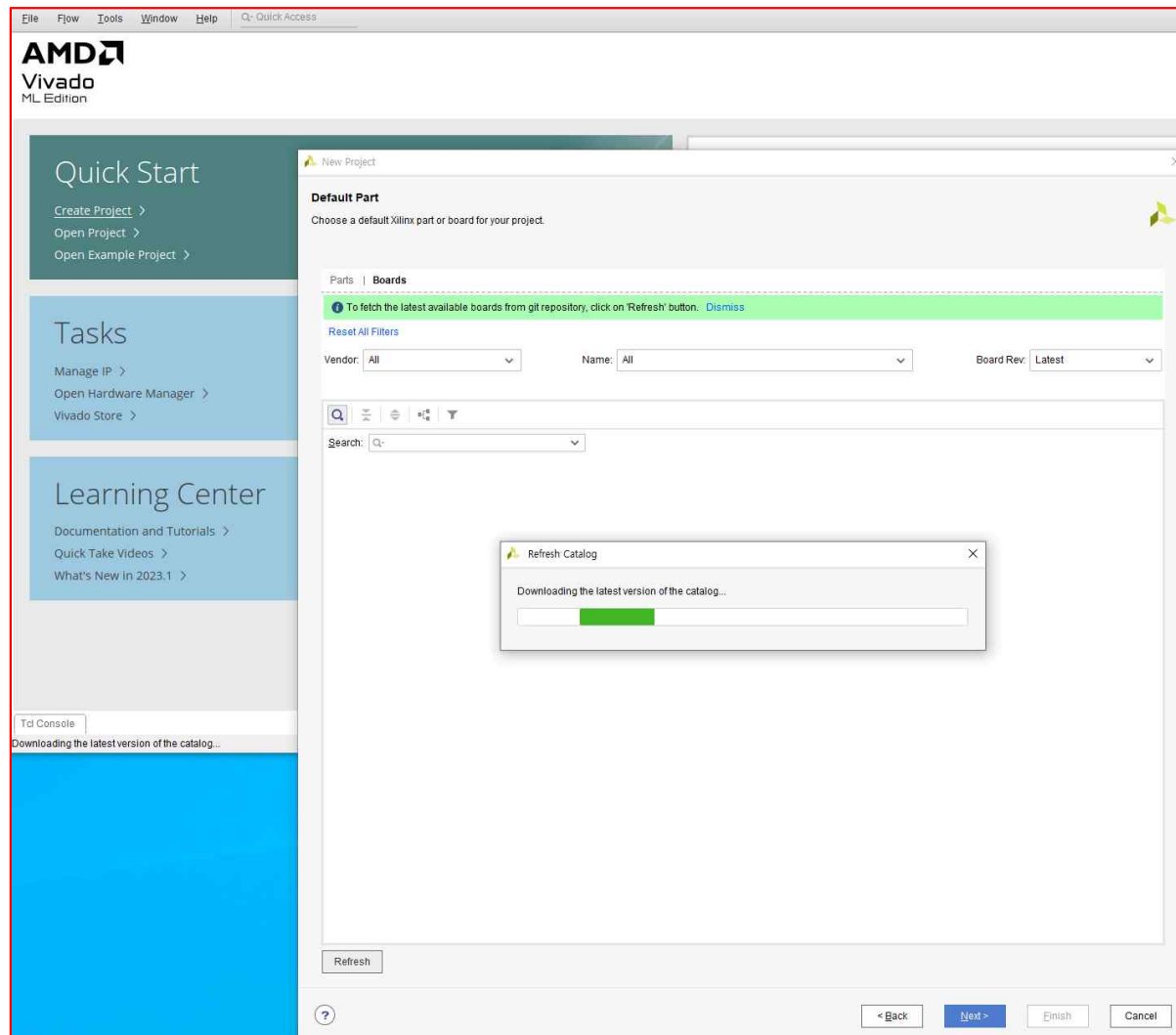
▪ **RTL Project → Next**



LED_Counter Implementation

Microblaze
BlazeTop.xpr

- LED_Counter Implementation ➔ Basic Template Implementation
 - Default Part ➔ Boards ➔ Basys3 ➔ Next



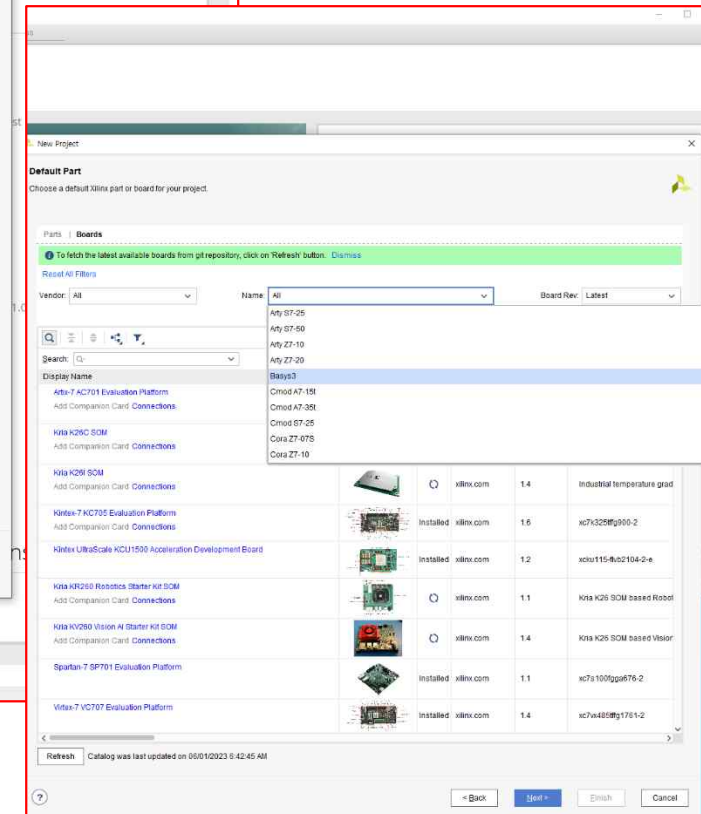
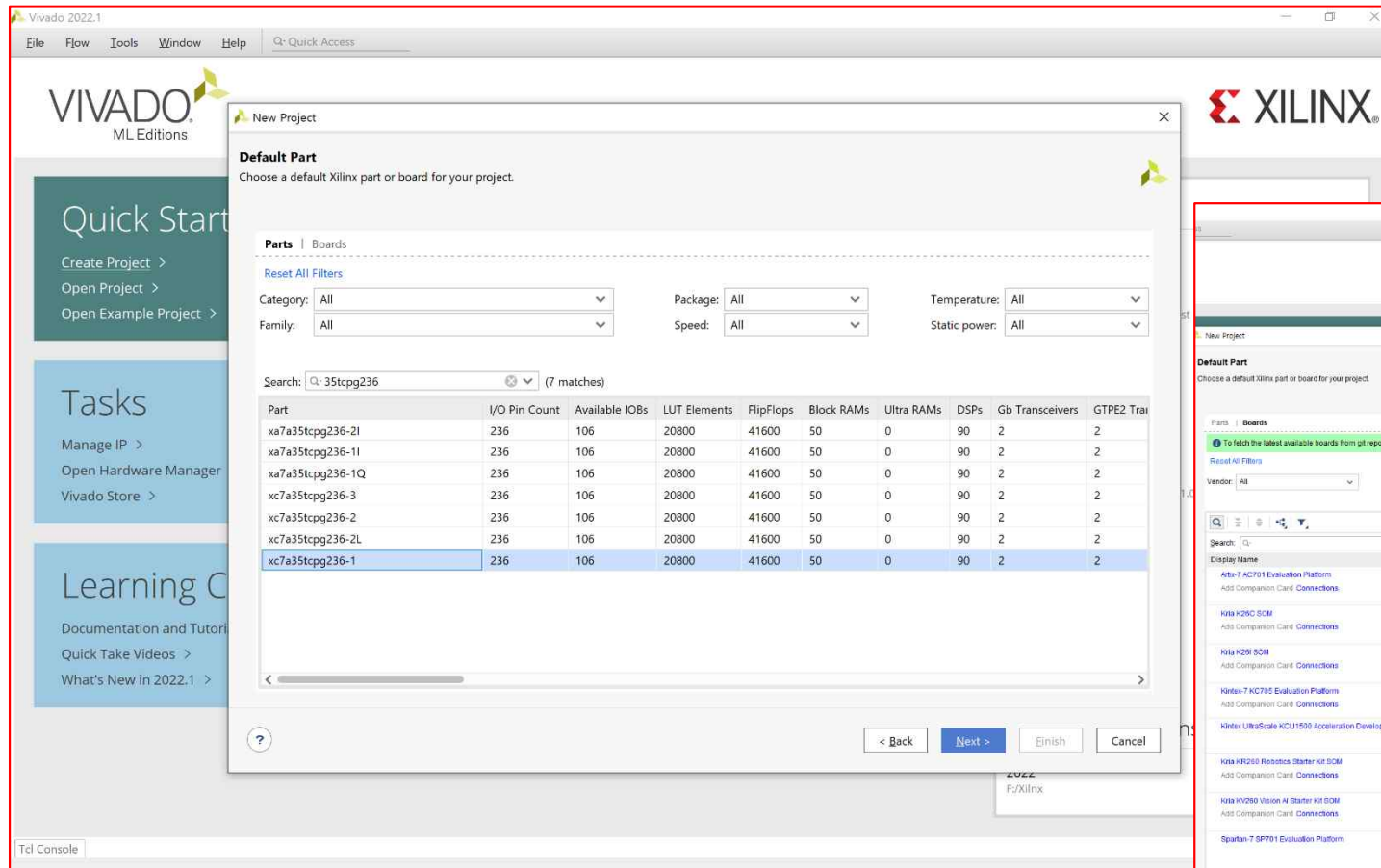
LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation ➔ Basic Template Implementation

▪ Default Part ➔ Parts ➔ xc7a35tcpg236-1 ➔ Next

✓ [or] Default Part ➔ Boards ➔ Basys3 ➔ Next



LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation ➔ Basic Template Implementation

▪ Default Part ➔ Parts ➔ **xc7a35tcpg236-1** ➔ Next

✓ [or] Default Part ➔ Boards ➔ **Basys3** ➔ Next

New Project

Default Part

Choose a default Xilinx part or board for your project.

Parts | Boards

To fetch the latest available boards from git repository, click on 'Refresh' button. Dismiss

Reset All Filters

Vendor: All Name: Basys3 Board Rev: Latest

Search: Q

Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count	Board Rev
Basys3			digilentinc.com	1.2			

Install

Refresh Catalog was last updated on 06/01/2023 6:42:45 AM

< Back Next > Finish Cancel

New Project

Default Part

Choose a default Xilinx part or board for your project.

Parts | Boards

To fetch the latest available boards from git repository, click on 'Refresh' button. Dismiss

Reset All Filters

Vendor: All Name: Basys3 Board Rev: Latest

Search: Q

Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count	Board Rev	Available IOBs	LUT Elements	FlipFlops
Basys3			digilentinc.com	1.2	xc7a35tcpg236-1	236	C.0	106	20800	41600

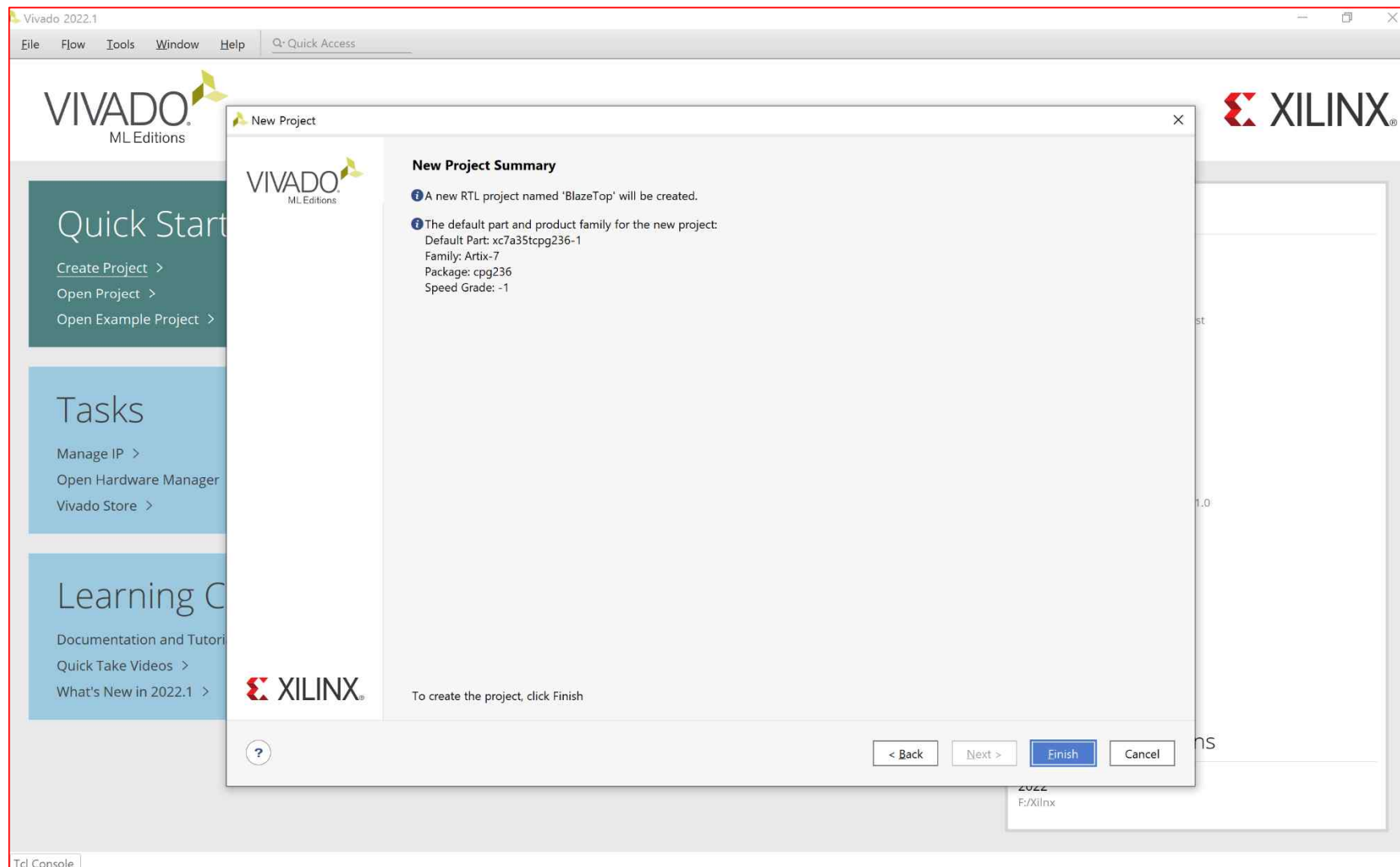
Refresh Catalog was last updated on 06/01/2023 6:42:45 AM

< Back Next > Finish Cancel

LED_Counter Implementation

Microblaze
BlazeTop.xpr

- LED_Counter Implementation ➔ Basic Template Implementation
 - New Project Summary ➔ Finish

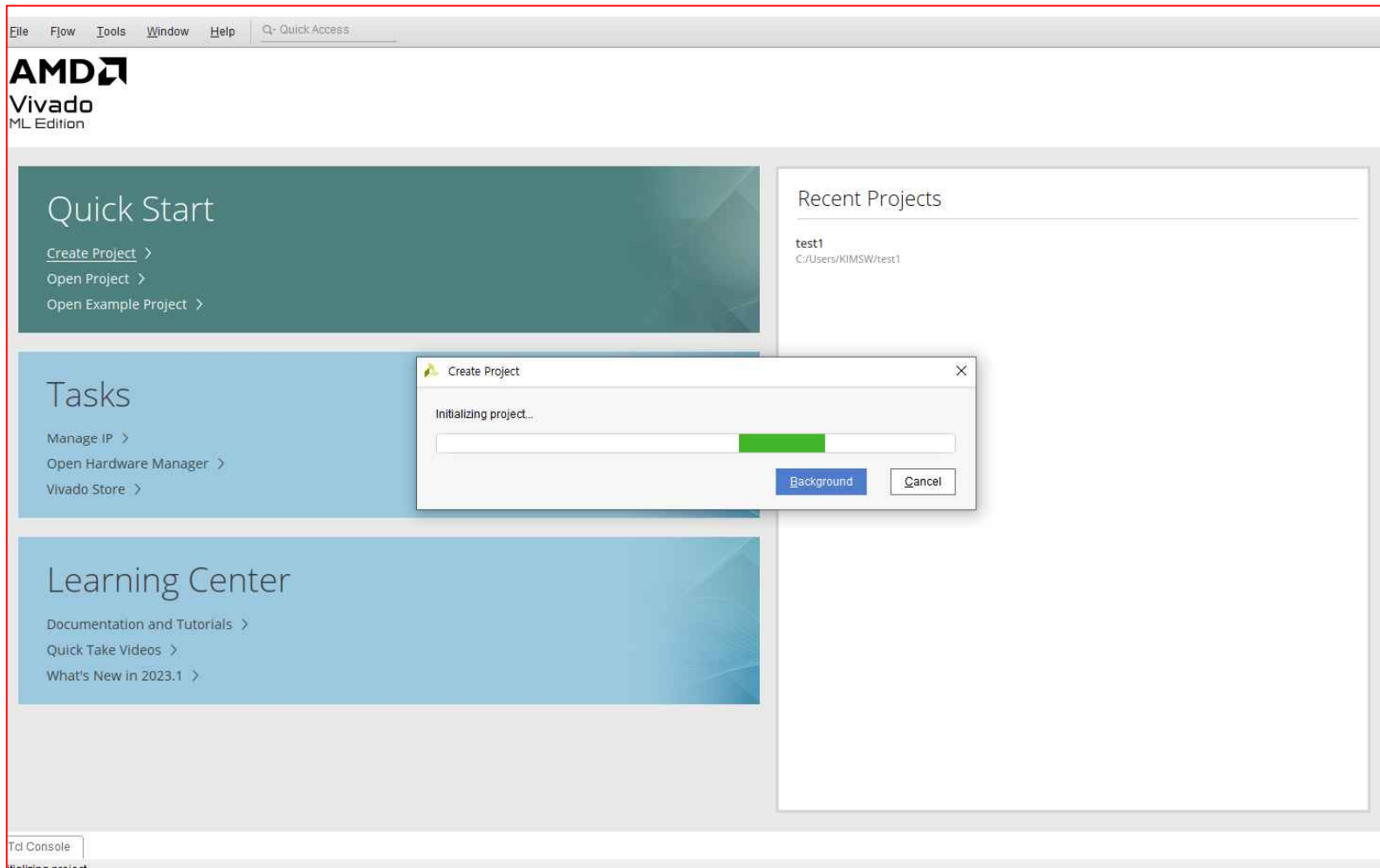


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation

- Create Project 완료



LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation

- Create Project 완료 ➔ **PROJECT MANAGER**

The screenshot shows the Vivado 2022.1 Project Manager interface for the project 'BlazeTop'. The left sidebar contains a 'Flow Navigator' with sections for PROJECT MANAGER, IP INTEGRATOR, SIMULATION, RTL ANALYSIS, SYNTHESIS, IMPLEMENTATION, and PROGRAM AND DEBUG. The main area is divided into three panes: Sources, Properties, and Project Summary.

Sources Pane: Shows a tree view of the project sources, including Design Sources, Constraints, Simulation Sources (sim_1), and Utility Sources. The 'Hierarchy' tab is selected.

Properties Pane: Currently empty, displaying the message 'Select an object to see properties'.

Project Summary Pane: Provides an overview of the project settings and synthesis/implementation status.

Project Summary Details:

- Settings:** Project name: BlazeTop, Project location: F:/Xilinx/2022/BlazeTop/BlazeTop.xpr, Product family: Artix-7, Project part: xc7a35tcpg236-1, Top module name: Not defined, Target language: Verilog, Simulator language: Mixed.
- Synthesis:** Status: Not started, Messages: No errors or warnings, Part: xc7a35tcpg236-1, Strategy: Vivado Synthesis Defaults, Report Strategy: Vivado Synthesis Default Reports, Incremental synthesis: Automatically selected checkpoint.
- Implementation:** Status: Not started, Messages: No errors or warnings, Part: xc7a35tcpg236-1, Strategy: Vivado Implementation, Report Strategy: Vivado Implementation, Incremental implementation: None.

Design Runs Table:

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP	Start
synth_1	constrs_1	Not started																	
impl_1	constrs_1	Not started																	

LED_Counter Implementation

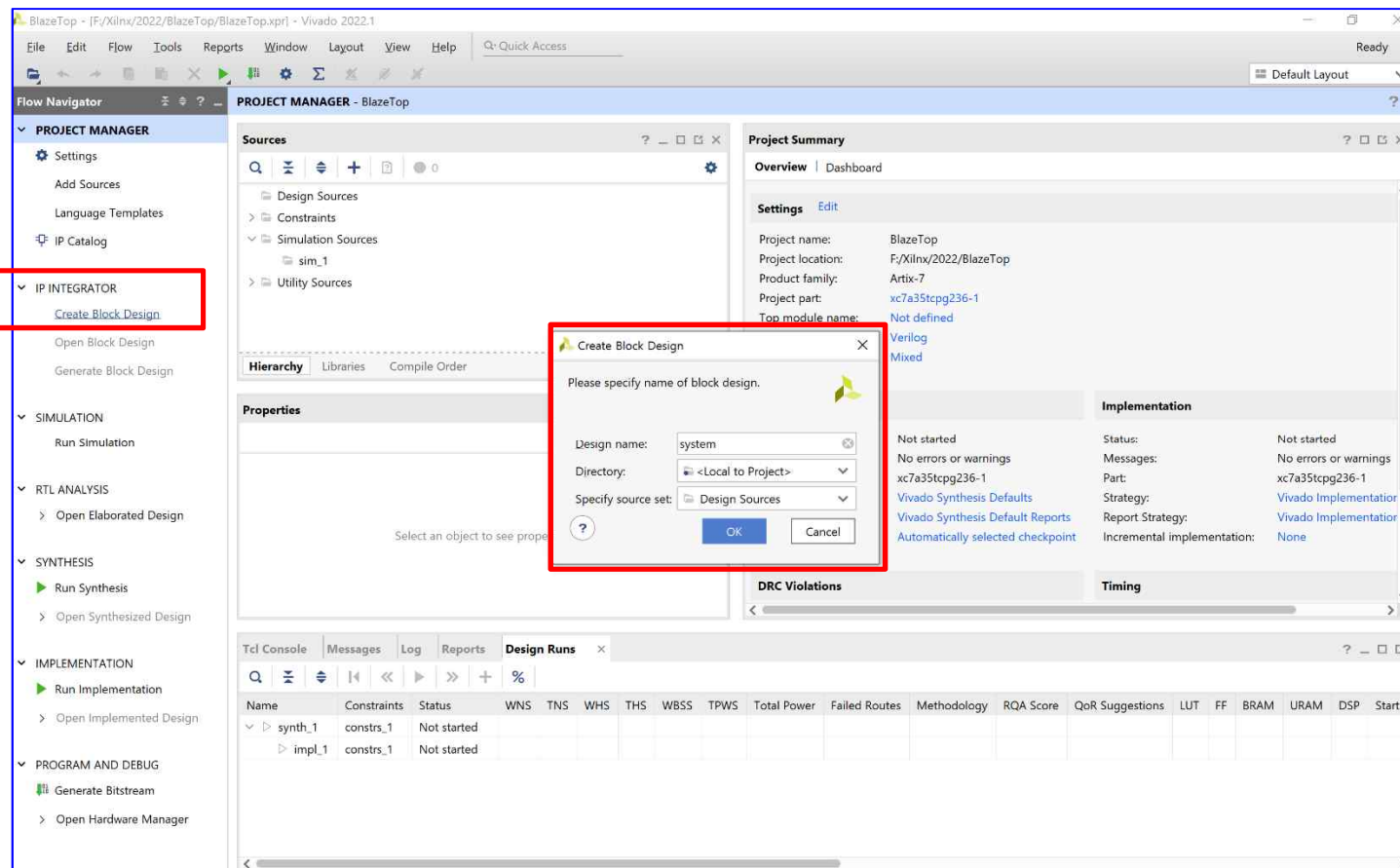
- *Create Project*
- *Create Block Design*

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation → Microblaze 생성

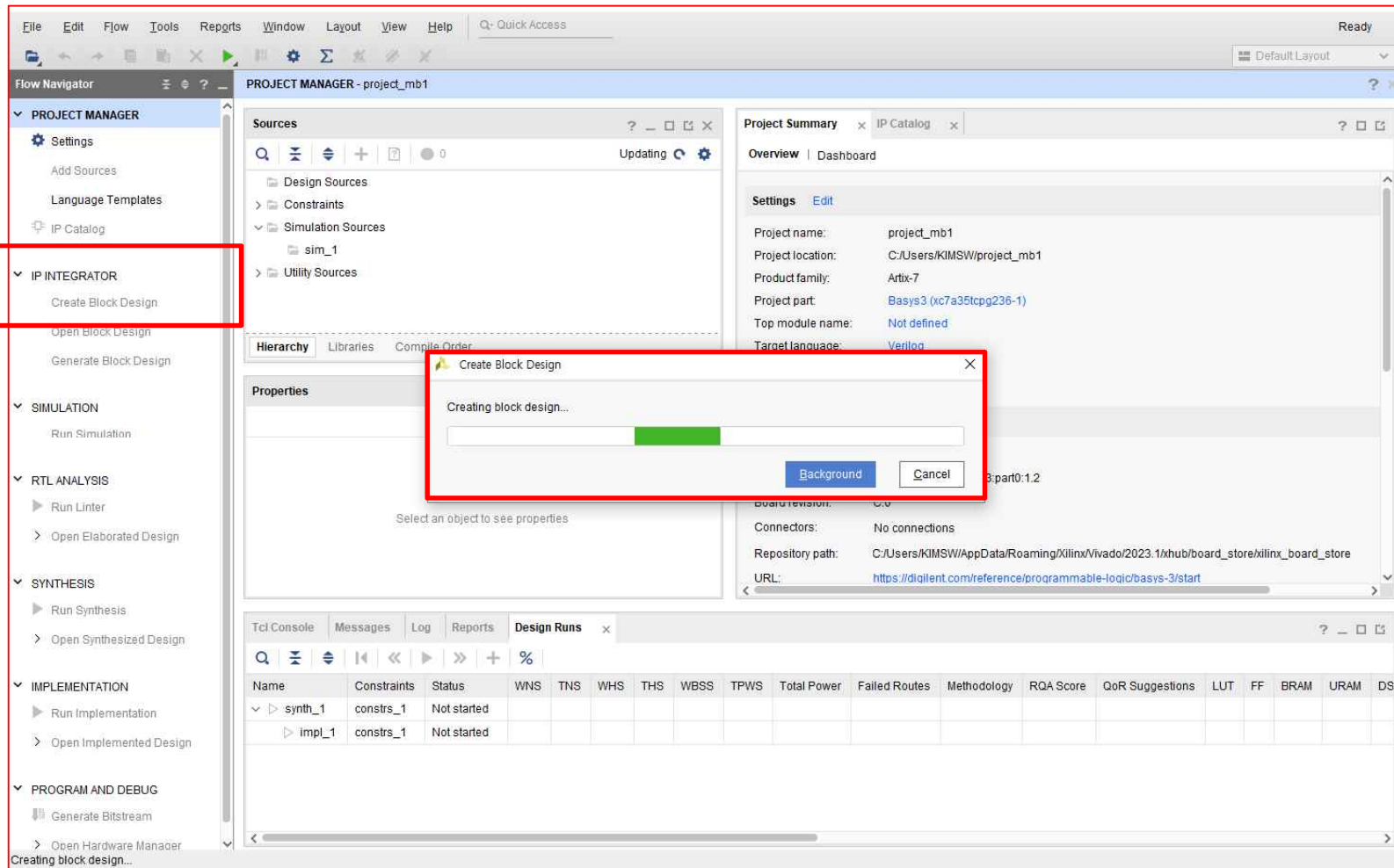
- PROJECT MANAGER → IP INTEGRATOR → Create Block Design
- Microblaze : 사용자가 원하는 IP Block 을 선택 , 속성 지정 및 각 Block 들을 연결해서 사용
- Design Name [(ex) system] 입력 → OK



LED_Counter Implementation

Microblaze
BlazeTop.xpr

- LED_Counter Implementation → Microblaze 생성
 - IP INTEGRATOR → Create Block Design



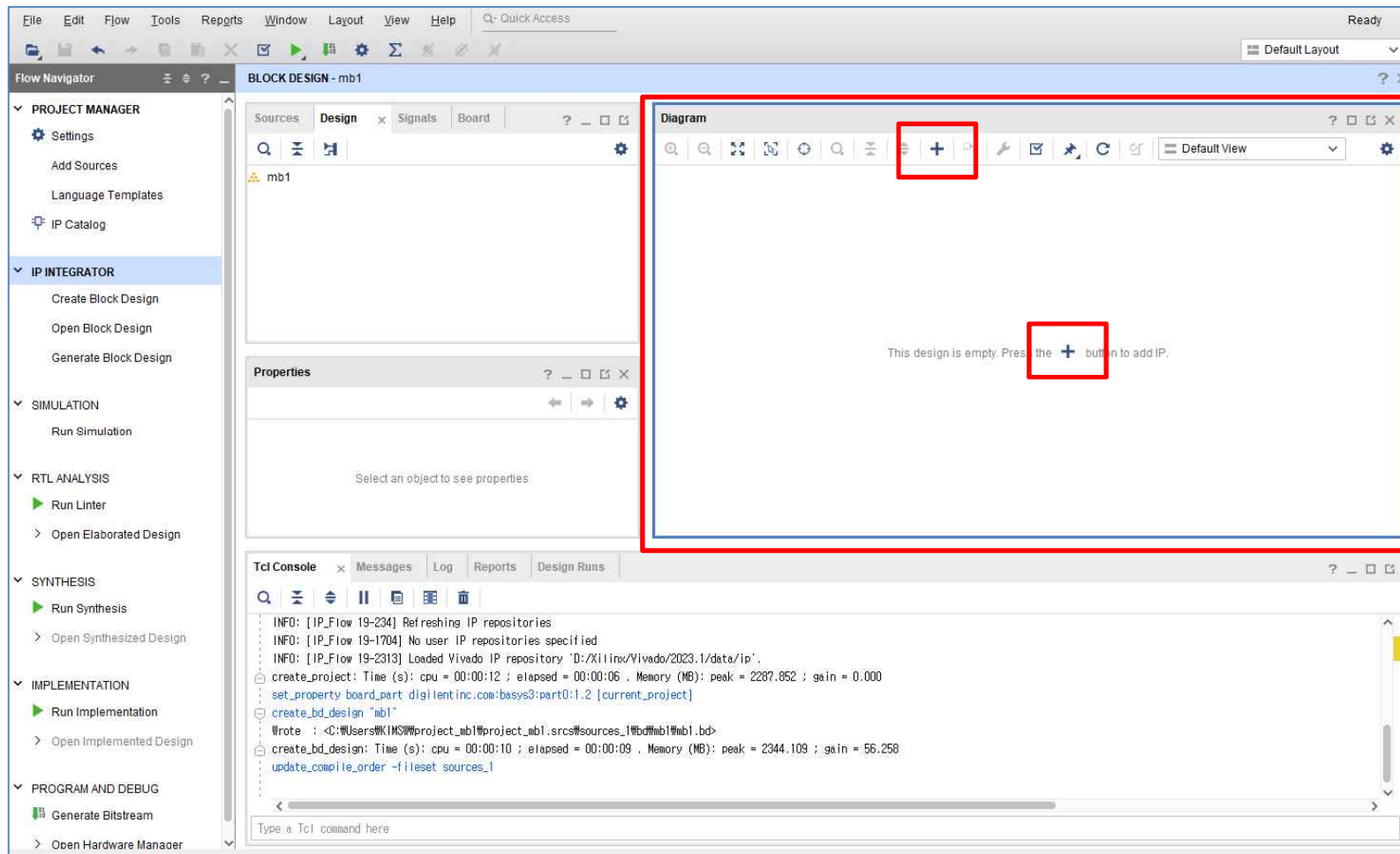
LED_Counter Implementation

- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*

LED_Counter Implementation

Microblaze
BlazeTop.xpr

- LED_Counter Implementation → Microblaze 생성
 - Diagram Window → Add IP → “+” 아이콘 클릭

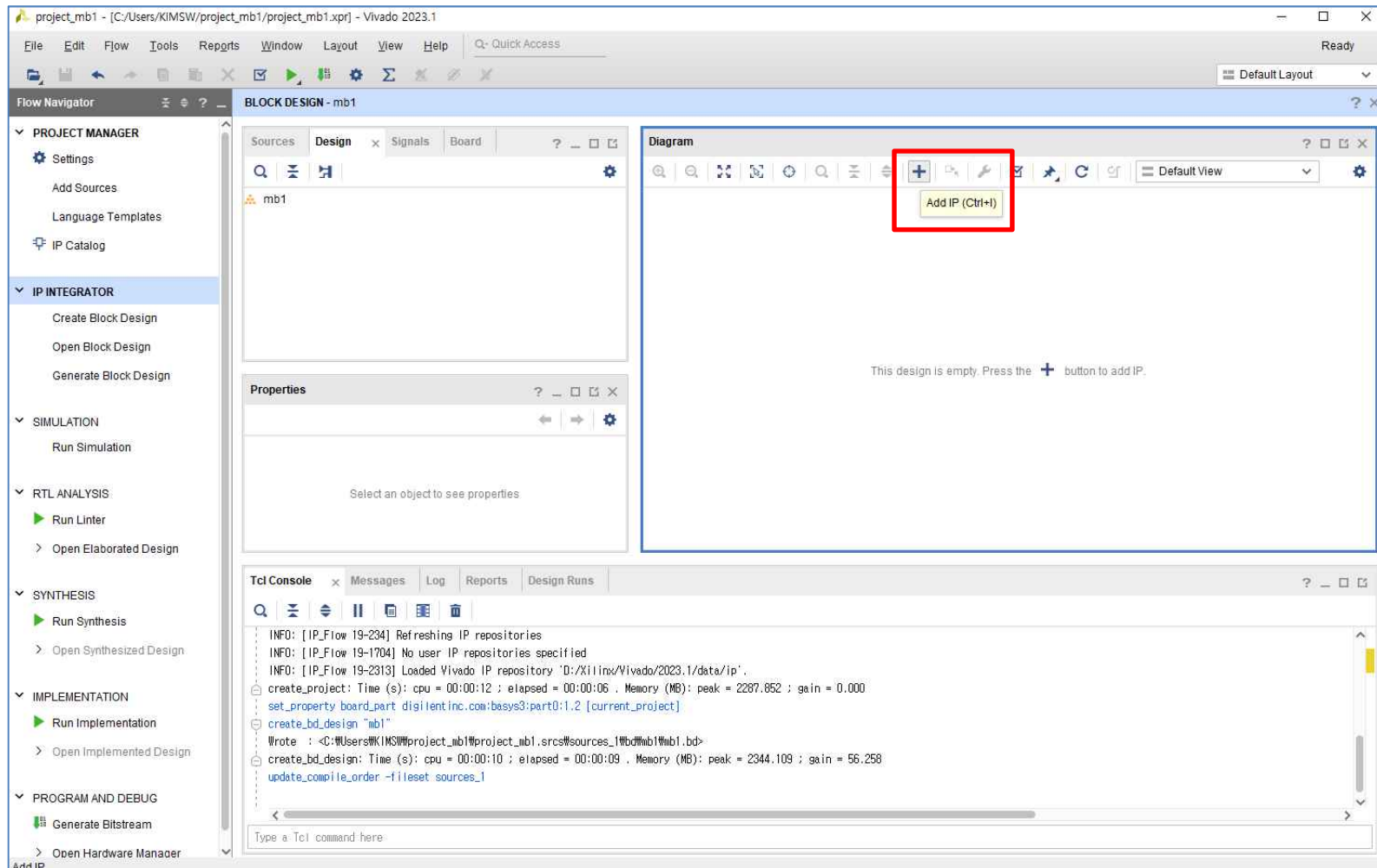


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation → Microblaze 생성

- Diagram Window → Add IP → “+” 아이콘 클릭 → “MicroBlaze” 검색 및 IP [MicroBlaze] 추가(1/4)

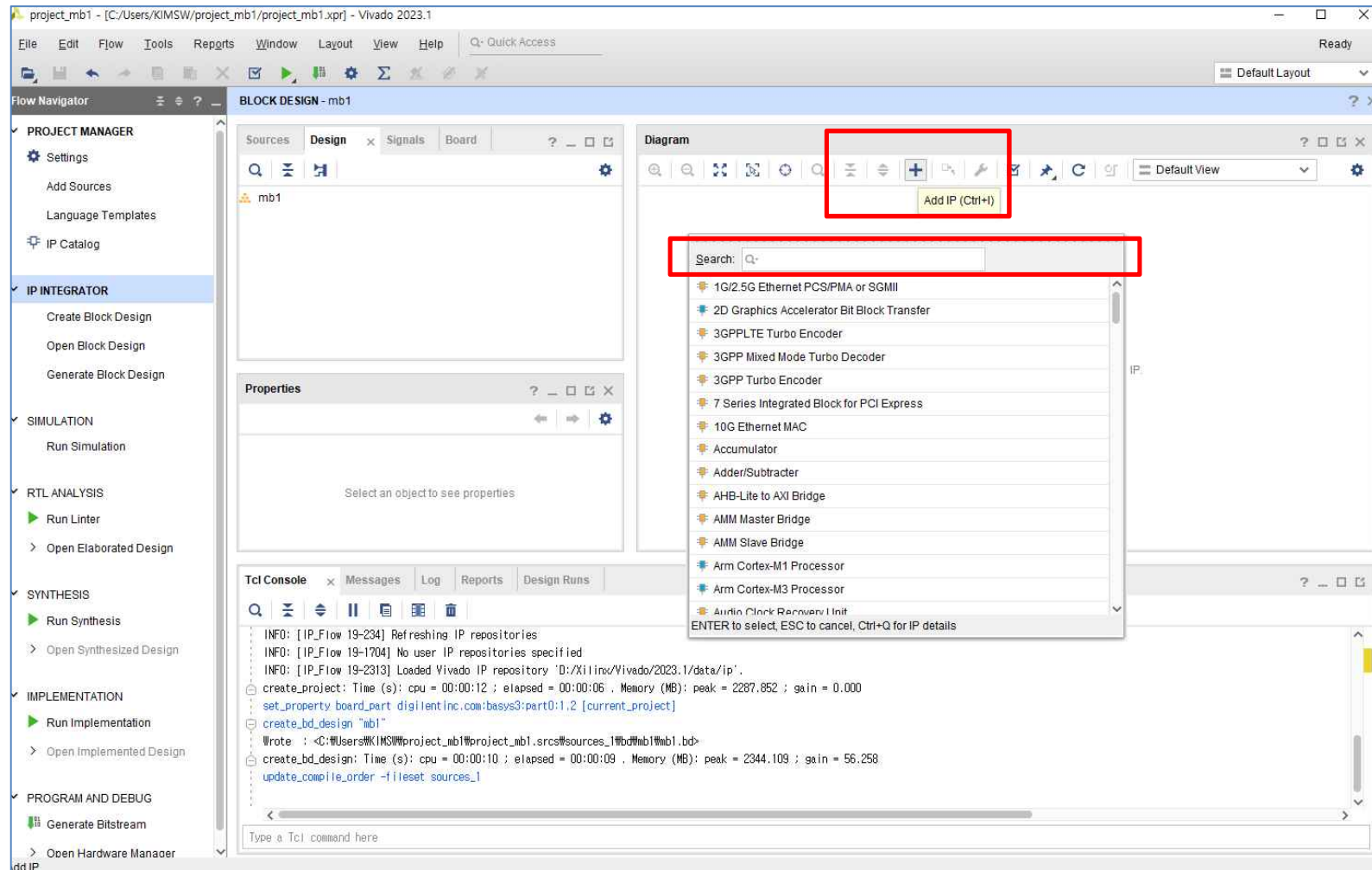


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation → Microblaze 생성

- Diagram Window → Add IP → “+” 아이콘 클릭 → “MicroBlaze” 검색 및 IP [MicroBlaze] 추가(2/4)

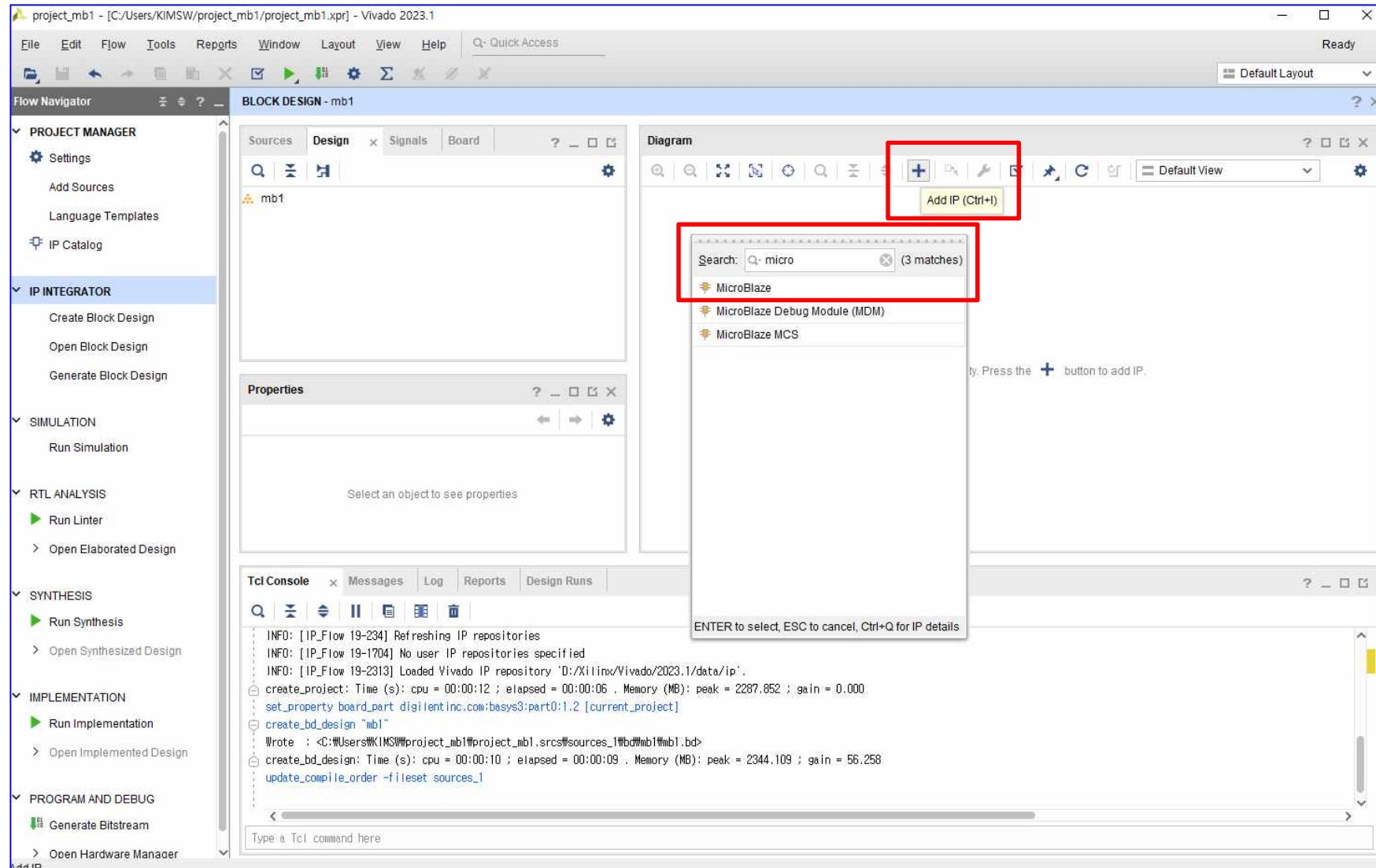


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation → Microblaze 생성

- Diagram Window → Add IP → “+” 아이콘 클릭 → “MicroBlaze” 검색 및 IP [MicroBlaze] 추가(3/4)

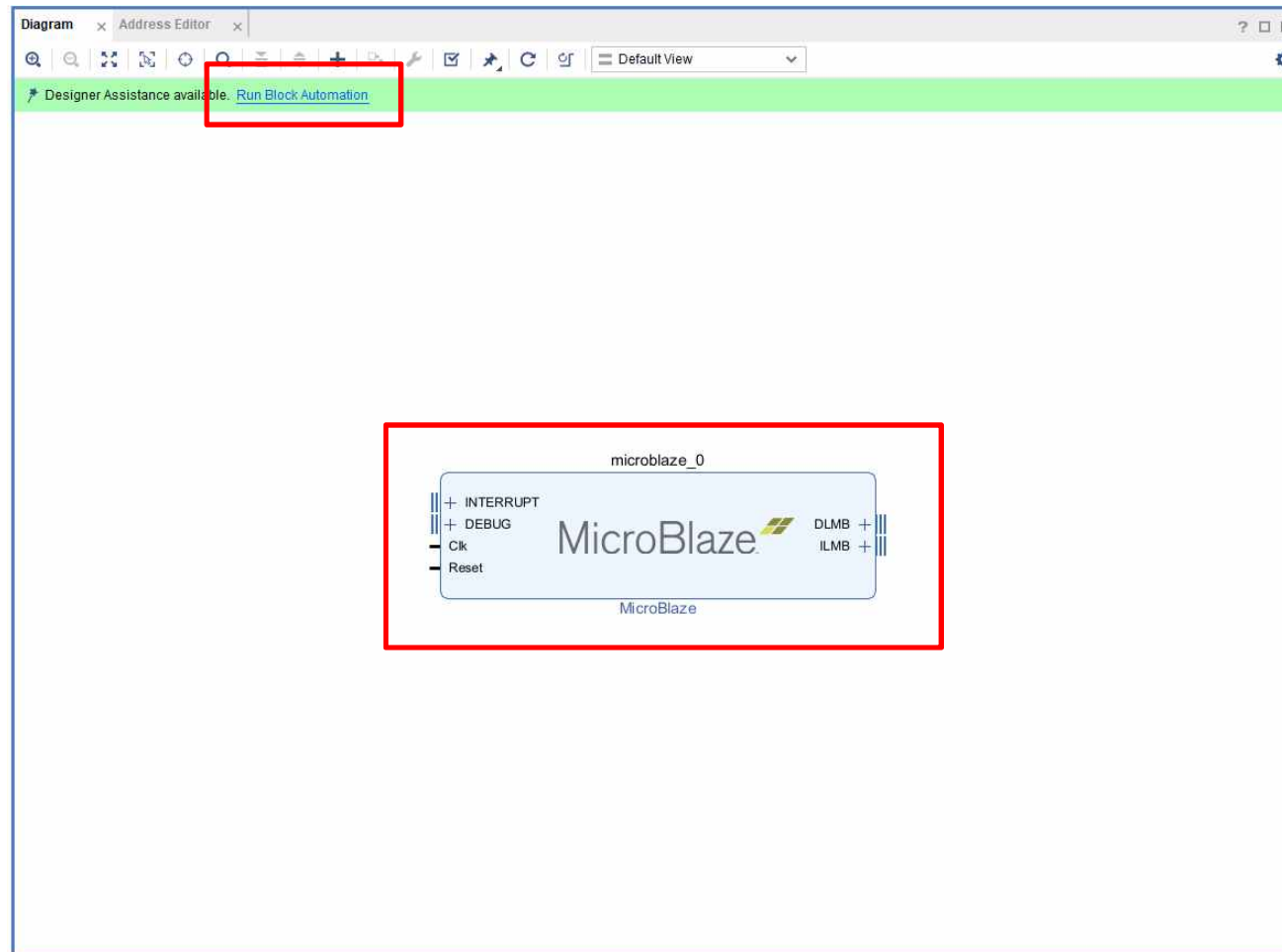


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation → Microblaze 생성

- Diagram Window → Add IP → “+” 아이콘 클릭 → “MicroBlaze” 검색 및 IP [MicroBlaze] 추가(4/4)
- Run Block Automation → Options

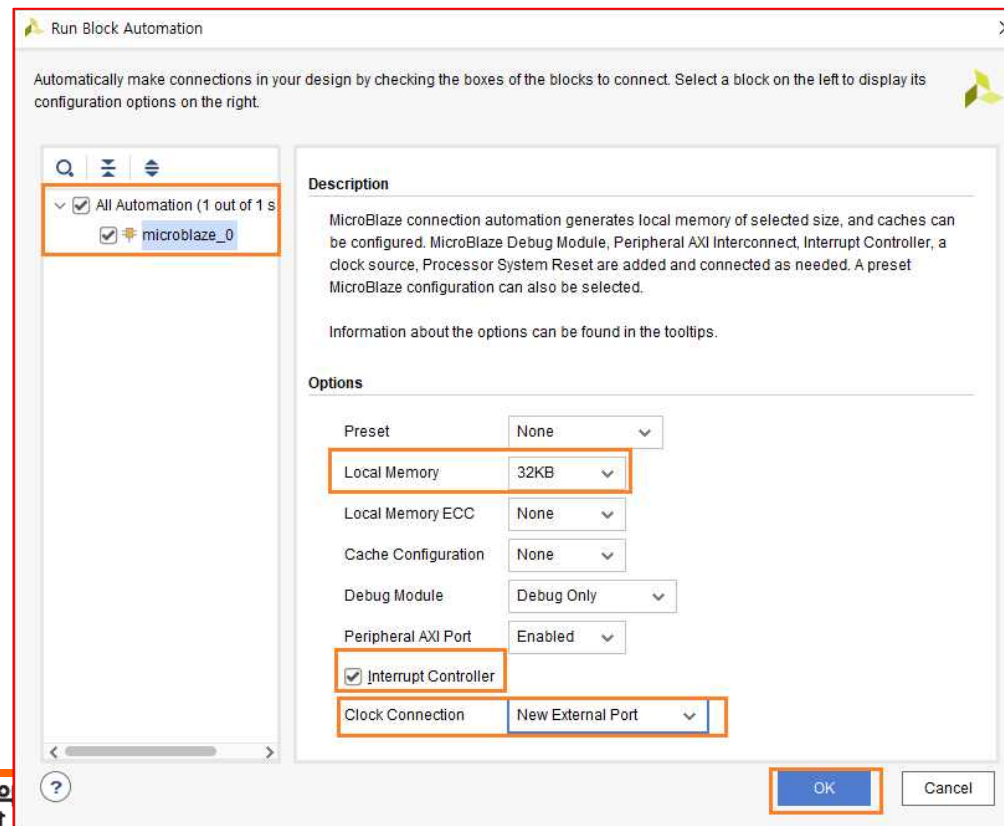


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation → Microblaze 생성

- Run Block Automation → Options
- Run Block Automation 윈도우에서 속성을 아래와 같이 설정하고 OK 를 클릭
- Local Memory : 32KB [기본값은 8KB 이지만, 32KB로 변경]
- Interrupt Controller : checked, Interrupt Controller를 사용하기 위하여 선택
- Clock Connection : New External Port [기본값은 New Clock Wizard → 보드에 있는 100Mhz]
- clock을 그대로 사용하기 위하여 New External Port를 선택



Microblaze
BlazeTop.xpr

- *Run Block Automation ➔ Options*
- *Run Block Automation* 윈도우에서 속성을 아래와 같이 설정하고 OK 를 클릭
- *Local Memory : 32KB* [기본값은 8KB 이지만, 32KB로 변경]
- *Interrupt Controller : checked, Interrupt Controller*를 사용하기 위하여 선택
- *Clock Connection : New External Port* [기본값은 New Clock Wizard ➔ 보드에 있는 100MHz]
- *clock*을 그대로 사용하기 위하여 *New External Port*를 선택

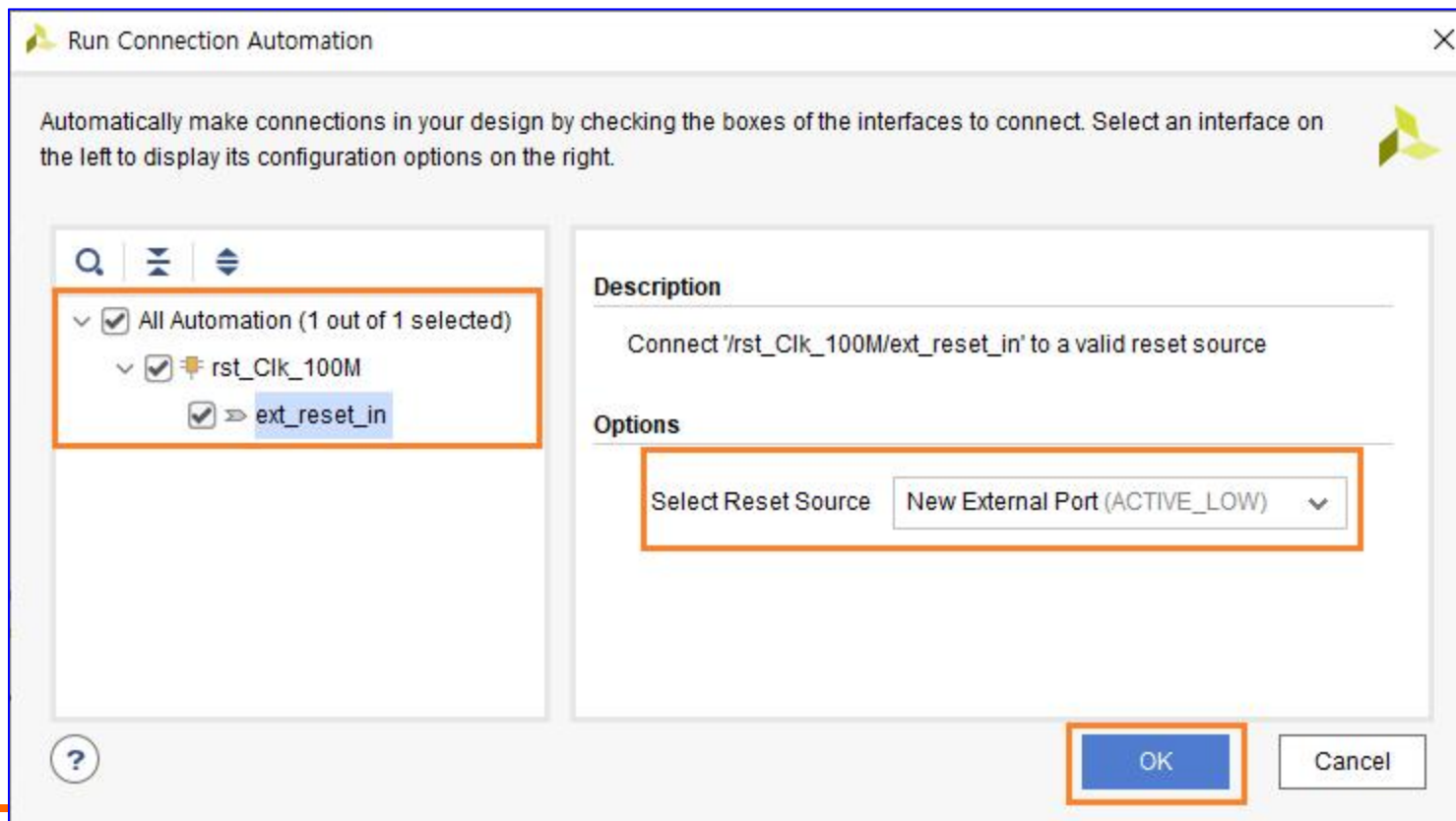


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation

- **Run Connection Automation** → **Run Connection Automation** 윈도우에서는 왼쪽에는 아직 연결되지 않은 Block을 나타내고 해당 핀을 선택하면 오른쪽에 속성을 설정할 수 있음
- **rst_Clk_100M Block**의 **ext_reset_in** 핀이 연결되지 않았음 → **ext_reset_in** 핀을 선택하고 오른쪽에서 속성을 설정
- 보드에 있는 **RESET** 핀은 **Active Low** 이므로, “**New External Port (Active_Low)**”를 선택하고 **OK**를 클릭

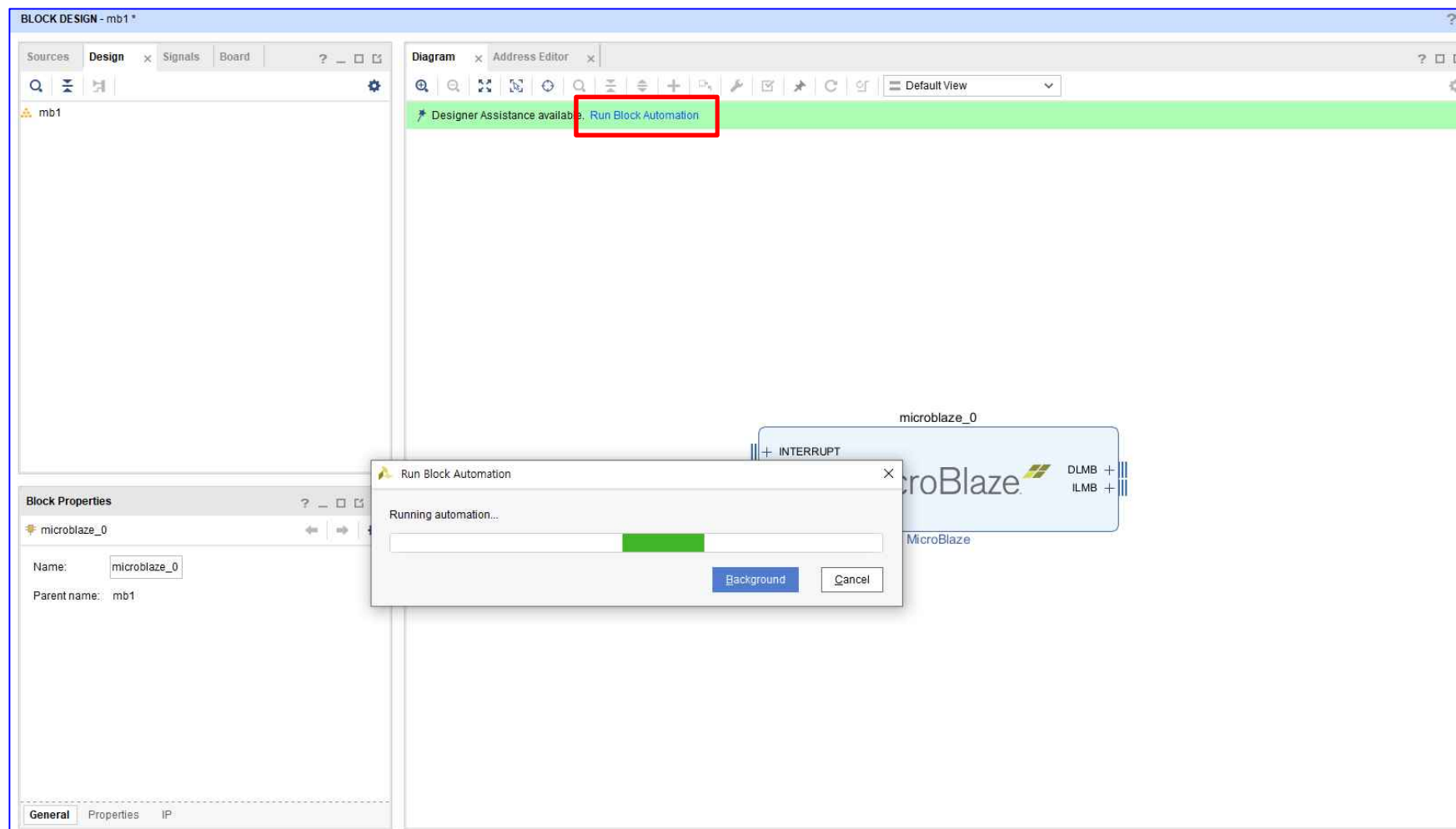


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation

- Run Connection Automation → Block 자동 연결
- IP 추가 및 수정 후 → [Run Block Automation] & [Run Connection Automation] 기능 적용

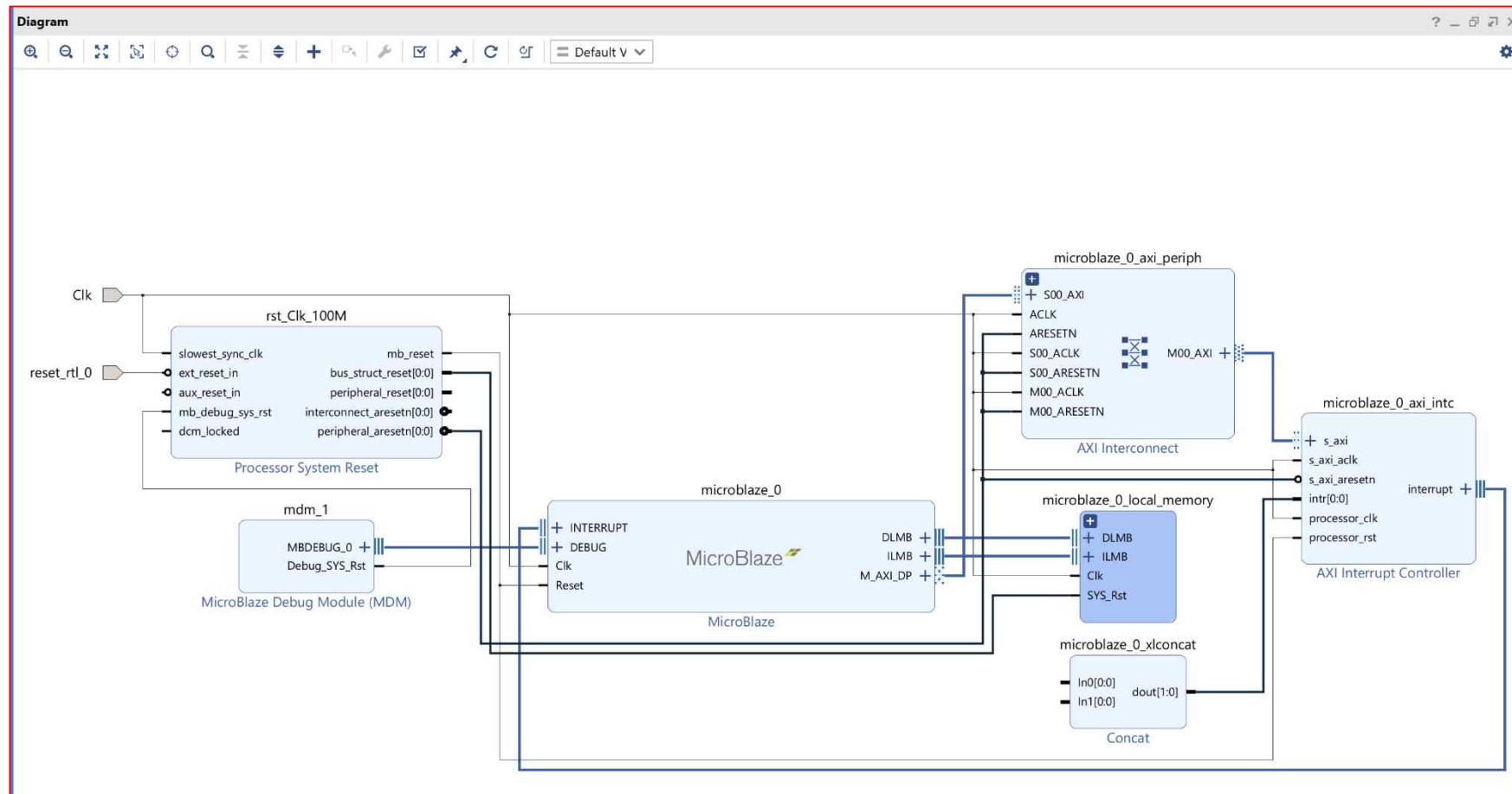


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation

- Run Connection Automation ➔ Block 자동 연결
- [All Automation ➔ OK] 생성된 Block 자동 연결
- 생성 완료 ➔ Clk, reset_rtl_0 포트가 추가됨



LED_Counter Implementation

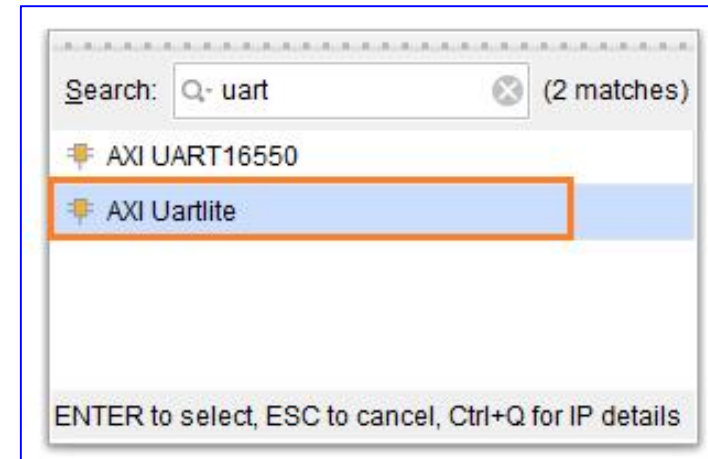
- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*

LED_Counter Implementation

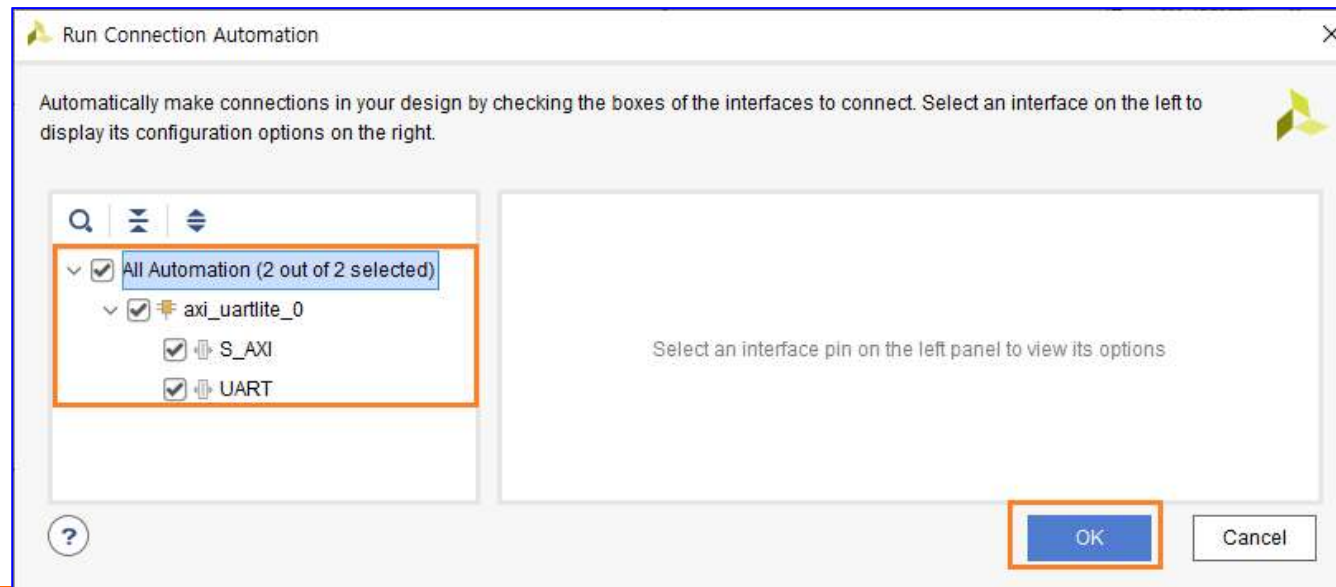
Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation

- Add IP 아이콘 (+)을 클릭해서 *UART Block*을 추가
- *uart* 입력 후 나오는 *IP* 중에 “*AXI Uartlite*”를 *Double Click* 해서 추가



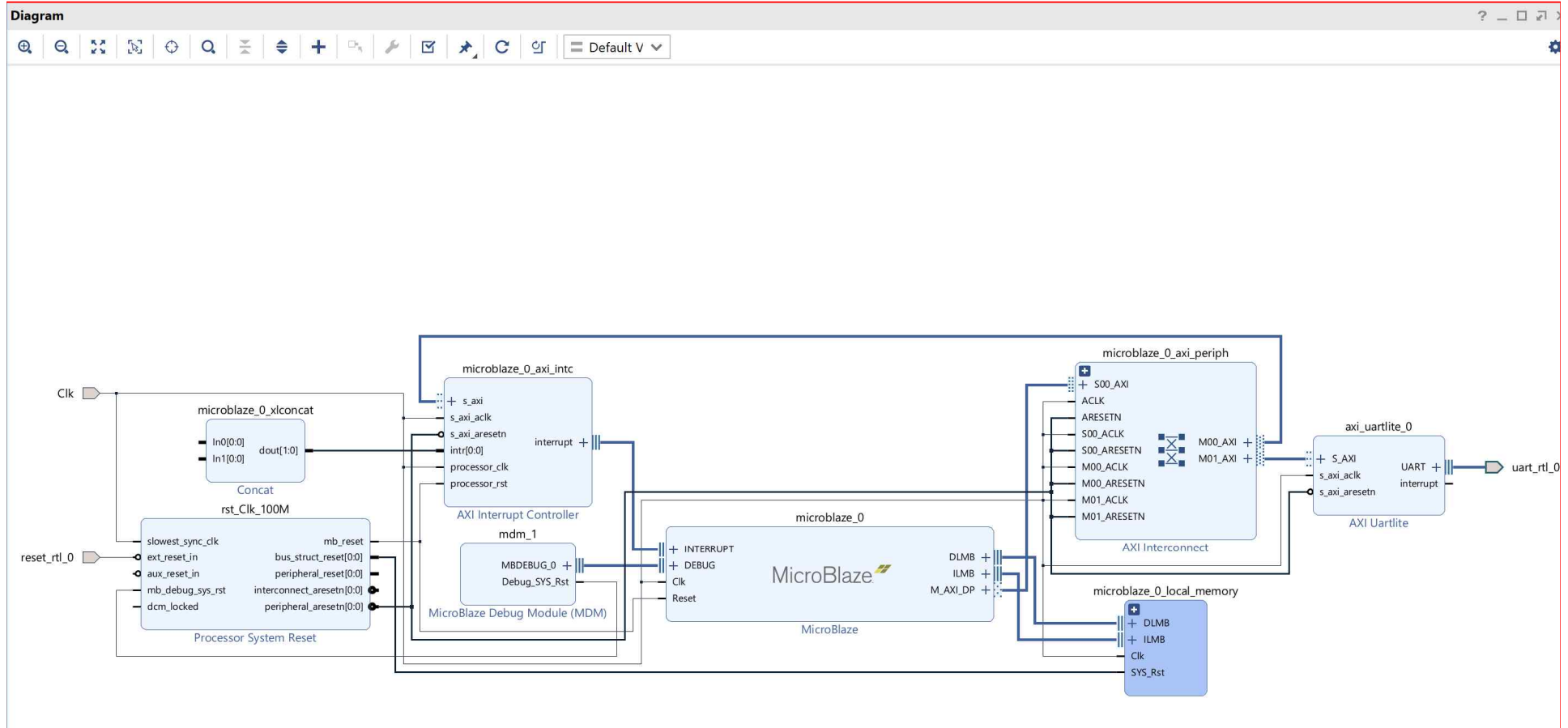
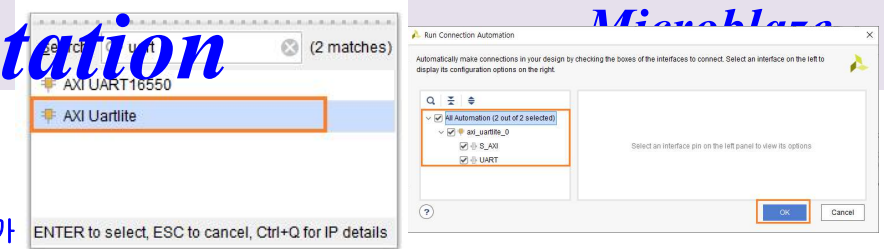
- “*Run Connection Automation(2 out of 2 selected)*”을 클릭해서 연결 ➔ *All Automation*을 선택 해서 모든 *Block*을 선택하고 **OK** 클릭



LED Counter Implementation

➤ LED_Counter Implementation

- Add IP 아이콘 (+)을 클릭해서 UART Block을 추가
- uart 입력 후 나오는 IP 중에 “AXI Uartlite”를 Double Click 해서 추가
- “Run Connection Automation(2 out of 2selected)”을 클릭해서 연결 ➔ All Automation을 선택해서 모든 Block을 선택하고 OK 클릭



LED_Counter Implementation

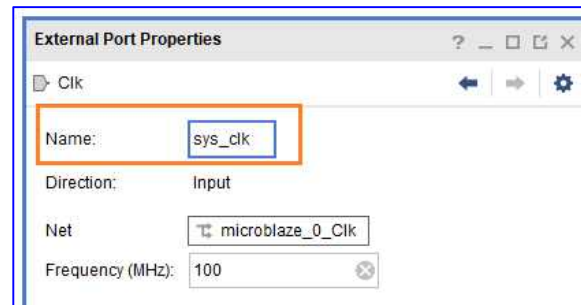
- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*

LED_Counter Implementation

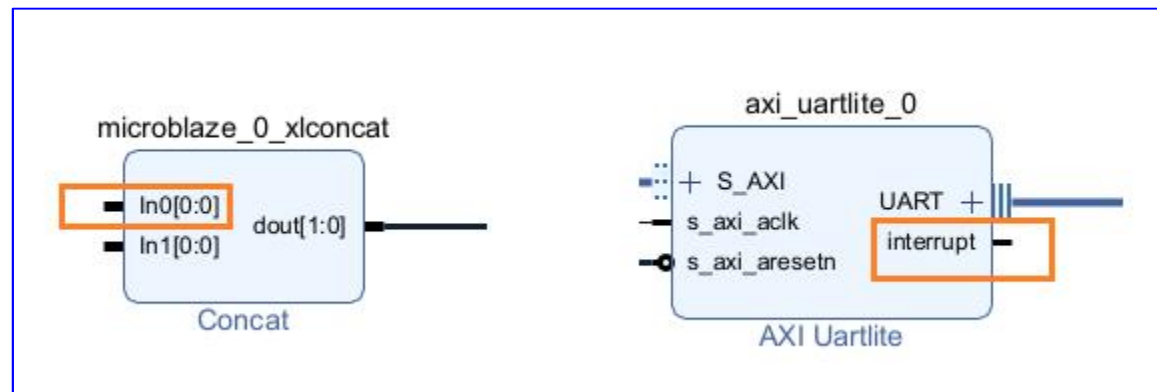
Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation

- Port 이름을 아래와 같이 변경 ➔ 해당 포트를 선택하면 *Diagram* 오른쪽에 해당 핀의 속성이 나타남



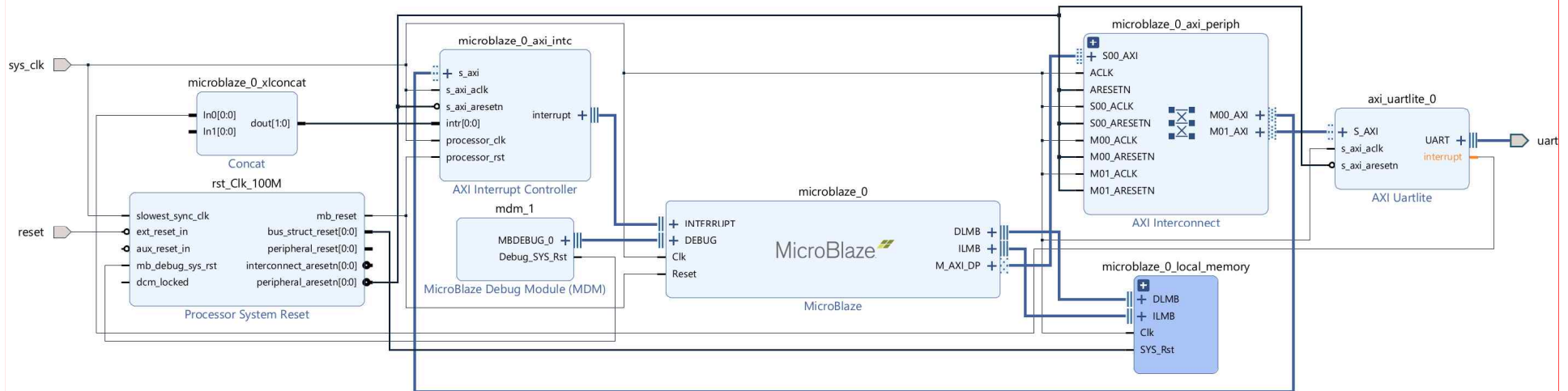
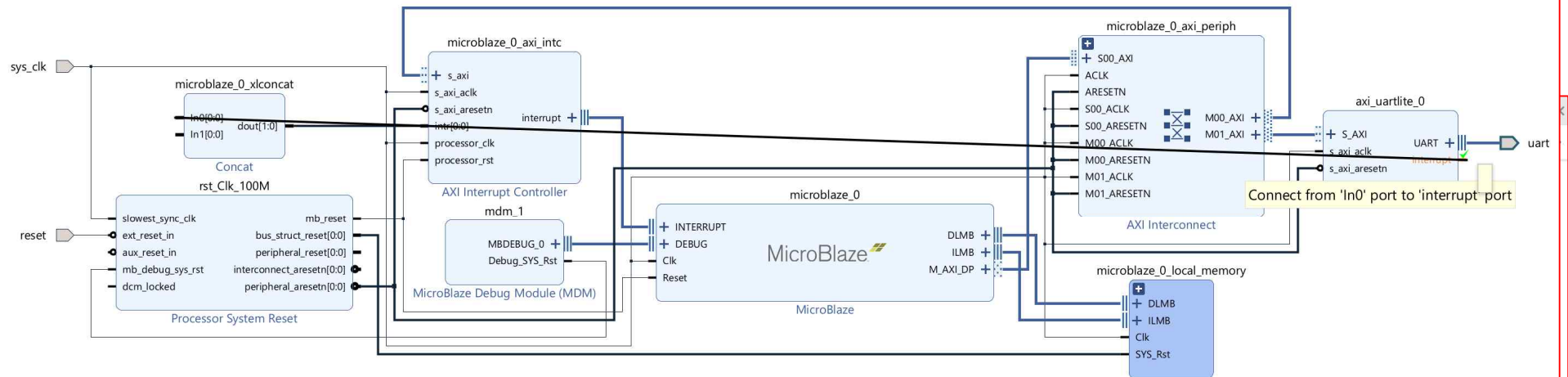
- 3개의 Port 이름을 변경 ➔ [Clk ➔ sys_clk], [reset_rtl_0 ➔ reset], [uart_rtl_0 ➔ uart]
- Interrupt 핀은 사용자가 수동으로 연결해야 함 ➔ “microblaze_0_xlconcat” block의 In0[0:0] 핀과 “axi_uartlite_0” block의 Interrupt 핀을 연결, 해당 핀으로 마우스를 이동으로 연필 모양으로 아이콘이 변경되며, 해당 핀을 클릭하고 드래그 해서 연결할 핀으로 이동하면 연결



LED_Counter Implementation

➤ LED_Counter Implementation

- Port 이름을 아래와 같이 변경 ➔ 해당 포트를 선택하면 Diagram 오른쪽에 해당 핀의 속성이 나타남
- 3개의 Port 이름을 변경 ➔ [Clk ➔ sys_clk], [reset_rtl_0 ➔ reset], [uart_rtl_0 ➔ uart]
- Interrupt 핀은 사용자가 수동으로 연결해야 함 ➔ “microblaze_0_xlconcat” block의 In0[0:0] 핀과 “axi_uartlite_0” block의 Interrupt 핀을 연결, 해당 핀으로 마우스를 이동으로 연결 모양으로 아이콘이 변경되며, 해당 핀을 클릭하고 드래그 해서 연결할 핀으로 이동하면 연결

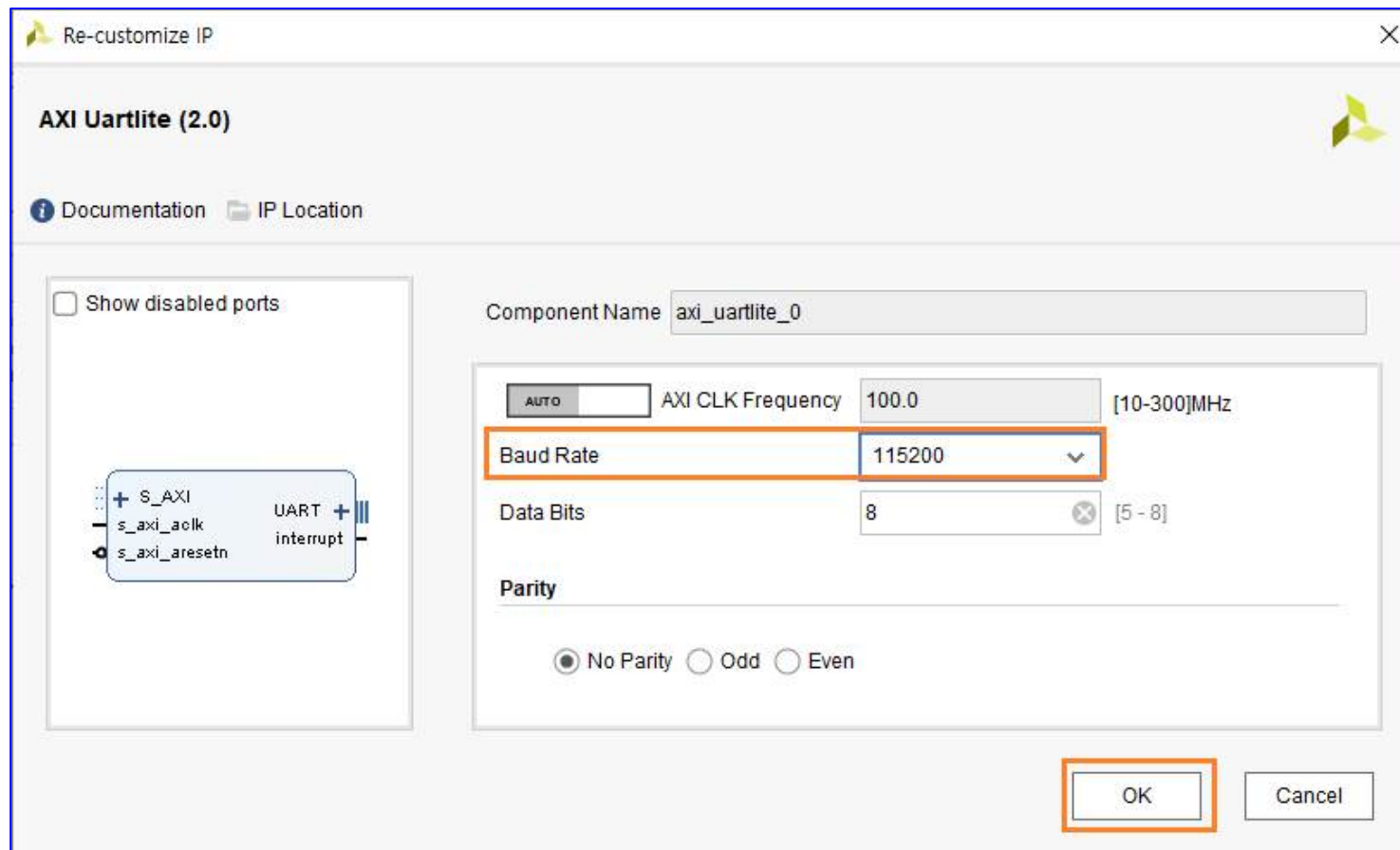


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation

- *uart block*의 *buadrate*를 수정
- *axi_uartlite_0 block*을 더블 클릭하면 속성창이 나타남 ➔ *Baud Rate*을 115200으로 수정하고 *OK*를 클릭



LED_Counter Implementation

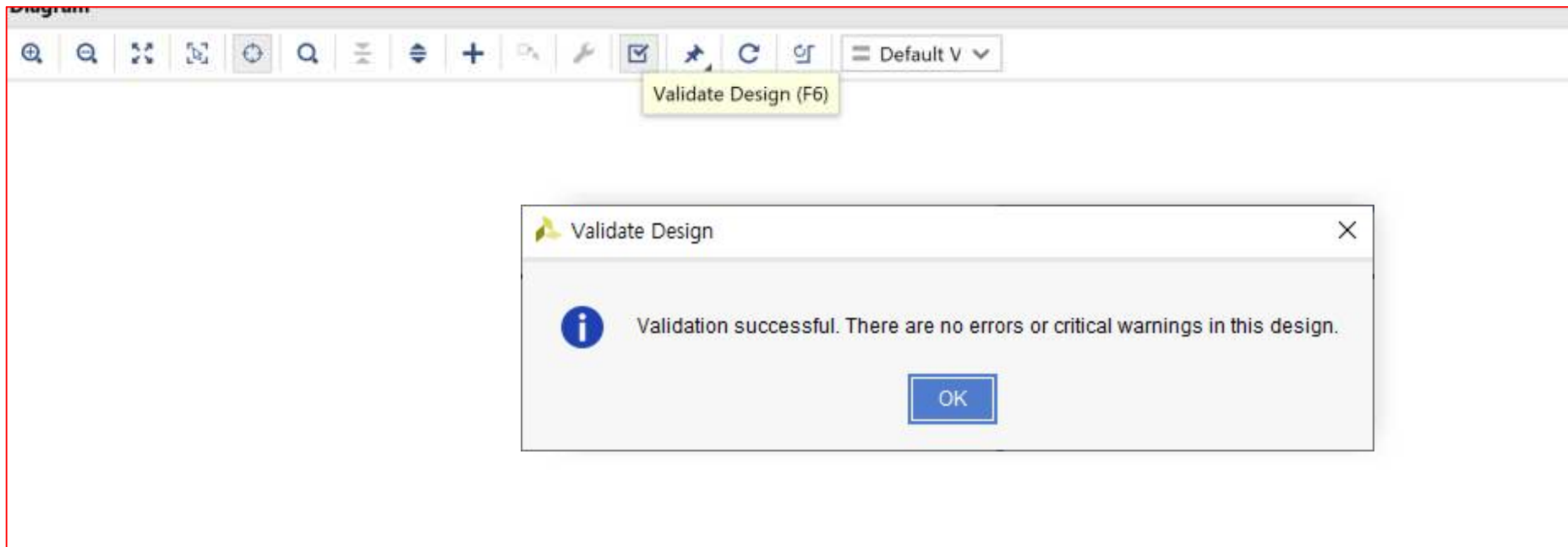
- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation

- Block Design이 완료
- **Validate Design** [단축키: F6]
- Block Design이 완료되면 에러가 없는지 확인 ➔ 상단 아이콘 중에 **Validate Design ()** 클릭
- 에러가 없으면 아래와 같은 윈도우가 나타남
- 에러가 발생하면 관련 설정을 검토하여 에러를 수정하고 다시 에러를 확인

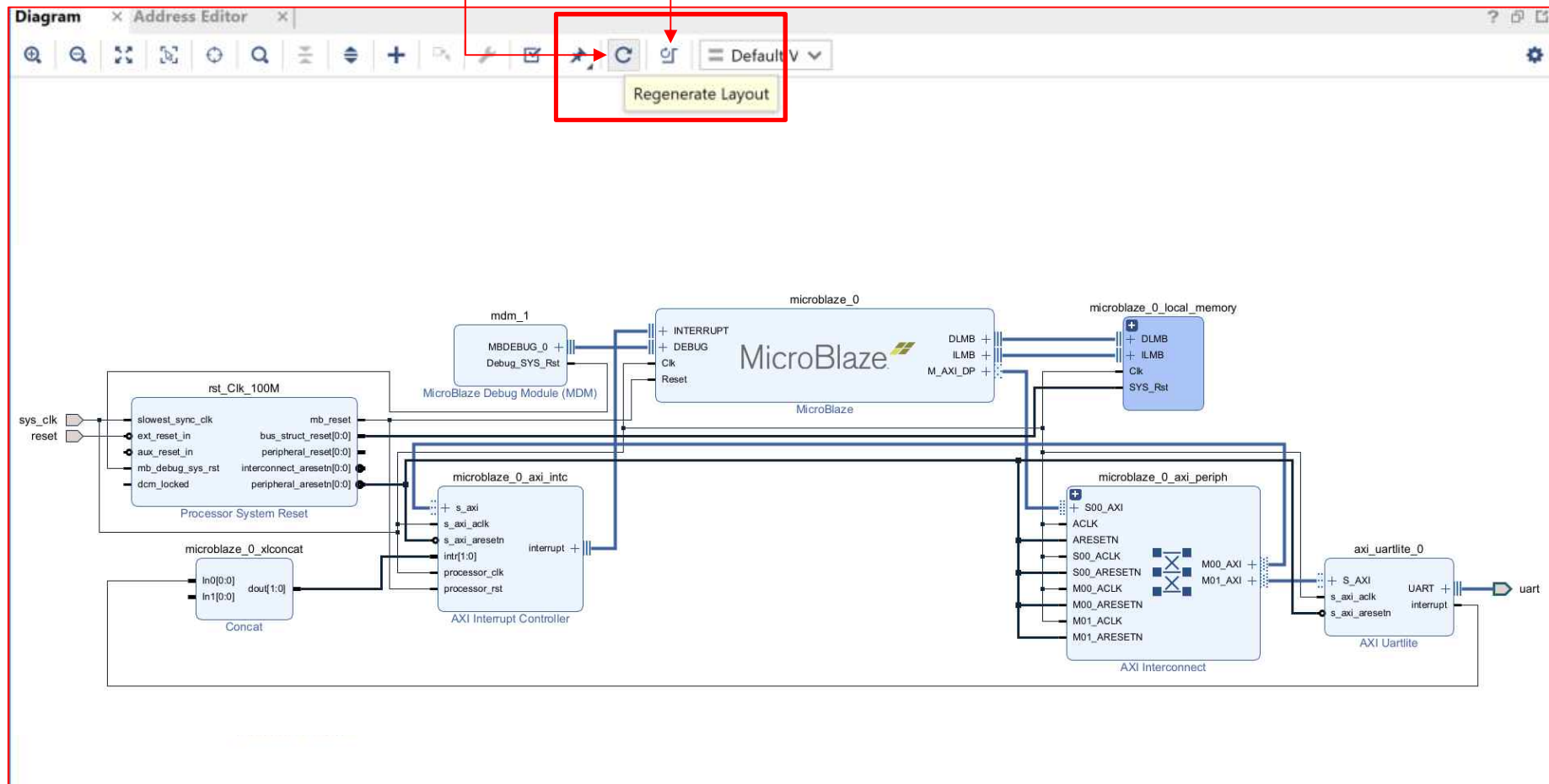


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ LED_Counter Implementation

- 배치 및 정리 ➔ *Regenerate Layout* [or *Optimize Routing*] 클릭



LED_Counter Implementation

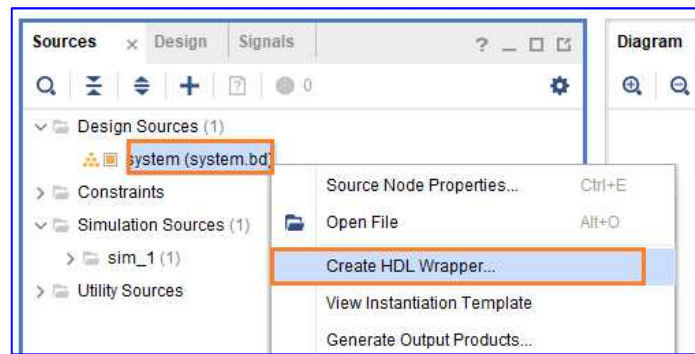
- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*
- *Create HDL Wrapper*

LED_Counter Implementation

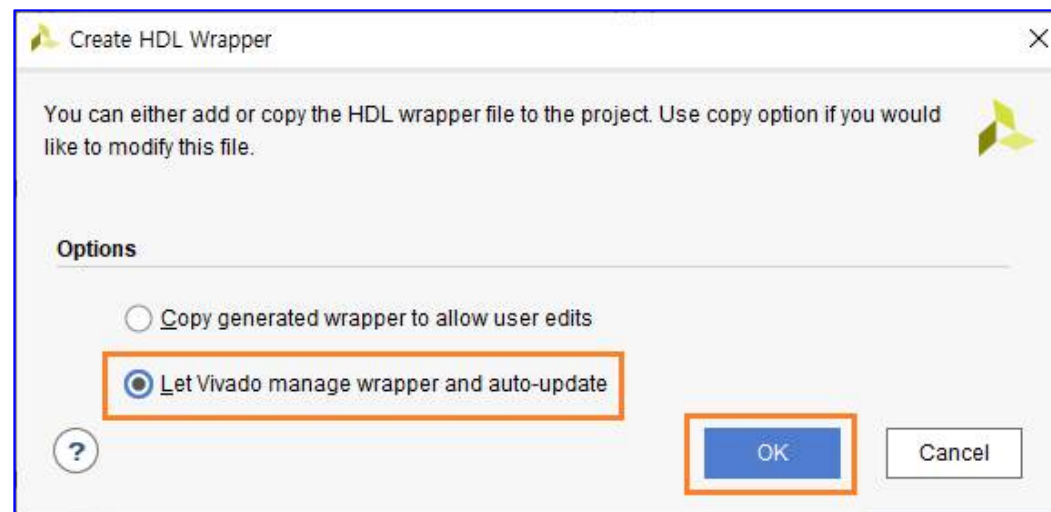
Microblaze
BlazeTop.xpr

➤ Create HDL Wrapper

- Block Design 완료
- 코드로 변환 ➔ [Design Sources – system ➔ 우클릭 ➔ Create HDL Wrapper... 를 클릭]



- “Let Vivado manage wrapper and auto-update” 선택 ➔ OK를 클릭



LED Counter Implementation

Microblaze
BlazeTop.xpr

➤ Create HDL Wrapper

- `system_wrapper.v` 파일이 생성

```
Tcl Console x Messages Log Reports Design Runs
[WARNING: [xilinx.com:ip:axi_intc:4.1-6] /microblaze_0_axi_intc: Property SENSITIVITY = "NULL" for interrupt input 1 not recognized]
make_wrapper -files [get_files D:/ebook/fpga_Microblaze/logic2/ch04/BlazeTop/BlazeTop.srcs/sources_1/bd/system/system.bd] -top
Wrote : <D:/ebook/fpga_Microblaze/logic2/ch04/BlazeTop/BlazeTop.srcs/sources_1/bd/system/ui/bd_c954508f.ui>
Wrote : <D:/ebook/fpga_Microblaze/logic2/ch04/BlazeTop/BlazeTop.gen/sources_1/bd/system/synth/system.v>
Verilog Output written to : D:/ebook/fpga_Microblaze/logic2/ch04/BlazeTop/BlazeTop.gen/sources_1/bd/system/sim/system.v
Verilog Output written to : D:/ebook/fpga_Microblaze/logic2/ch04/BlazeTop/BlazeTop.gen/sources_1/bd/system/hdl/system_wrapper.v
make_wrapper: Time (s): cpu = 00:00:03 ; elapsed = 00:00:07 ; Memory (MB): peak = 1700.488 ; gain = 138.602
add_files -norecurse d:/ebook/fpga_Microblaze/logic2/ch04/BlazeTop/BlazeTop.gen/sources_1/bd/system/hdl/system_wrapper.v
```

- `system_wrapper.v` 파일 ➔ 사용자가 추가한 포트 (`reset`, `sys_clk`, `uart_rxd`, `uart_txd`) 가 있고, 내부에 *Block Design*에서 구현한 *system* 모듈

BLOCK DESIGN - system

Sources x Design Signals ? _ □ □

Design Sources (1)
 system_wrapper (system_wrapper.v) (1)

Constraints
Simulation Sources (1)
Utility Sources

Hierarchy IP Sources Libraries Compile Order

Source File Properties

system_wrapper.v

Enabled

Location: f:/Xilinx/2022/BlazeTop/BlazeTop.gen/sources_1/bd/sy

Type: Verilog ...

Library: xil_defaultlib ...

General Properties

Address Editor x Address Map x Diagram x system_wrapper.v x

f:/Xilinx/2022/BlazeTop/BlazeTop.gen/sources_1/bd/system/hdl/system_wrapper.v

```
10 `timescale 1 ps / 1 ps
11
12 module system_wrapper
13     (reset,
14      sys_clk,
15      uart_rxd,
16      uart_txd);
17     input reset;
18     input sys_clk;
19     input uart_rxd;
20     output uart_txd;
21
22     wire reset;
23     wire sys_clk;
24     wire uart_rxd;
25     wire uart_txd;
26
27     system system_i
28         (.reset(reset),
29          .sys_clk(sys_clk),
30          .uart_rxd(uart_rxd),
31          .uart_txd(uart_txd));
32 endmodule
```

Tcl Console x Messages Log Reports Design Runs

```
Verilog Output written to : f:/Xilinx/2022/BlazeTop/BlazeTop.gen/sources_1/bd/system/synth/system.v
Verilog Output written to : f:/Xilinx/2022/BlazeTop/BlazeTop.gen/sources_1/bd/system/sim/system.v
Verilog Output written to : f:/Xilinx/2022/BlazeTop/BlazeTop.gen/sources_1/bd/system/hdl/system_wrapper.v
make_wrapper: Time (s): cpu = 00:00:05 ; elapsed = 00:00:09 ; Memory (MB): peak = 2525.066 ; gain = 177.516
add_files -norecurse f:/Xilinx/2022/BlazeTop/BlazeTop.gen/sources_1/bd/system/hdl/system_wrapper.v
```

LED_Counter Implementation

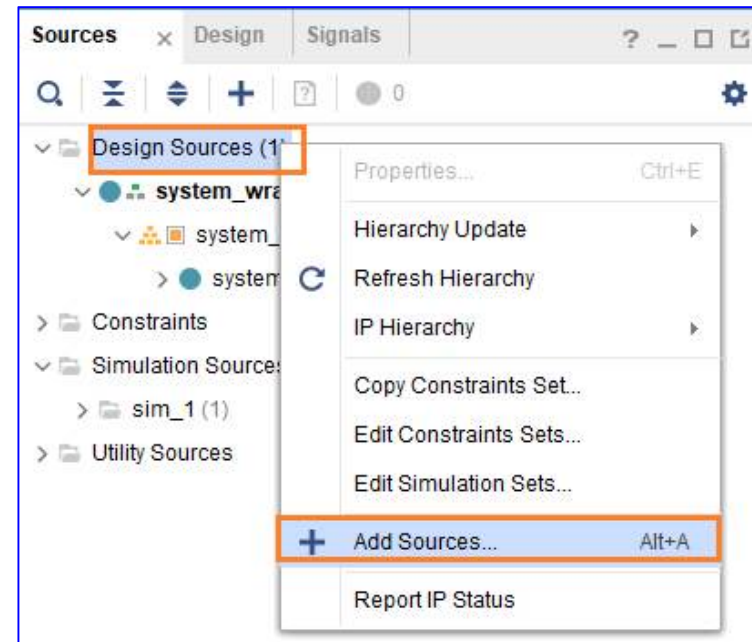
- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*
- *Create HDL Wrapper*
- *User Logic Implementation → Led_On/Off_Logic → led_control*

LED_Counter Implementation

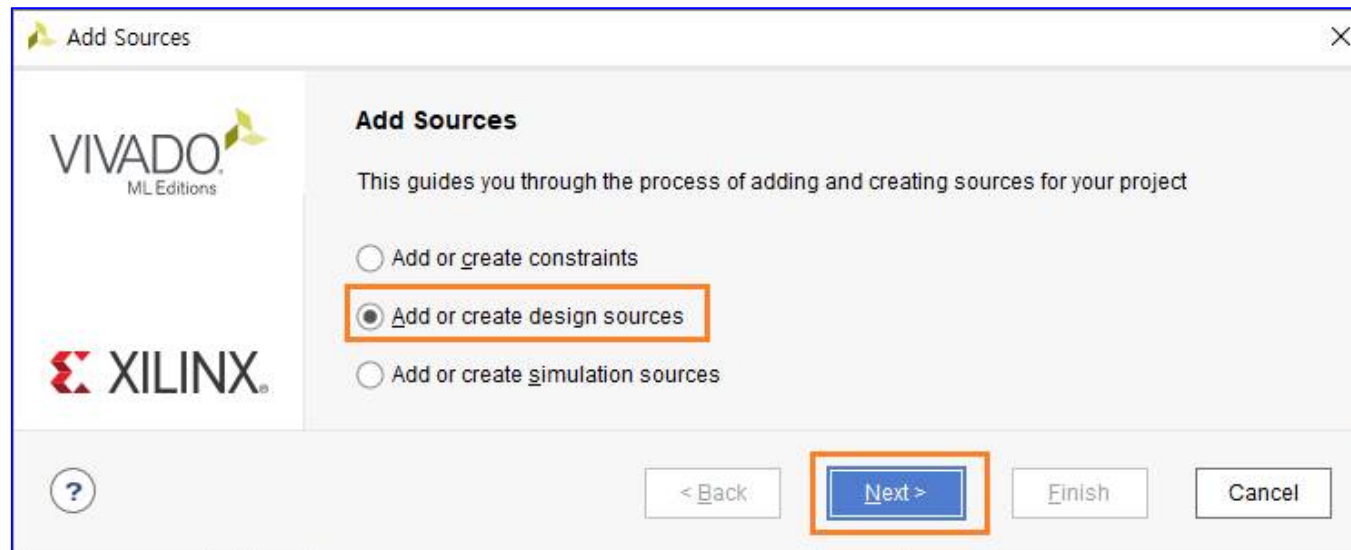
Microblaze
BlazeTop.xpr

➤ User Logic Implementation

- Led를 On/Off 하는 Logic 구현
- Design Sources → 우클릭 → Add Sources... 를 클릭



- “Add or create design sources”을 선택하고 Next를 클릭

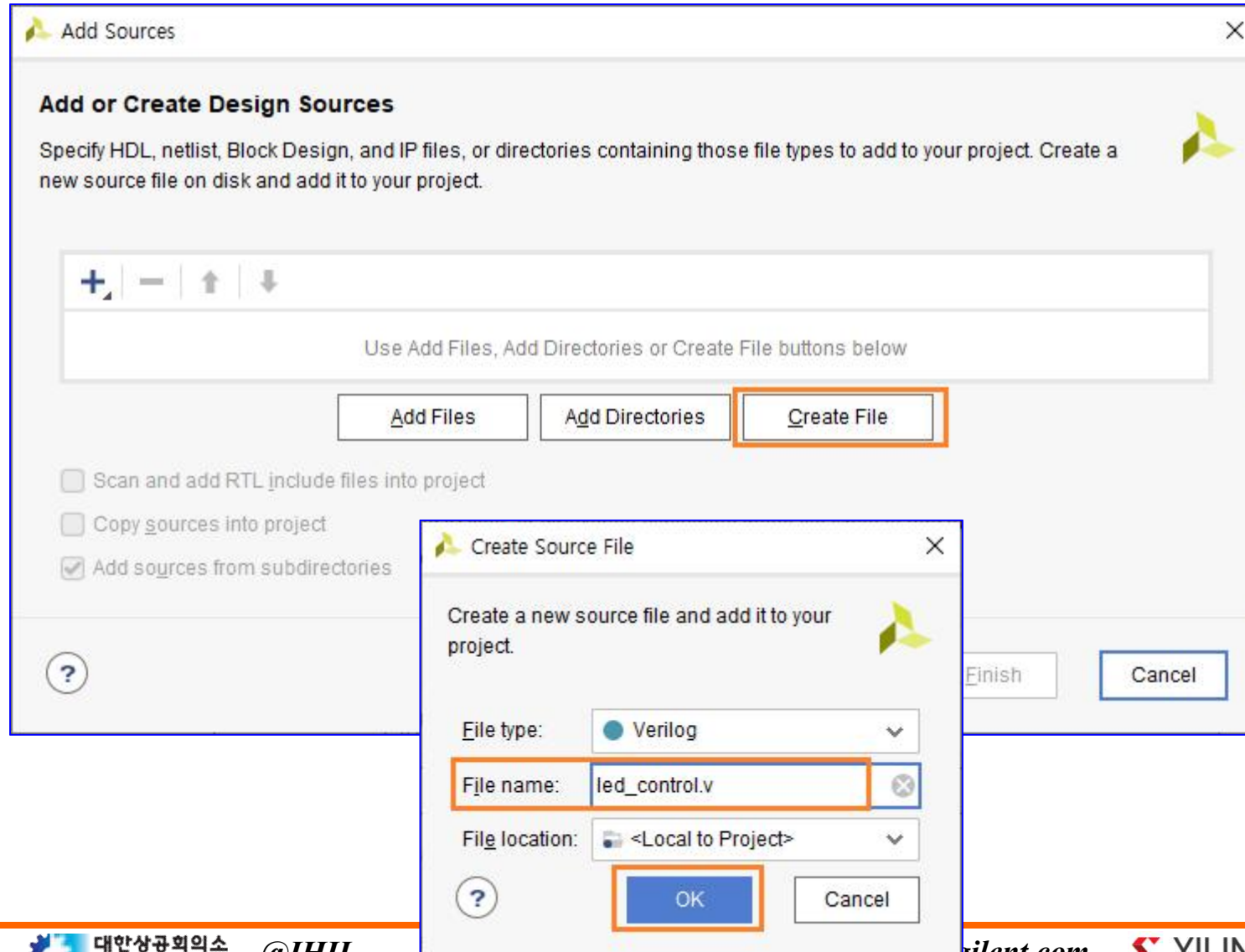


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ User Logic Implementation

- Create File 클릭 ➔ Create Source File 윈도우 ➔ File name : led_control.v 입력 ➔ OK 클릭

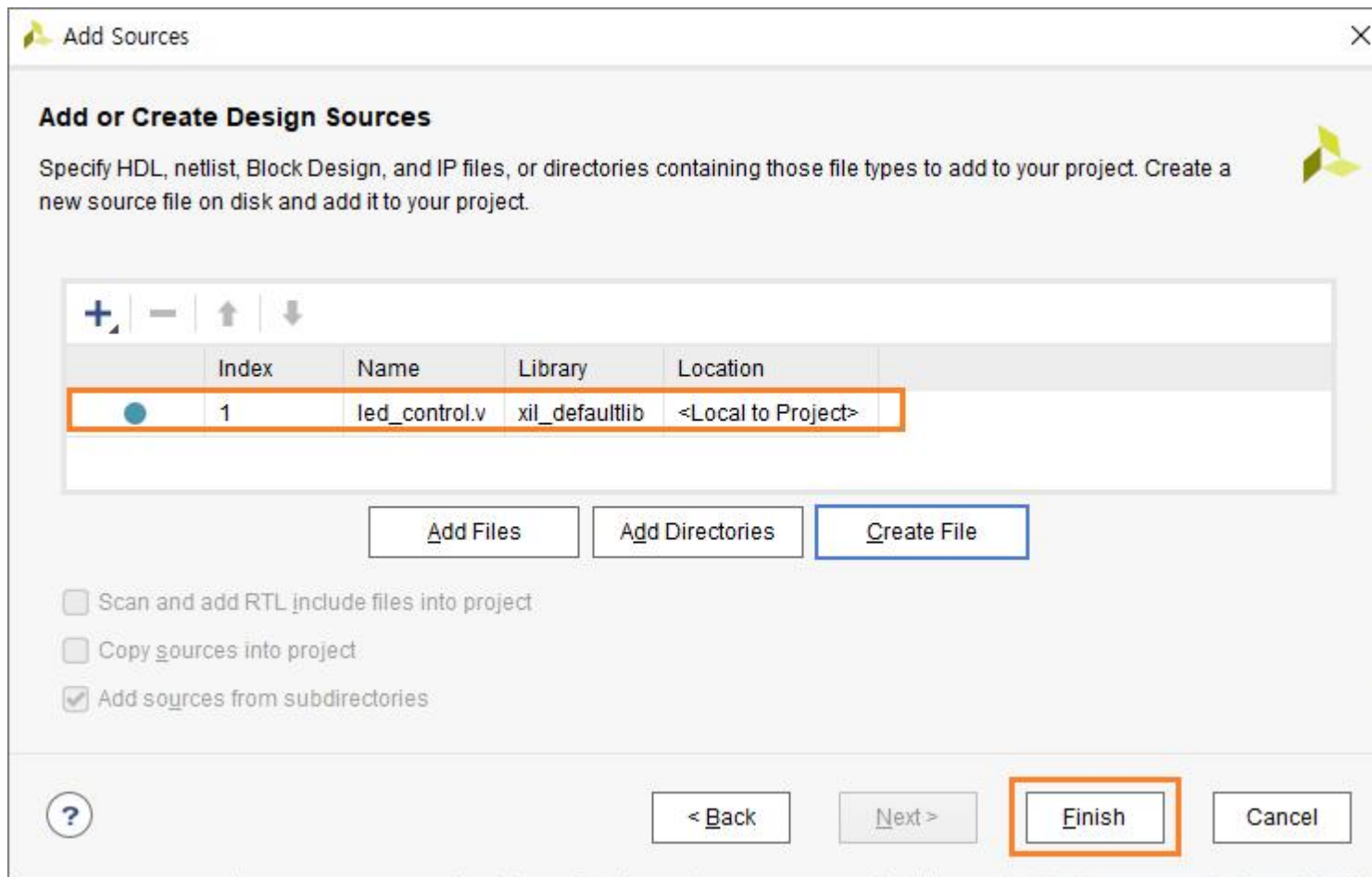


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ User Logic Implementation

- `led_control.v` 파일 추가 ➔ **Finish**를 클릭합니다. (이미 생성된 소스 파일을 프로젝트에 추가하려면 **Add Files** 버튼을 클릭하여 해당 파일을 추가)

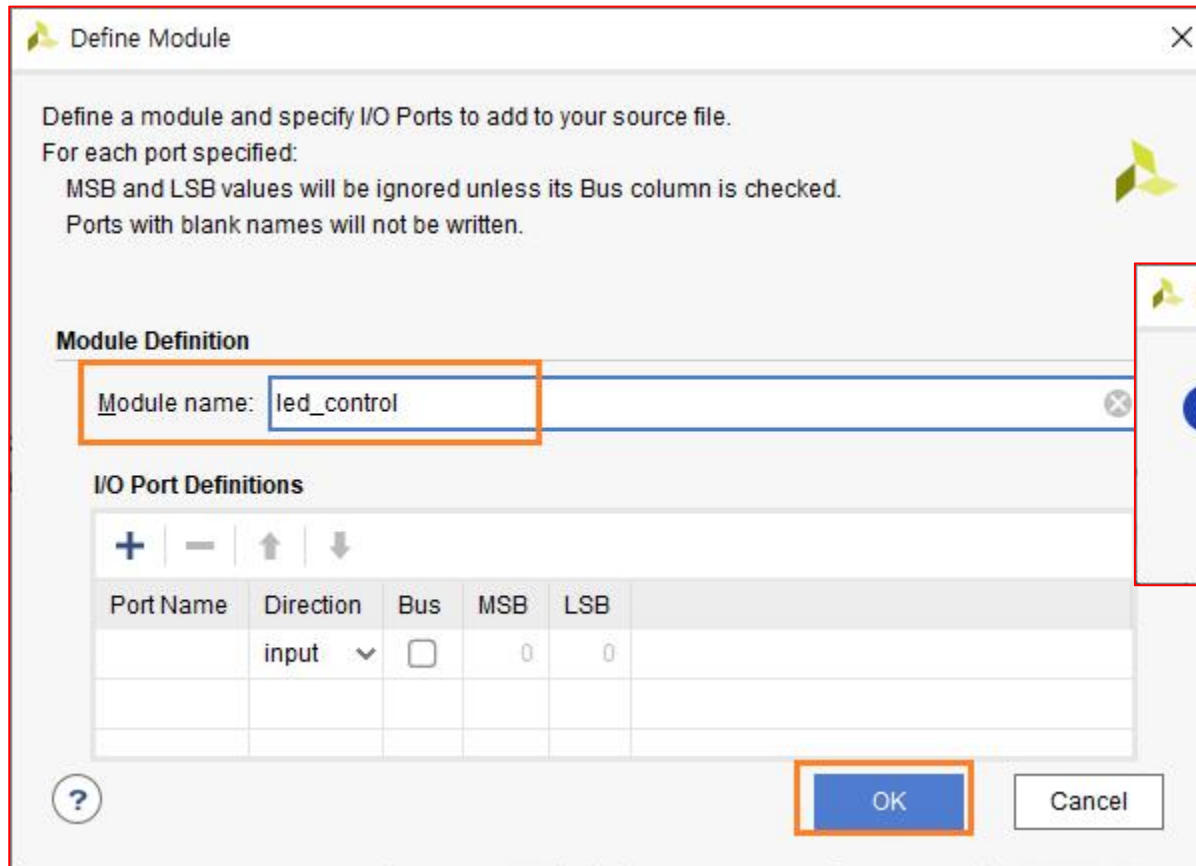


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ User Logic Implementation

- Define Module 윈도우에서는 모듈 이름과 핀들을 설정 ➔ 기본값을 사용, OK 클릭 ➔ YES 클릭



Define Module

Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

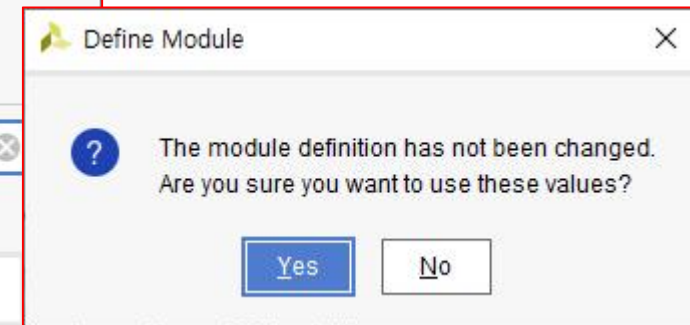
Module Definition

Module name: led_control

I/O Port Definitions

Port Name	Direction	Bus	MSB	LSB
	input	☐	0	0

OK Cancel



Define Module

The module definition has not been changed.
Are you sure you want to use these values?

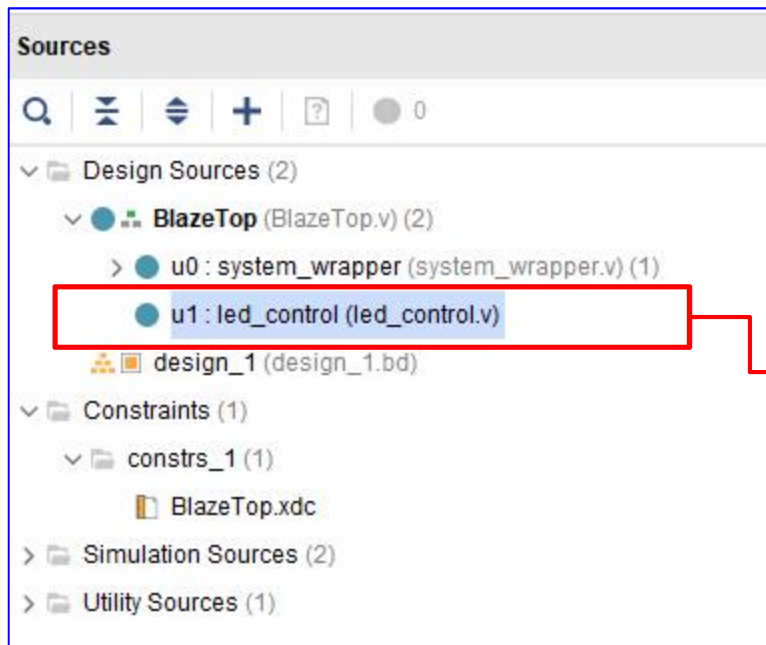
Yes No

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ User Logic Implementation

- *led_control module* 이 추가된 *Design Source*



- *led_control.v* 파일을 열어 *Code* 추가
- *100Mhz Clock*을 이용하여 *0.5초마다 4개의 LED*를 순차적으로 On/Off 하는 코드

```
Project Summary x led_control.v x
H:/Xilinx/2022/BlazeTop/BlazeTop.srcs/sources_1/new/led_control.v

1  `timescale 1ns / 1ps
2
3  module led_control(
4      reset,
5      clock,
6      led
7  );
8
9      input  reset;
10     input  clock;
11     output [3:0] led ;
12
13     reg    [25:0] cnt ;
14
15     always @ (posedge clock or negedge reset)
16     begin
17         if(~reset) cnt <= 26'd0;
18         else      cnt <= (cnt==26'd50_000_000) ? 26'd0 : cnt+1'b1;
19     end
20
21     reg [3:0] led;
22
23     always@(posedge clock or negedge reset)
24     begin
25         if(~reset) led <= 4'd0;
26         else      led <= (cnt==26'd50_000_000) ? led+1'b1 : led;
27     end
28
29     end
30 endmodule
31
```

LED_Counter Implementation

- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*
- *Create HDL Wrapper*
- *User Logic Implementation → Led_On/Off_Logic → led_control*
- *User Logic Implementation → Led_On/Off_Logic → **BlazeTop***

LED_Counter Implementation

➤ User Logic Implementation

- Design Source에 **led_counter, system_wrapper** 모듈을 포함하는 **Top** 모듈을 추가
- **led_control** 모듈을 추가할 때와 동일한 방법으로 **BlazeTop.v** 모듈을 추가하고, 아래와 같이 코드를 추가

```
Address Editor x Address Map x Diagram x system_wrapper.v x
f:/Xilinx/2022/BlazeTop/BlazeTop.gen/sources_1/bd/system/hdl/system_wrapper.v

10 `timescale 1 ps / 1 ps
11
12 module system_wrapper
13 (
14     reset,
15     sys_clk,
16     uart_rxd,
17     uart_txd);
18 input reset;
19 input sys_clk;
20 input uart_rxd;
21 output uart_txd;
22
23 wire reset;
24 wire sys_clk;
25 wire uart_rxd;
26 wire uart_txd;
27
28 system system_i
29 (.reset(reset),
30  .sys_clk(sys_clk),
31  .uart_rxd(uart_rxd),
32  .uart_txd(uart_txd));
33 endmodule
```

```
Project Summary x led_control.v x
H:/Xilinx/2022/BlazeTop/BlazeTop.srcs/sources_1/new/led_control.v

1 `timescale 1ns / 1ps
2
3 module led_control(
4     reset,
5     clock,
6     led
7 );
8
9 input reset;
10 input clock;
11 output [3:0] led;
12
13 reg [25:0] cnt;
14
15 always @ (posedge clock or negedge reset)
16 begin
17     if(~reset) cnt <= 26'd0;
18     else cnt <= (cnt==26'd50_000_000) ? 26'd0 : cnt+1'b1;
19 end
20
21
22 reg [3:0] led;
23
24 always@(posedge clock or negedge reset)
25 begin
26     if(~reset) led <= 4'd0;
27     else led <= (cnt==26'd50_000_000) ? led+1'b1 : led;
28 end
29 endmodule
30
31
```

```
Project Summary x led_control.v x BlazeTop.v x
H:/Xilinx/2022/BlazeTop/BlazeTop.srcs/sources_1/new/BlazeTop.v

1 `timescale 1ns / 1ps
2
3 module BlazeTop(
4     reset,
5     sys_clk,
6     uart_rxd,
7     uart_txd,
8     led
9 );
10
11 input reset;
12 input sys_clk;
13 input uart_rxd;
14 output uart_txd;
15 output [3:0] led;
16
17 system_wrapper u0(
18     .reset(reset),
19     .sys_clk(sys_clk),
20     .uart_rxd(uart_rxd),
21     .uart_txd(uart_txd)
22 );
23
24 wire [3:0] led;
25
26 led_control u1(
27     .reset(reset),
28     .clock(sys_clk),
29     .led(led)
30 );
31 endmodule
```

LED_Counter Implementation

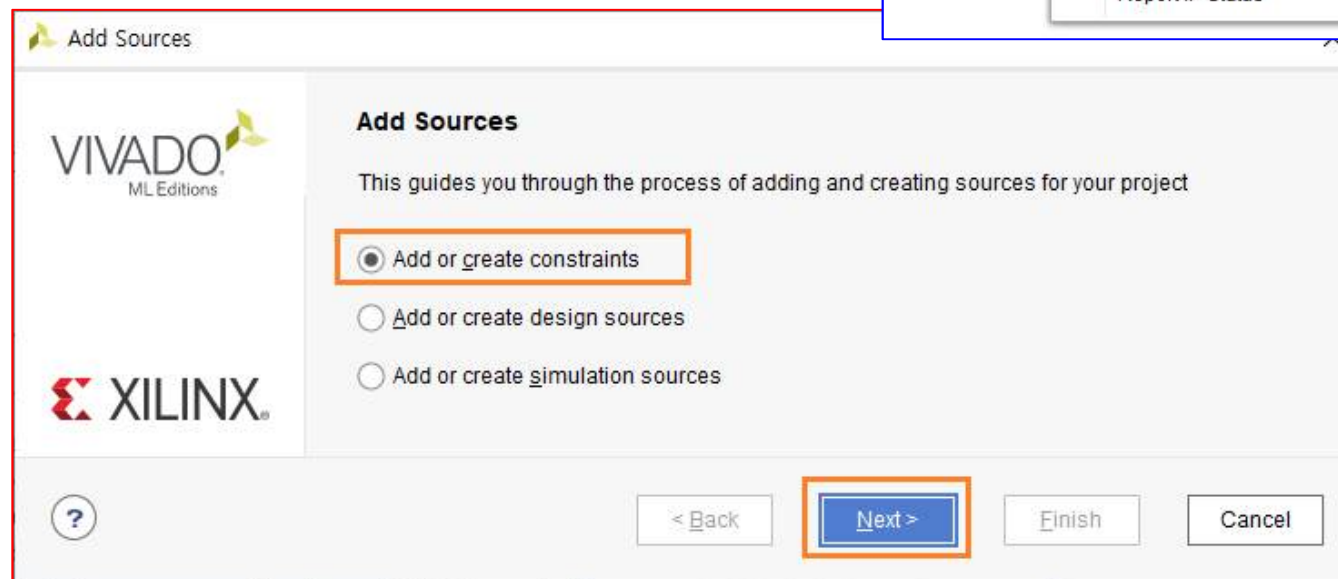
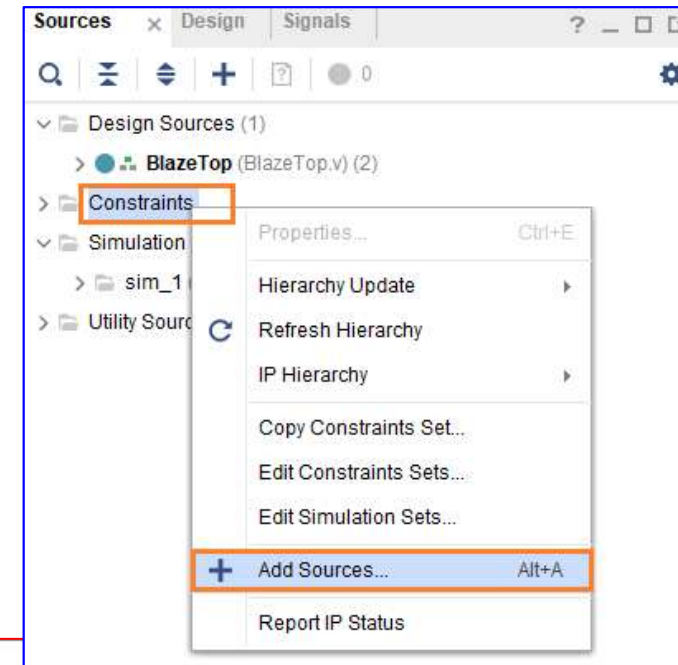
- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*
- *Create HDL Wrapper*
- *User Logic Implementation → Led_On/Off_Logic → led_control*
- *User Logic Implementation → Led_On/Off_Logic → BlazeTop*
- ***XDC Implementation***

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ XDC Implementation

- *Constraints* → 우클릭 → *Add Sources...*를 클릭
- “*Add or Create constraints*”을 선택하고 *Next*를 클릭
- *File name : BlazeTop.xdc*로 입력해서 파일을 추가



LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ XDC Implementation

- **BlazeTop.xdc** 에 아래와 같이 코드를 추가(1/2)

```
BlazeTop.xdc
F:/Xilinx/2022/BlazeTop/BlazeTop.srcs/constrs_1/new/BlazeTop.xdc

1  ## This file is a general .xdc for the Basys3 rev B board
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project
5
6  ## Clock signal
7  set_property -dict { PACKAGE_PIN W5  IOSTANDARD LVCMOS33 } [get_ports sys_clk]
8  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports sys_clk]
9
10 ## Switches
11 #set_property -dict { PACKAGE_PIN V17  IOSTANDARD LVCMOS33 } [get_ports {sw[0]}]
12 #set_property -dict { PACKAGE_PIN V16  IOSTANDARD LVCMOS33 } [get_ports {sw[1]}]
13 #set_property -dict { PACKAGE_PIN W16  IOSTANDARD LVCMOS33 } [get_ports {sw[2]}]
14 #set_property -dict { PACKAGE_PIN W17  IOSTANDARD LVCMOS33 } [get_ports {sw[3]}]
15 #set_property -dict { PACKAGE_PIN W15  IOSTANDARD LVCMOS33 } [get_ports {sw[4]}]
16 #set_property -dict { PACKAGE_PIN V15  IOSTANDARD LVCMOS33 } [get_ports {sw[5]}]
17 #set_property -dict { PACKAGE_PIN W14  IOSTANDARD LVCMOS33 } [get_ports {sw[6]}]
18 #set_property -dict { PACKAGE_PIN W13  IOSTANDARD LVCMOS33 } [get_ports {sw[7]}]
19 #set_property -dict { PACKAGE_PIN V2   IOSTANDARD LVCMOS33 } [get_ports {sw[8]}]
20 #set_property -dict { PACKAGE_PIN T3   IOSTANDARD LVCMOS33 } [get_ports {sw[9]}]
21 #set_property -dict { PACKAGE_PIN T2   IOSTANDARD LVCMOS33 } [get_ports {sw[10]}]
22 #set_property -dict { PACKAGE_PIN R3   IOSTANDARD LVCMOS33 } [get_ports {sw[11]}]
23 #set_property -dict { PACKAGE_PIN W2   IOSTANDARD LVCMOS33 } [get_ports {sw[12]}]
24 #set_property -dict { PACKAGE_PIN U1   IOSTANDARD LVCMOS33 } [get_ports {sw[13]}]
25 #set_property -dict { PACKAGE_PIN T1   IOSTANDARD LVCMOS33 } [get_ports {sw[14]}]
26 set_property -dict { PACKAGE_PIN R2   IOSTANDARD LVCMOS33 } [get_ports {reset}]
27
```


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ XDC Implementation

- *BlazeTop.xdc* 에 아래와 같이 코드를 추가(2/2)

```
28
29 ## LEDs
30 set_property -dict { PACKAGE_PIN U16  IOSTANDARD LVCMOS33 } [get_ports {led[0]}]
31 set_property -dict { PACKAGE_PIN E19  IOSTANDARD LVCMOS33 } [get_ports {led[1]}]
32 set_property -dict { PACKAGE_PIN U19  IOSTANDARD LVCMOS33 } [get_ports {led[2]}]
33 set_property -dict { PACKAGE_PIN V19  IOSTANDARD LVCMOS33 } [get_ports {led[3]}]
34 #set_property -dict { PACKAGE_PIN W18  IOSTANDARD LVCMOS33 } [get_ports {led[4]}]
35 #set_property -dict { PACKAGE_PIN U15  IOSTANDARD LVCMOS33 } [get_ports {led[5]}]
36 #set_property -dict { PACKAGE_PIN U14  IOSTANDARD LVCMOS33 } [get_ports {led[6]}]
37 #set_property -dict { PACKAGE_PIN V14  IOSTANDARD LVCMOS33 } [get_ports {led[7]}]
38 #set_property -dict { PACKAGE_PIN V13  IOSTANDARD LVCMOS33 } [get_ports {led[8]}]
```

```
130
131 ##USB-RS232 Interface
132 set_property -dict { PACKAGE_PIN B18  IOSTANDARD LVCMOS33 } [get_ports uart_rxd]
133 set_property -dict { PACKAGE_PIN A18  IOSTANDARD LVCMOS33 } [get_ports uart_txd]
134
```

```
150
151 ## Configuration options, can be used for all designs
152 set_property CONFIG_VOLTAGE 3.3 [current_design]
153 set_property CFGBVS VCCO [current_design]
154
155 ## SPI configuration mode options for QSPI boot, can be used for all designs
156 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
157 set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
158 set_property CONFIG_MODE SPIx4 [current_design]
159
```

LED_Counter Implementation

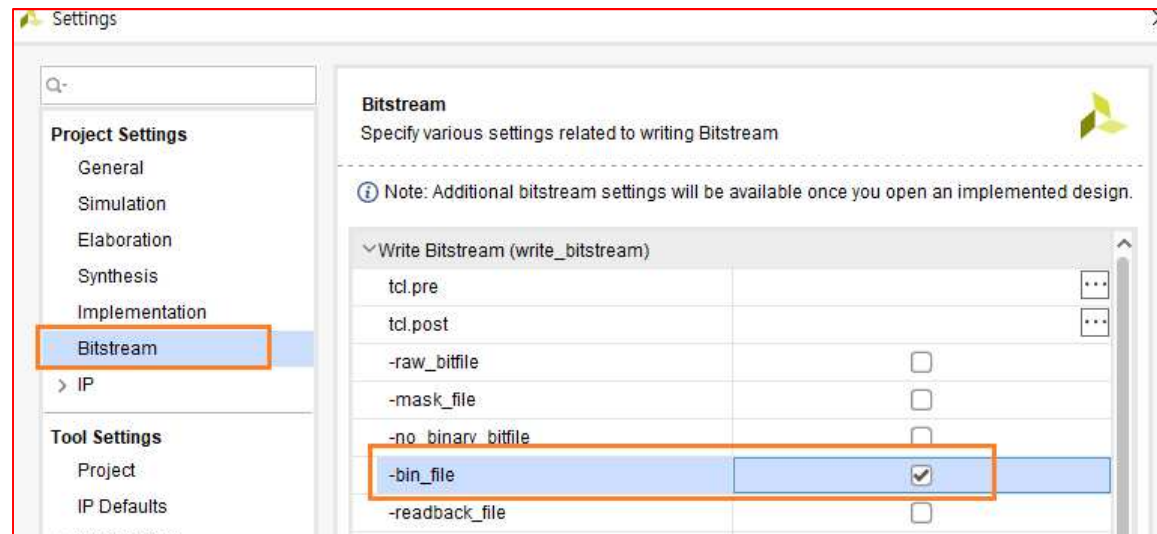
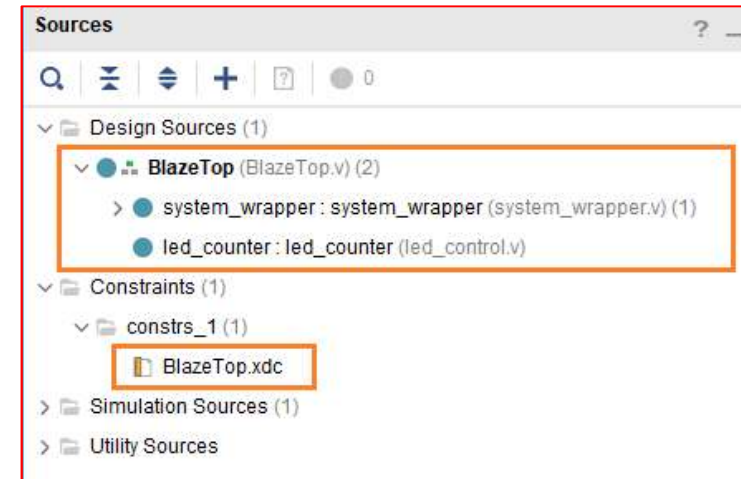
- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*
- *Create HDL Wrapper*
- *User Logic Implementation → Led_On/Off_Logic → led_control*
- *User Logic Implementation → Led_On/Off_Logic → BlazeTop*
- *XDC Implementation*
- ***Generate Bitstream***

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ Generate Bitstream

- 구현된 Source 구조 ➔ *BlazeTop* 모듈이 Top 모듈로 설정
- *Project Settings* 에서 *Bitstream* 생성시 *bin*파일을 생성하도록 설정 ➔ *PROJECT MANAGER* ➔ *Settings* 을 클릭 ➔ *Settings* 윈도우에서 *Bitstream*을 클릭하고 [*-bin_file*]항목을 check 하고 *Apply* ➔ *OK*를 클릭



- *PROMGRAM AND DEBUG* – *Generate Bitstream*을 클릭해서 *Bitstream*을 생성
- 완료 되면 *Cancel* 버튼을 클릭

LED_Counter Implementation

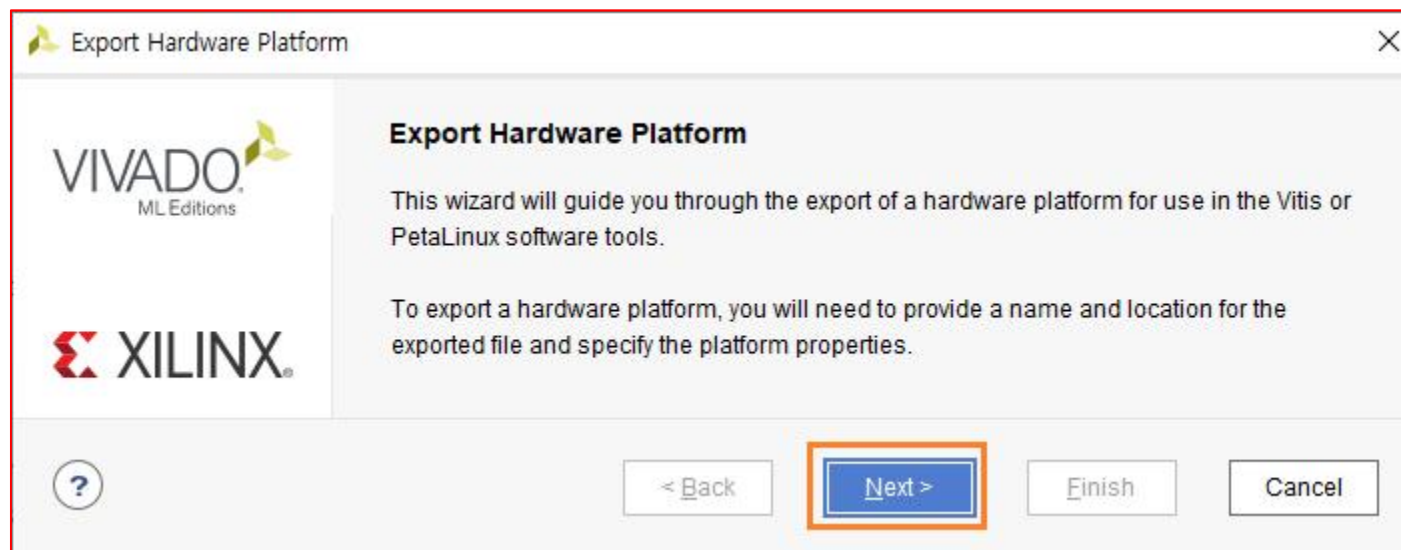
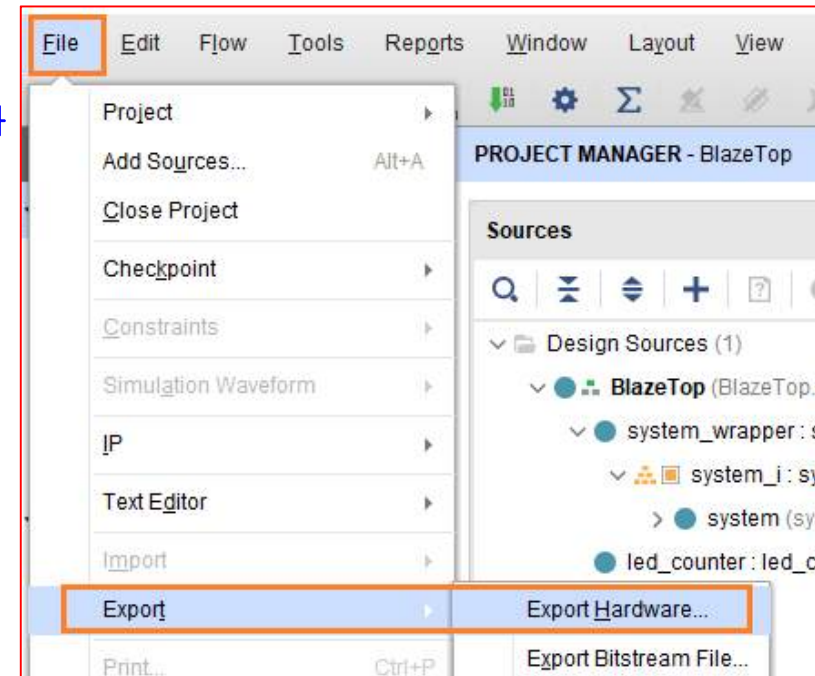
- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*
- *Create HDL Wrapper*
- *User Logic Implementation → Led_On/Off_Logic → led_control*
- *User Logic Implementation → Led_On/Off_Logic → BlazeTop*
- *XDC Implementation*
- *Generate Bitstream*
- *Export Hardware*

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ Export Hardware

- 생성된 로직을 Vitis (Embedded sw 개발 tool)에서 사용하기 위해서는 Export Hardware
- 메뉴에서 [File -> Export -> Export Hardware]를 클릭
- Next를 클릭

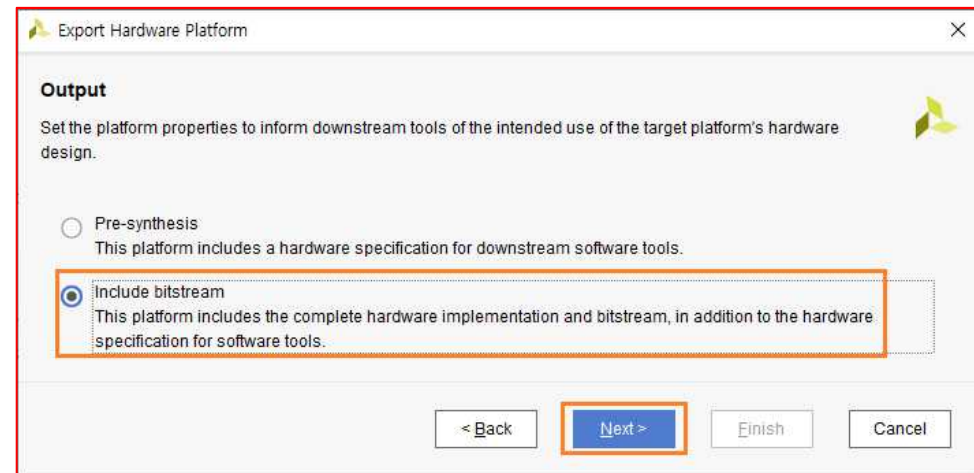


LED_Counter Implementation

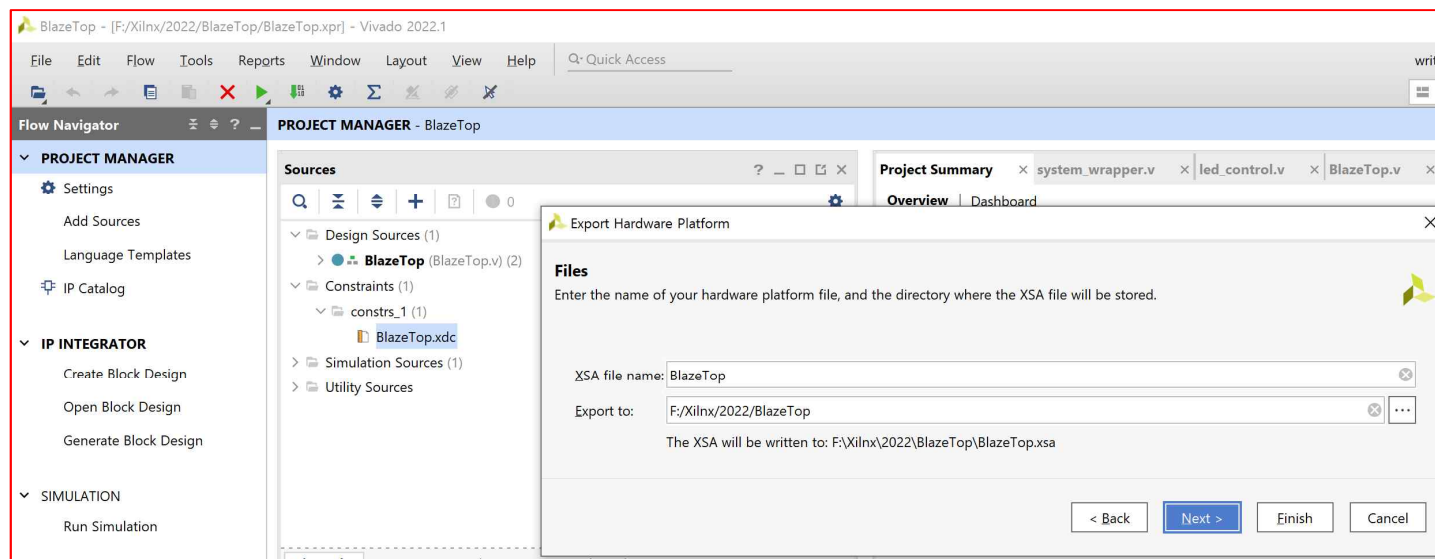
Microblaze
BlazeTop.xpr

➤ Export Hardware

- “Include bitstream”을 선택하고 Next를 클릭



- 생성되는 파일은 **xsa** 확장자를 가진 파일
- 파일명과 경로를 확인하고 Next를 클릭

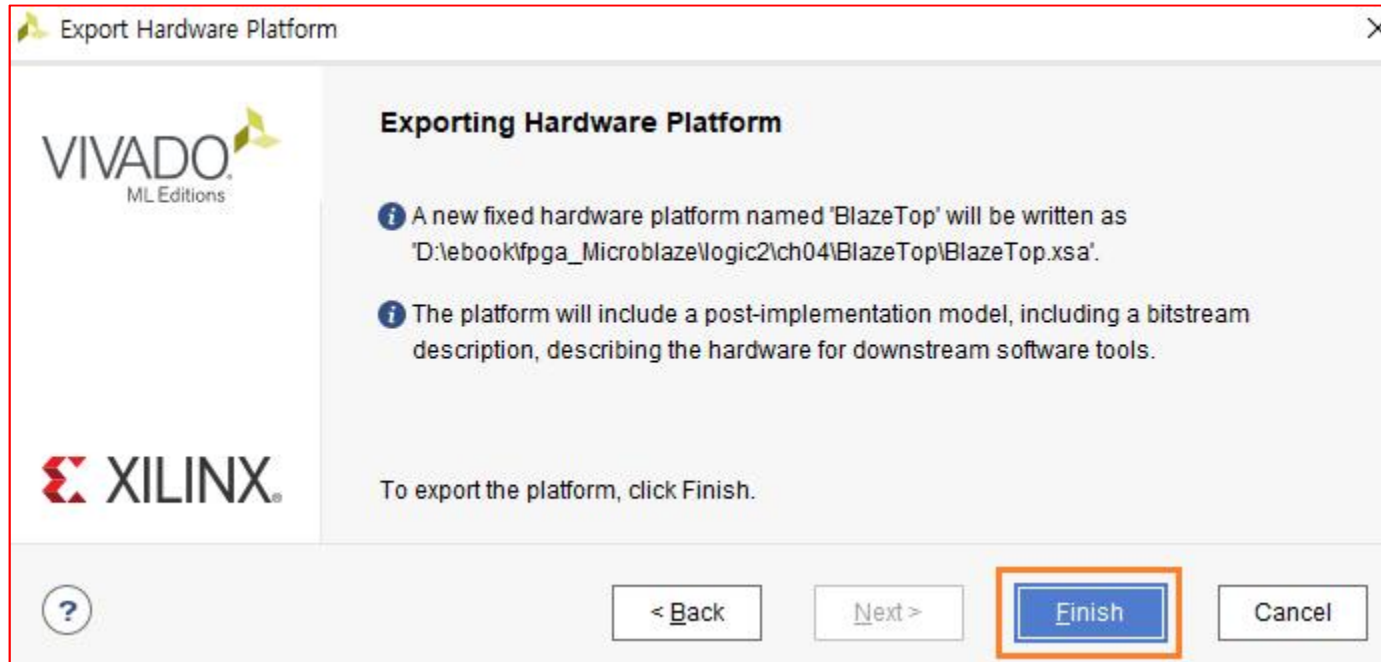


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ Export Hardware

- *Finish*를 클릭하면 XSA 파일이 생성



- 기본 *Template* 이 완료 ➔ 다른 응용에서 기본 *Template* 폴더를 복사하여 이름을 변경하여 사용
- [*Next*] *UART*를 통하여 *Hello world Message*를 출력하고, *LED*를 *On/Off* 하는 것을 구현
- [참고] *Vivado*는 폴더의 경로를 바꾸어서 사용해도 별 문제가 되지 않지만, *Vitis*는 프로젝트를 생성하고 폴더(경로나 이름)를 변경하면 인식을 못하는 문제 ➔ 이때, 프로젝트 환경 파일을 수동으로 수정해서 사용하면 되지만 번거로우므로, *Vitis*를 진행하기 전에 기본 *Template*를 복사해서 사용하면 편리

LED_Counter Implementation

Microblaze
BlazeTop.xpr

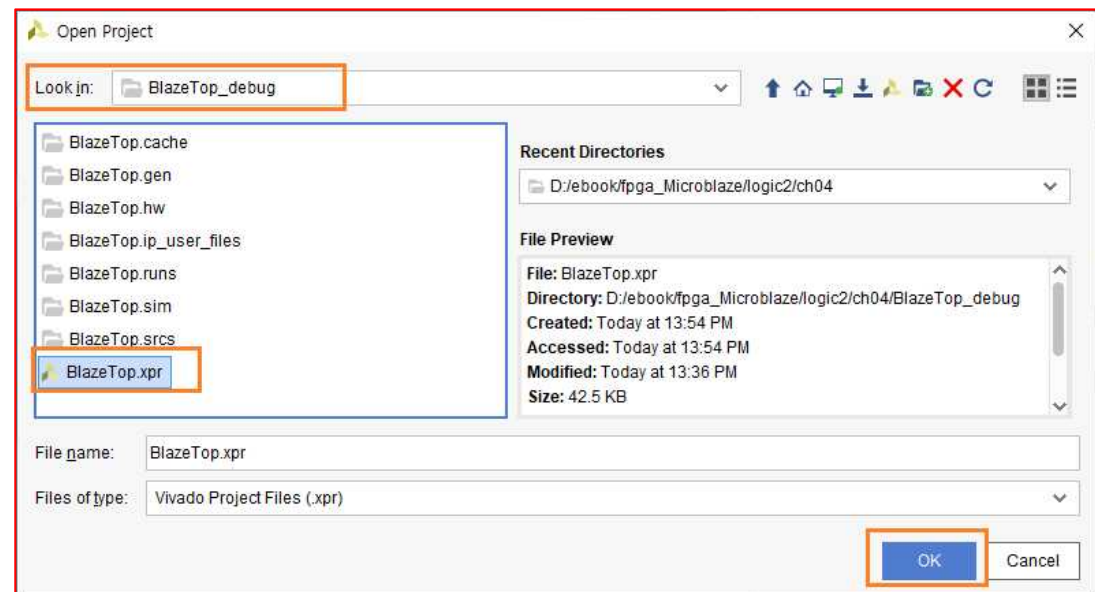
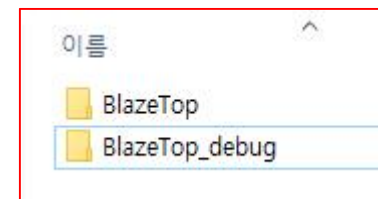
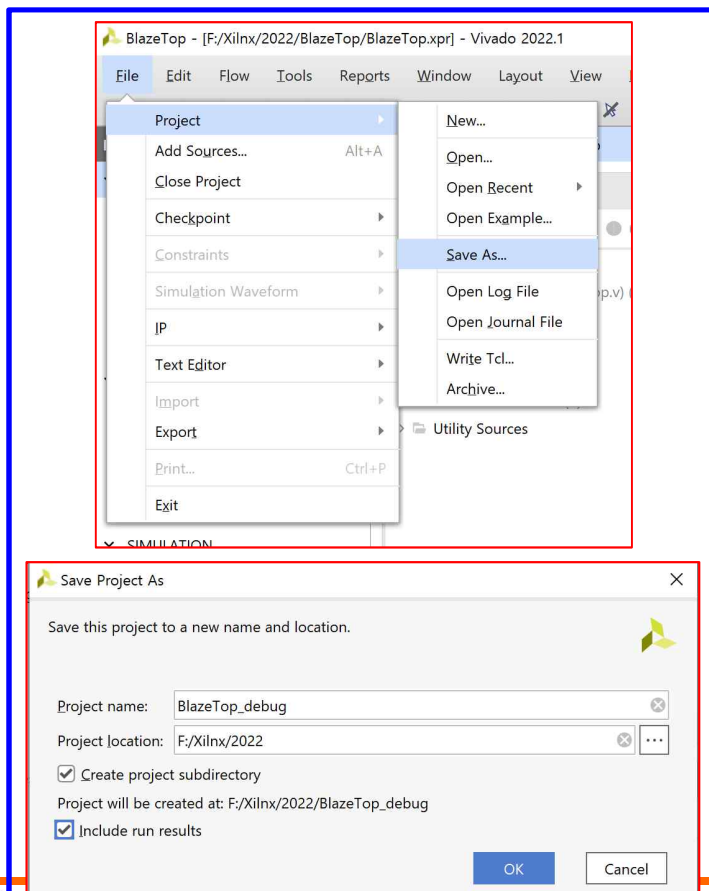
- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*
- *Create HDL Wrapper*
- *User Logic Implementation → Led_On/Off_Logic → led_control*
- *User Logic Implementation → Led_On/Off_Logic → BlazeTop*
- *XDC Implementation*
- *Generate Bitstream*
- *Export Hardware*
- ***SW Implementation → Launch Vitis IDE & Create Application Project***

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation (Skip 가능)

- Vitis를 이용하여 SW를 구현 ➔ Hello world Implementation 및 LED On/Off
- 먼저 앞에서 생성했던 폴더를 복사하여 이름을 [BlazeTop_debug]로 변경
- BlazeTop 폴더는 앞에서 생성한 기본 Template 폴더 BlazeTop_debug는 BlazeTop 폴더를 복사한 후 이름을 변경
- Vivado에서 BlazeTop Project를 닫고[File – Close Project], Open Project를 클릭 ➔ BlazeTop_debug 폴더에 있는 BlazeTop.xpr 파일을 Open

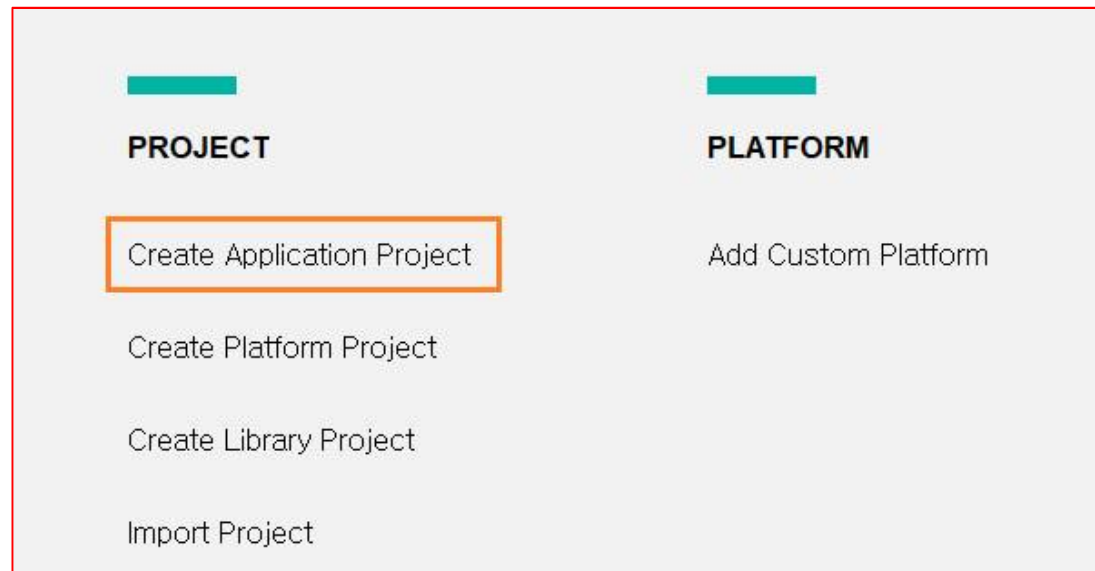
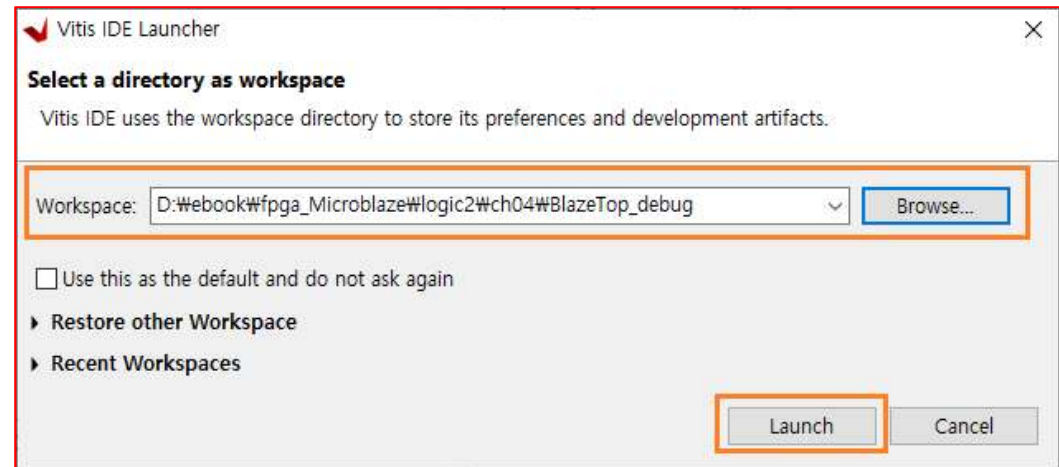


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

- XSA 파일까지 생성하였기 때문에 바로 Vitis에서 진행 가능 → **Tools → Launch Vitis IDE**를 클릭하면 Vitis가 실행
- Workspace가 BlazeTop_debug를 확인하고 Launch를 클릭 Workspace 위치가 다르면 Browse... 클릭해서 해당 위치를 변경)
- 새로운 프로젝트를 생성하기 위하여 **Create Application Project** → Click

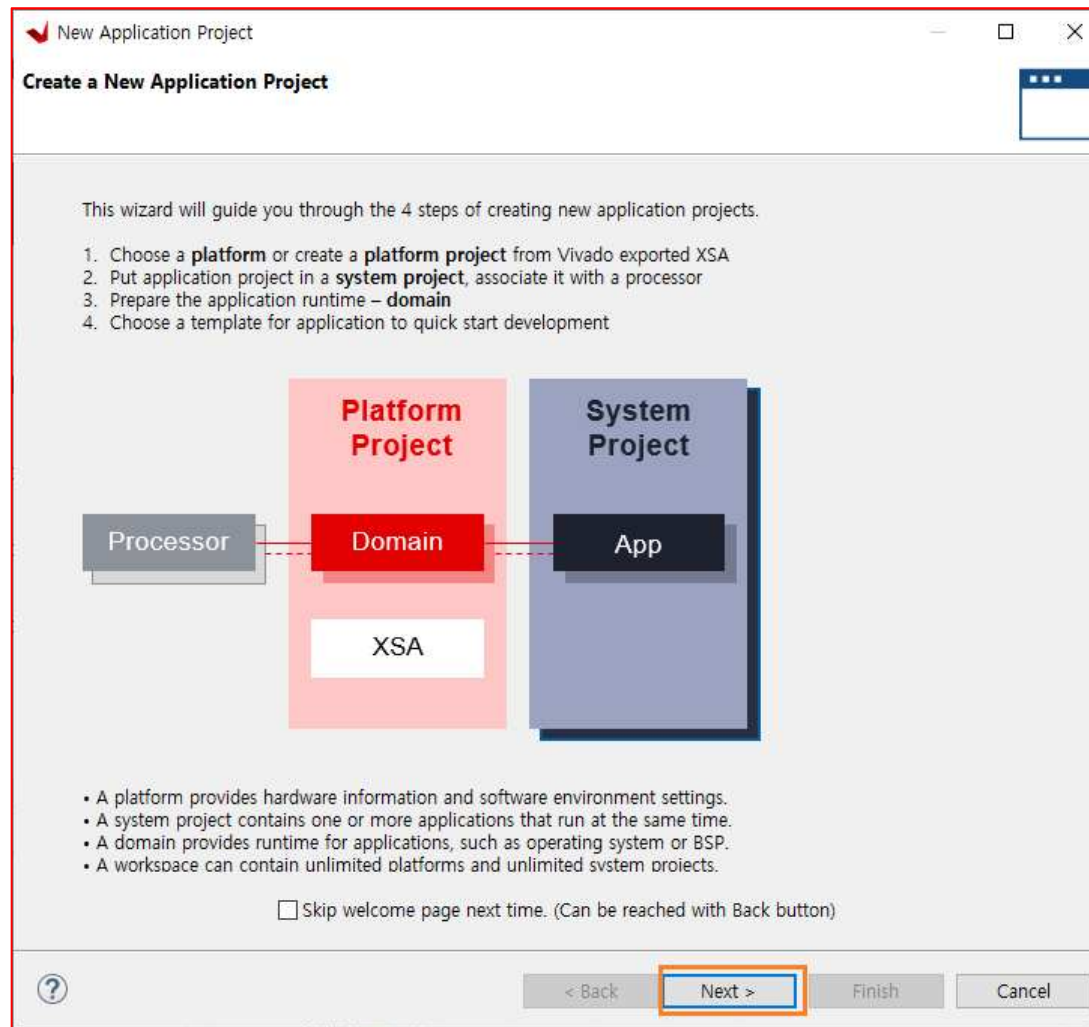


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

- Create a New Application Project ➔ Next 클릭



LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

- Create a new platform from hardware (XSA) 탭 선택 ➔ Browe... 클릭 ➔ Vivado에서 생성한 *.xsa 파일 Open

The screenshot shows the Vivado IDE interface. The 'New Application Project' dialog is open, with the 'Create a new platform from hardware (XSA)' tab selected. The 'Hardware Specification' section contains a text box with the prompt 'Provide your XSA file or use a pre-built board description' and a list of pre-built board descriptions: vck190, vmk180, zc702, zc706, zcu102, zcu106, and zed. A 'Browse...' button is visible to the right of the list. In the background, a file explorer window is open, showing the contents of the 'BlazeTop_debug' directory. The file explorer has columns for '이름' (Name), '수정된 날짜' (Modified Date), '유형' (Type), and '크기' (Size). The file 'BlazeTop.xsa' is highlighted in the list.

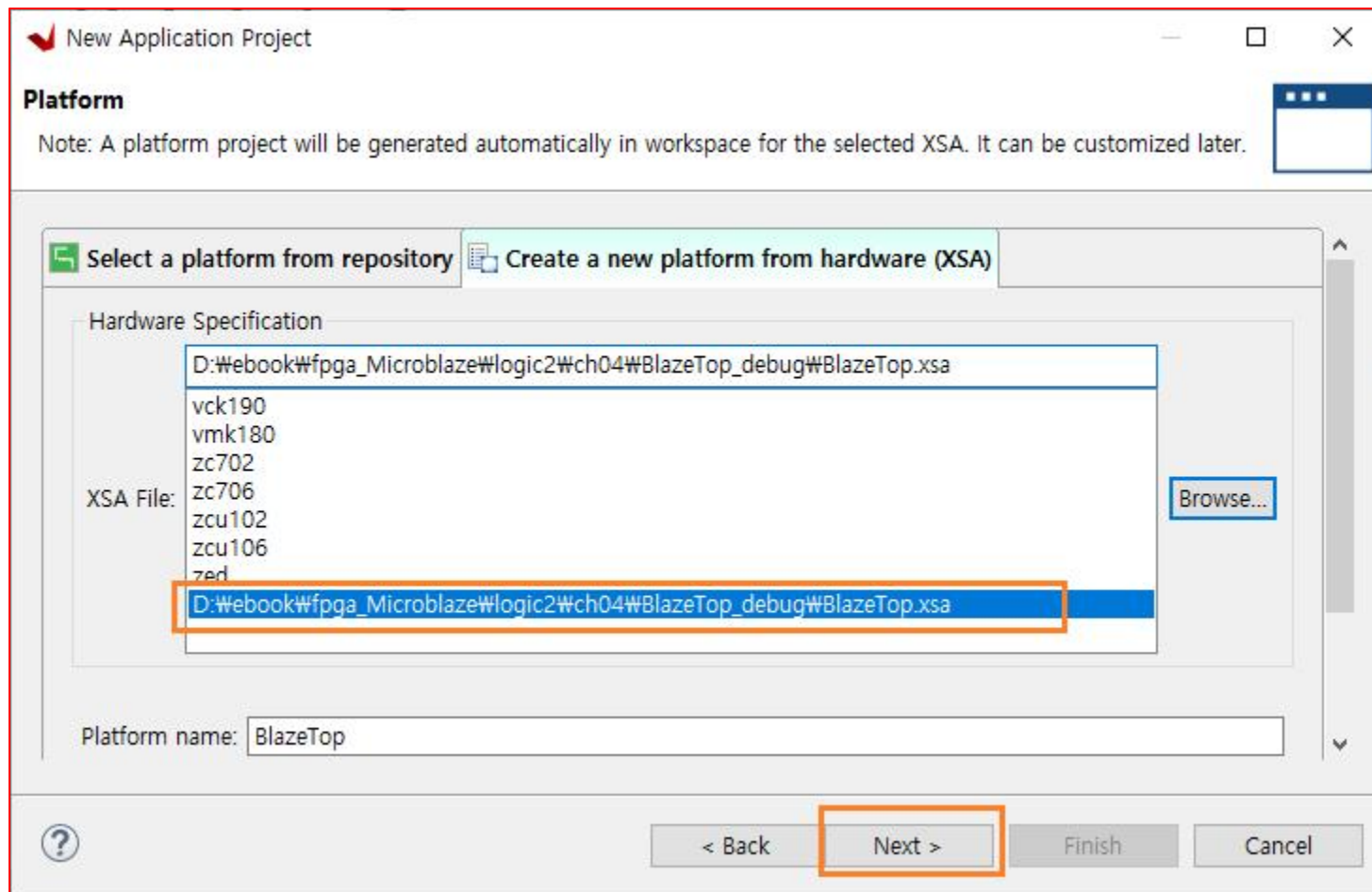
이름	수정된 날짜	유형	크기
.metadata	2023-08-10 오후 2:03	파일 폴더	
BlazeTop.cache	2023-08-10 오후 1:54	파일 폴더	
BlazeTop.gen	2023-08-10 오후 1:58	파일 폴더	
BlazeTop.hw	2023-08-10 오후 1:54	파일 폴더	
BlazeTop.ip_user_files	2023-08-10 오후 12:39	파일 폴더	
BlazeTop.runs	2023-08-10 오후 1:54	파일 폴더	
BlazeTop.sim	2023-08-10 오후 12:39	파일 폴더	
BlazeTop.srcs	2023-08-10 오후 1:54	파일 폴더	
RemoteSystemsTempFiles	2023-08-10 오후 2:03	파일 폴더	
BlazeTop.xsa	2023-08-10 오후 1:44	XSA 파일	

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

- XSA 파일을 설정하면 Next 버튼이 활성화
- Platform → Create a new platform from hardware(XSA) → Next를 클릭



LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

- Application Project Details ➔ Application project name (BlazeTopApp)을 설정 ➔ Next를 클릭

New Application Project

Application Project Details
Specify the application project name and its system project properties

Application project name: BlazeTopApp

System Project
Create a new system project for the application or select an existing one from the workspace

Select a system project
+ Create new...

System project details

System project name: BlazeTopApp_system

Target processor

Select target processor for the Application project.

Processor	Associated applications
microblaze_0	BlazeTopApp

Show all processors in the hardware specification ☒

< Back Next > Finish Cancel

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

- Domain ➔ [standalone_microblaze_0] ➔ Next 클릭

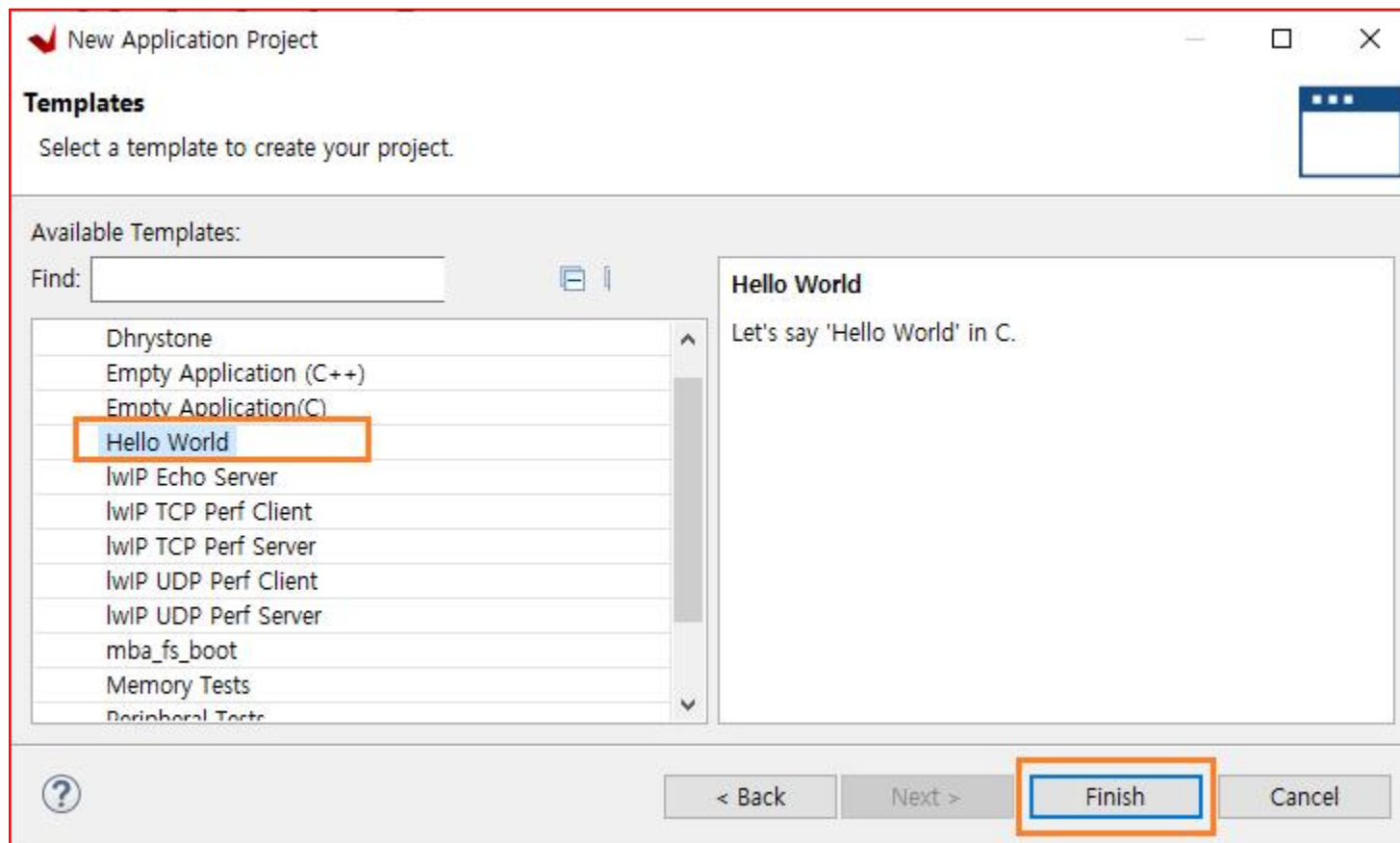
The screenshot shows the 'New Application Project' wizard window. The title bar says 'New Application Project'. The main heading is 'Domain'. Below it, the text says 'Select a domain for your project or create a new domain'. There is a small icon of a window with three dots in the top right corner. Below this, the text says 'Select the domain that the application would link to or create a new domain'. A note below that says 'Note: New domain created by this wizard will have all the requirements of the application template selected in the next step'. On the left, there is a box labeled 'Select a domain' with a '+ Create new...' button. On the right, there is a 'Domain details' section with the following fields: 'Name:' (standalone_microblaze_0), 'Display Name:' (standalone_microblaze_0), 'Operating System:' (standalone), 'Processor:' (microblaze_0), and 'Architecture:' (32-bit). At the bottom, there are four buttons: '?', '< Back', 'Next >', and 'Finish'. The 'Next >' button is highlighted with an orange box. There is also a 'Cancel' button.

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

- **Templates** 에서 **'Hello World'**를 선택하고 **Finish**를 클릭 ➔ 프로젝트 생성

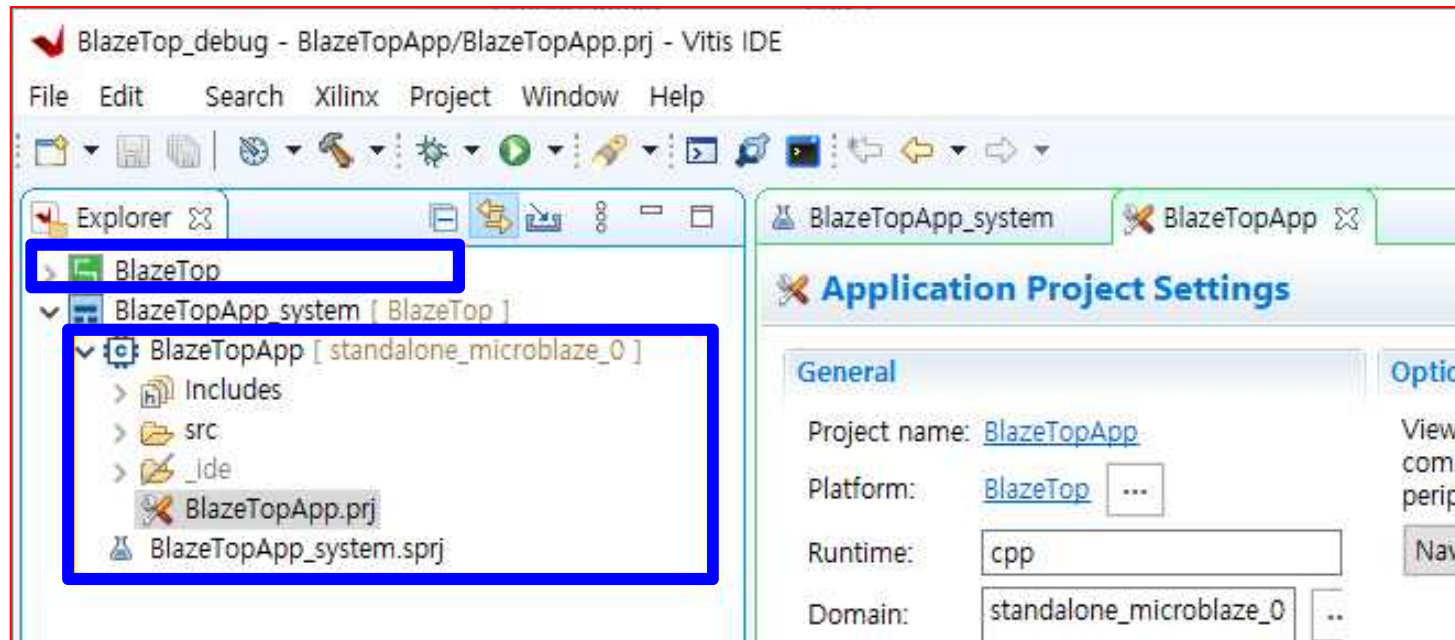


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

- Vitis에는 2개의 Project
 - ✓ 하나는 Vivado에서 생성 후 XSA 파일로 변환한 프로젝트 파일
 - ✓ 나머지 하나는 Vitis에서 생성한 SW용 프로젝트
 - ✓ 즉, **BlazeTop**이 Vivado에서 생성한 프로젝트, **BlazeTopApp**가 Vitis에서 생성한 프로젝트



- Vivado에서 HW관련된 내용들이 수정되어 xsa 파일이 수정되면, **BlazeTop을 Update → Re-Build**를 해야 함
- SW가 수정되면, **BlazeTopApp를 Re-Build** 해야 함

LED_Counter Implementation

Microblaze
BlazeTop.xpr

- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*
- *Create HDL Wrapper*
- *User Logic Implementation → Led_On/Off_Logic → led_control*
- *User Logic Implementation → Led_On/Off_Logic → BlazeTop*
- *XDC Implementation*
- *Generate Bitstream*
- *Export Hardware*
- *SW Implementation → Launch Vitis IDE & Create Application Project*
- ***SW Implementation → [helloworld.c] Code Implementation***

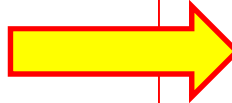
LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

- **BlazeTopApp** → [src] 폴더 → **helloworld.c** 의 내용 수정 → 1초마다 메시지를 표시하는 간단한 프로그램

```
48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51
52
53 int main()
54 {
55     init_platform();
56
57     print("Hello World\n\r");
58     print("Successfully ran Hello World application");
59     cleanup_platform();
60     return 0;
61 }
```



```
48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51 #include "sleep.h"
52
53 int main()
54 {
55     int cnt = 0;
56
57     init_platform();
58
59     print("Hello World ..\r\n");
60     while(1)
61     {
62         xil_printf("count : %d \r\n", cnt++);
63         sleep(1);
64     }
65     cleanup_platform();
66     return 0;
67 }
```

- **BlazeTopApp** → 우클릭 → **Build Project**를 → Click → **Error 없이 Build**

```
Build Console [BlazeTopApp, Debug]
mb-gcc -Wl,-T -Wl,.../src/lscript.ld -LD:/ebook/fpga_Microblaze/...
'Finished building target: BlazeTopApp.elf'
'Invoking: MicroBlaze Print Size'
mb-size BlazeTopApp.elf |tee "BlazeTopApp.elf.size"
text  data    bss    dec    hex filename
6184   328    3104   9616   2590 BlazeTopApp.elf
'Finished building: BlazeTopApp.elf.size'

14:36:19 Build Finished (took 668ms)
```

- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*
- *Create HDL Wrapper*
- *User Logic Implementation → Led_On/Off_Logic → led_control*
- *User Logic Implementation → Led_On/Off_Logic → BlazeTop*
- *XDC Implementation*
- *Generate Bitstream*
- *Export Hardware*
- *SW Implementation → Launch Vitis IDE & Create Application Project*
- *SW Implementation → [helloworld.c] Code Implementation*
- *SW Implementation → Build Project , Debug and Program Device*

LED Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

■ Build Project or Build All(1/2)

The screenshot shows the Vitis IDE interface for a project named 'BlazeTopApp_system'. The 'Debug' tab is active, and the 'Build Project' option is highlighted in the menu. The main editor displays the source code for 'helloworld.c', which includes UART configuration, standard library headers, and a main function that prints 'Hello World' and enters a loop.

```
41 * | UART TYPE  BAUD RATE
42 *
43 *   uartns550  9600
44 *   uartlite   Configurable only in HW design
45 *   ps7_uart   115200 (configured by bootrom/bsp)
46 */
47
48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51 #include "sleep.h"
52
53 int main()
54 {
55     int cnt = 0;
56
57     init_platform();
58
59     print("Hello World\n\r");
60     print("Successfully ran Hello World application");
61
62     while(1)
63     {
64         xil_printf("count : %d \r\n" , cnt++);
65         sleep(1);
66     }
67
68     cleanup_platform();
69     return 0;
70 }
71
72
```

The right-hand side of the IDE shows the 'Vari...' window with a table of variables:

Name	Type	Value
(x)= cnt	int	1

The bottom of the IDE shows the 'Console' window with the following output:

```
Program Device
Writing bitstream F:/Xilinx/2022/BlazeTop/BlazeTopApp/_ide/bitstream/download.bit...
0 Infos, 0 Warnings, 0 Critical Warnings and 0 Errors encountered.
update_mem completed successfully
update_mem: Time (s): cpu = 00:00:17 ; elapsed = 00:00:16 . Memory (MB): peak = 1745.277 ; gain :
INFO: [Common 17-206] Exiting update_mem at Wed Nov 15 03:13:27 2023...
```

The 'XSCT Console' window shows the following output:

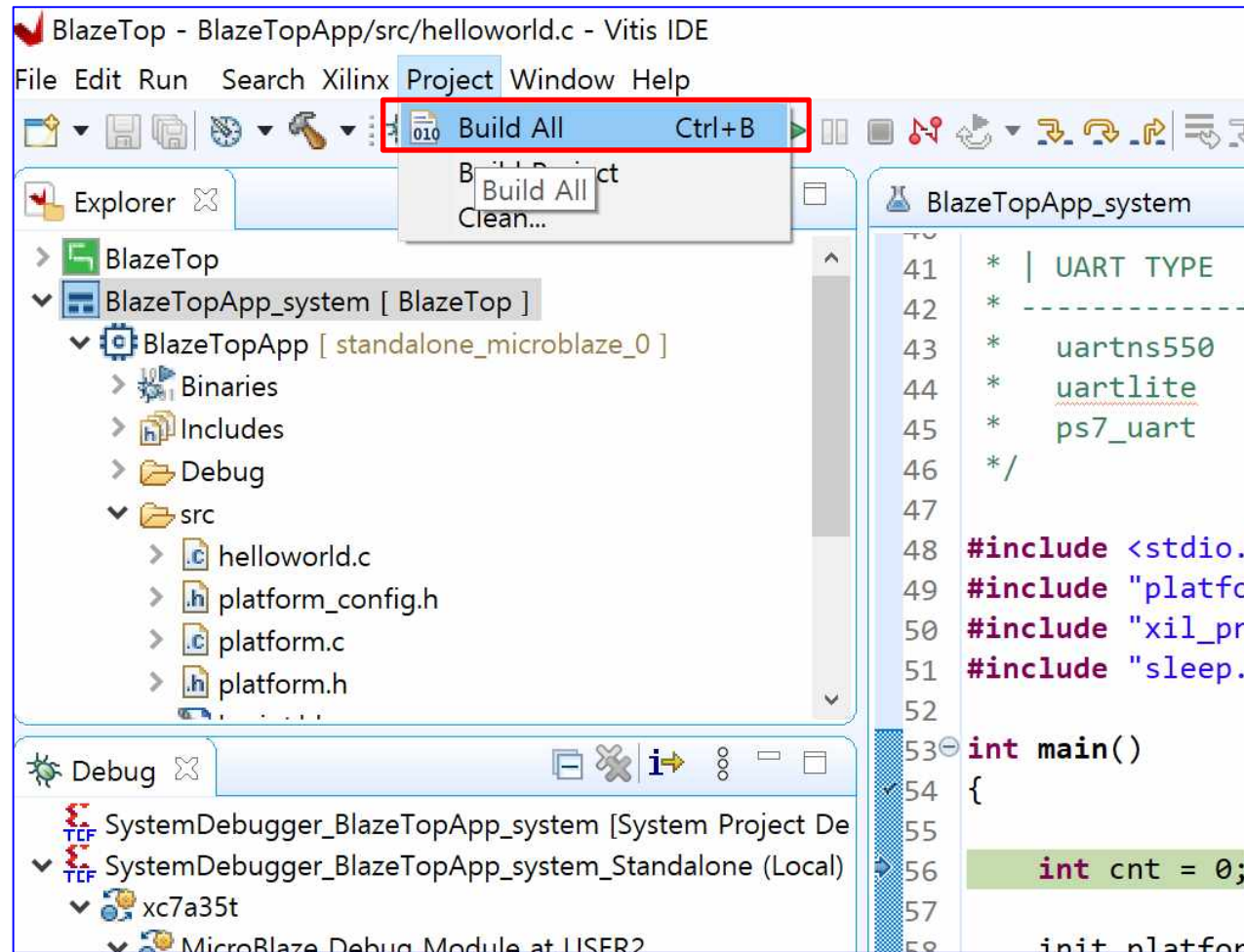
```
XSCT Process
Successfully downloaded F:/Xilinx/2022/BlazeTop/BlazeTopApp/_ide/bitstream/download.bit...
Info: MicroBlaze #0 (target 3) Running
Info: MicroBlaze #0 (target 3) Stopped at
main() at ../src/helloworld.c: 56
56:     int cnt = 0;
xsct%
xsct%
```

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

- Build Project or Build All(2/2)

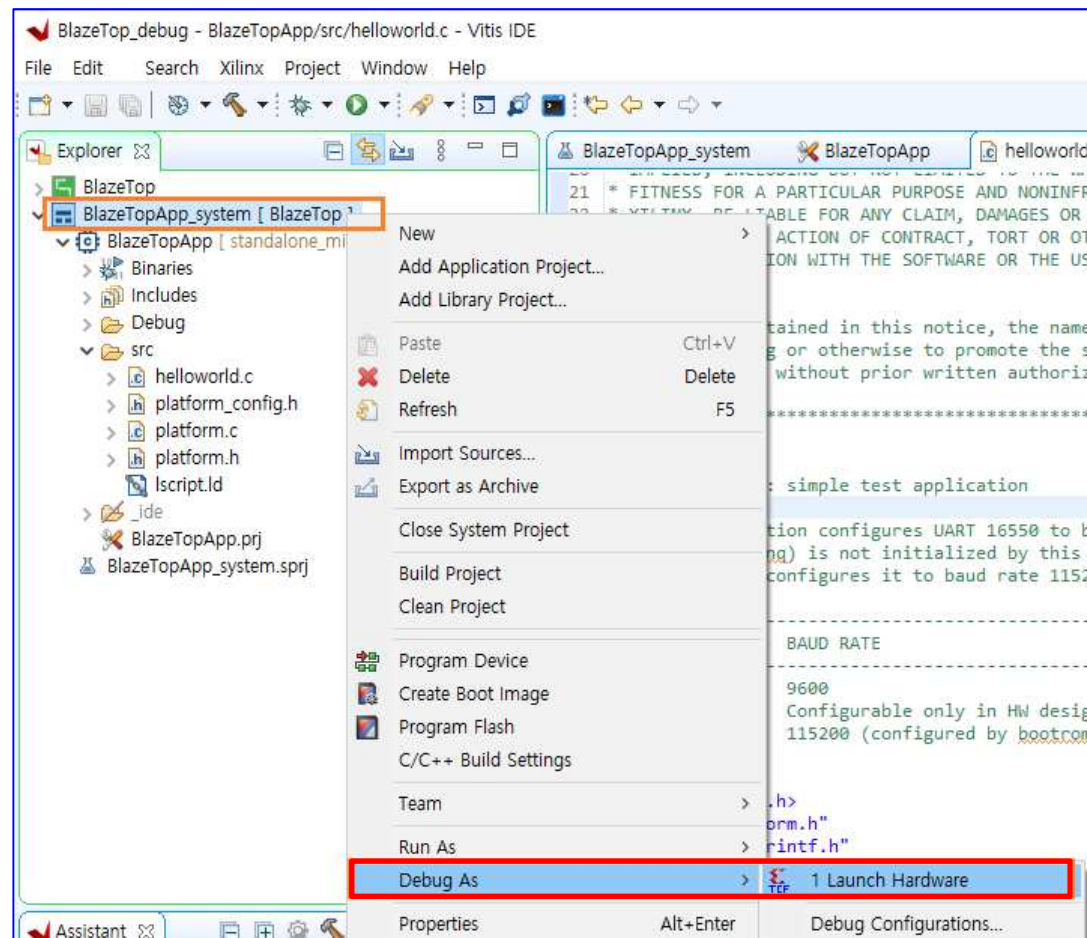


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation → Program Download and Check Result

- 결과 확인하기 위하여 보드에 다운로드
- Basys3 보드와 PC를 USB로 연결 → 전원 인가 → 보드가 인식
- BlazeTopApp_system → 우클릭 → Debug As → 1 Launch Hardware 클릭

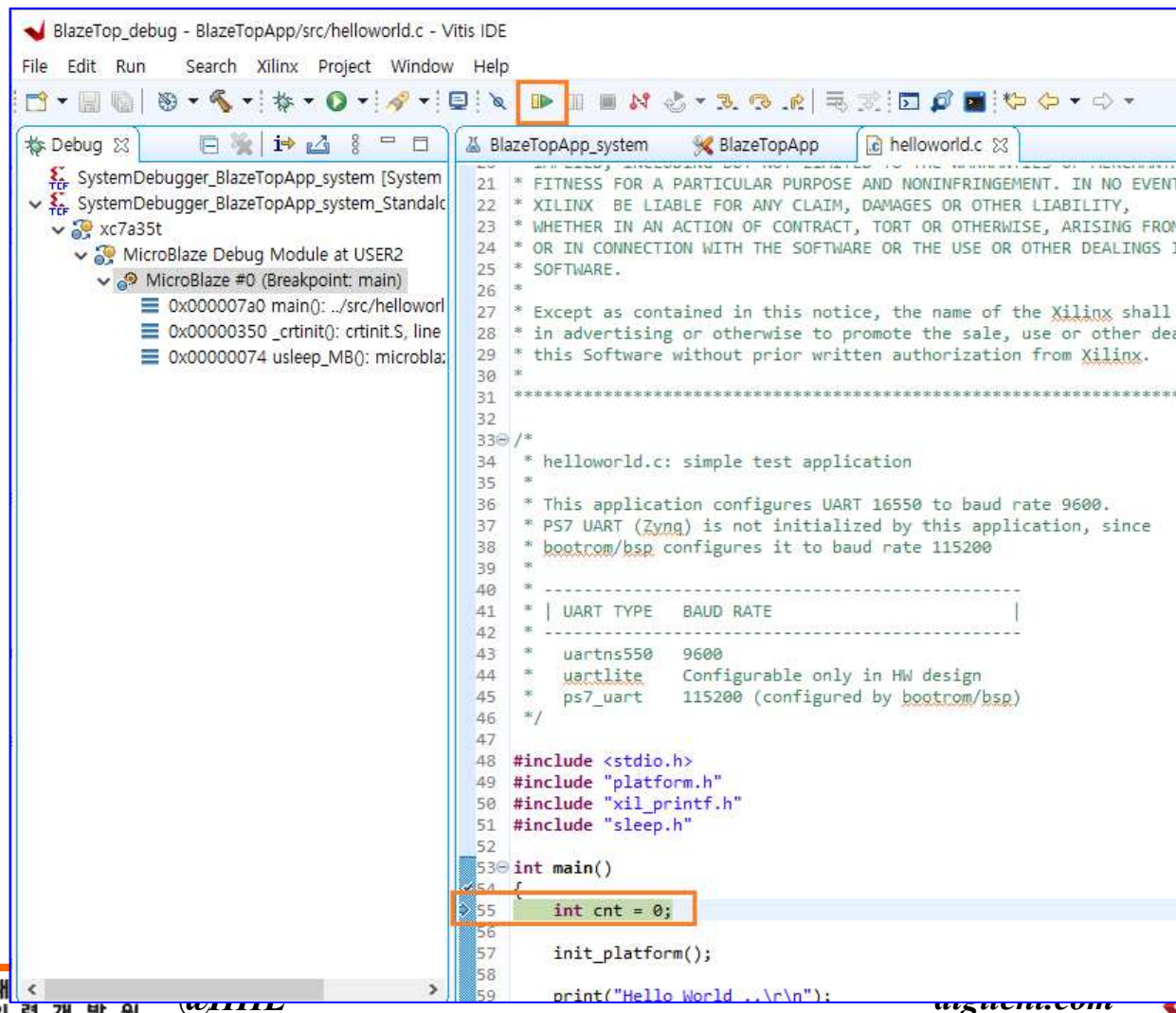


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation → Program Download

- 프로그램이 다운로드 ➡ 우측 하단에 다운로드 과정이 %로 표시 ➡ 모두 다운로드가 되면 디버깅 모드가 되고 main()함수의 시작부분에 커서가 위치



```
BlazeTop_debug - BlazeTopApp/src/helloworld.c - Vitis IDE
File Edit Run Search Xilinx Project Window Help

Debug
SystemDebugger_BlazeTopApp_system [System
SystemDebugger_BlazeTopApp_system_Standal
xc7a35t
MicroBlaze Debug Module at USER2
MicroBlaze #0 (Breakpoint: main)
0x000007a0 main(): ../src/helloworl
0x00000350 _crtinit(): crtinit.S, line
0x00000074 usleep_MB(): microbla

21 * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT
22 * XILINX BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
23 * WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
24 * OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
25 * SOFTWARE.
26 *
27 * Except as contained in this notice, the name of the Xilinx shall n
28 * in advertising or otherwise to promote the sale, use or other deal
29 * this Software without prior written authorization from Xilinx.
30 *
31 *****
32
33 /*
34 * helloworld.c: simple test application
35 *
36 * This application configures UART 16550 to baud rate 9600.
37 * PS7 UART (Zynq) is not initialized by this application, since
38 * bootrom/bsp configures it to baud rate 115200
39 *
40 *
41 * | UART TYPE   BAUD RATE |
42 * |-----|
43 * | uartns550   9600
44 * | uartrlite   Configurable only in HW design
45 * | ps7_uart    115200 (configured by bootrom/bsp)
46 */
47
48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51 #include "sleep.h"
52
53 int main()
54 {
55     int cnt = 0;
56
57     init_platform();
58
59     print("Hello World...\n");
```

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

■ Program Device(1/4)

The screenshot displays the Vitis IDE interface for a Microblaze project. The top toolbar shows various icons for file operations, compilation, and execution. The Explorer pane on the left shows the project hierarchy: BlazeTop > BlazeTopApp_system [BlazeTop] > BlazeTopApp [standalone_microblaze_0] > src. The Debug pane shows the system configuration, including the MicroBlaze #0 (Breakpoint: main). The Assistant pane on the bottom left shows a context menu for the 'BlazeTopApp_system' system, with the 'Program Device' option highlighted. The main editor window shows the source code for 'helloworld.c', which includes headers for stdio.h, platform.h, xil_printf.h, and sleep.h. The code defines a main function that initializes the platform, prints 'Hello World\n' and 'Successfully', and enters a while loop that prints 'count' and sleeps for 1 second. The Console pane at the bottom right shows the output of the 'Program Device' command, indicating that the bitstream was written successfully and the device was programmed.

```
BlazeTop - BlazeTopApp/src/helloworld.c - Vitis IDE
File Edit Run Search Xilinx Project Window Help

Explorer
BlazeTop
├── BlazeTopApp_system [BlazeTop]
│   └── BlazeTopApp [standalone_microblaze_0]
│       ├── Binaries
│       ├── Includes
│       ├── Debug
│       └── src
│           ├── helloworld.c
│           ├── platform_config.h
│           ├── platform.c
│           └── platform.h
└── ...

Debug
SystemDebugger_BlazeTopApp_system [System Project De
SystemDebugger_BlazeTopApp_system_Standalone (Local)
├── xc7a35t
│   └── MicroBlaze Debug Module at USER2
│       └── MicroBlaze #0 (Breakpoint: main)
│           ├── 0x000007a0 main(): ../src/helloworld.c, line 56
│           ├── 0x00000350 _crtinit(): crtinit.S, line 126
│           └── 0x00000074 usleep_MB(): microblaze_sleep.c, li

Assistant
BlazeTop [Platform]
├── BlazeTopApp_system [System]
│   ├── Settings...
│   ├── Build
│   ├── Clean
│   ├── Create Makefiles
│   ├── Run
│   ├── Debug
│   ├── Open in Explorer
│   ├── Export Vitis Project...
│   ├── Delete
│   └── Program Device
│       ├── Create Boot Image
│       ├── Program the Device
│       └── Program Flash
└── Refresh Project Models

Console
Vitis Serial Termi...

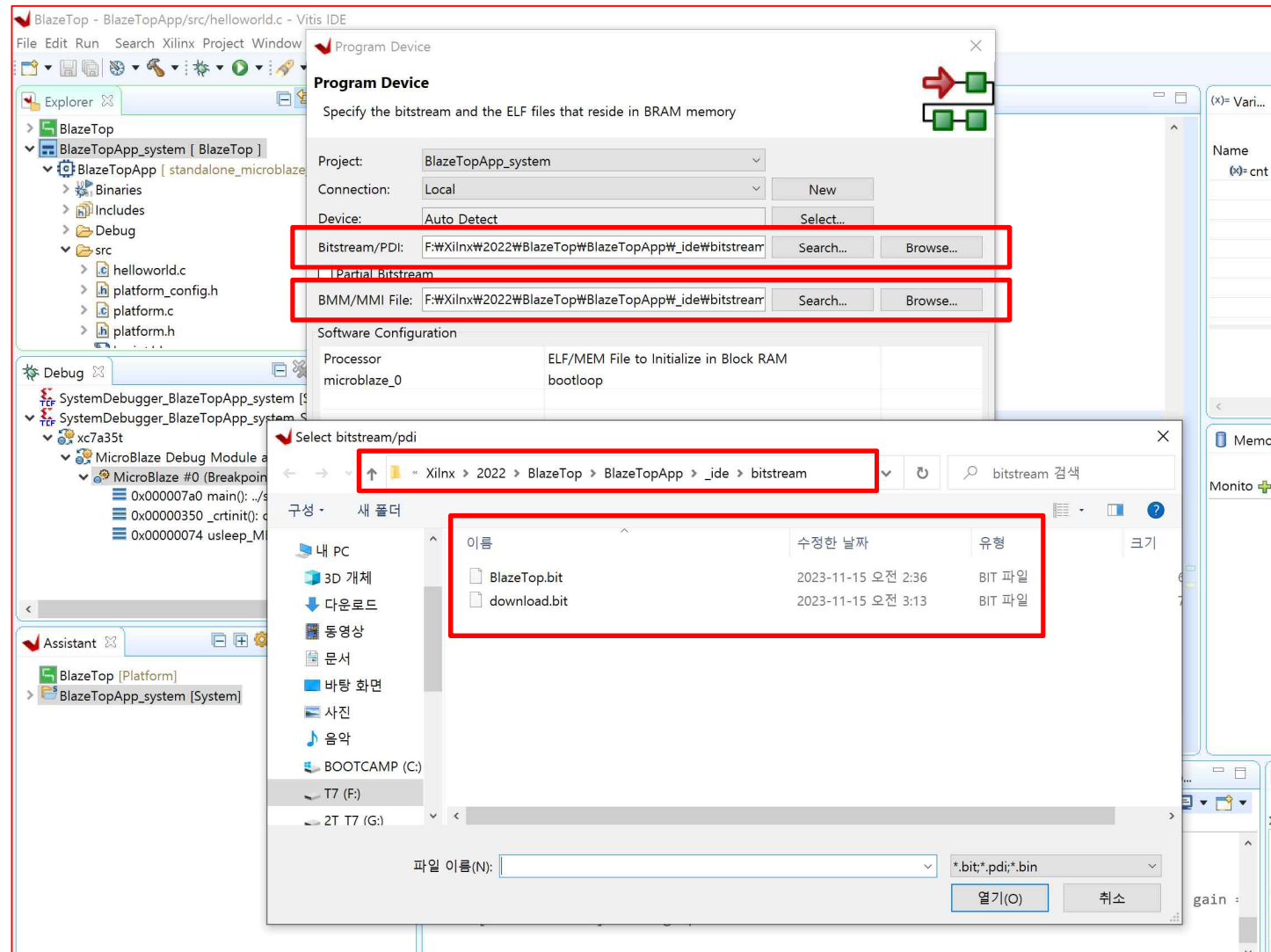
Program Device
Writing bitstream F:/Xilinx/20
0 Infos, 0 Warnings, 0 Critic
update_mem completed successf
update_mem: Time (s): cpu = 0
INFO: [Common 17-206] Exiting
```


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

■ Program Device(2/4)

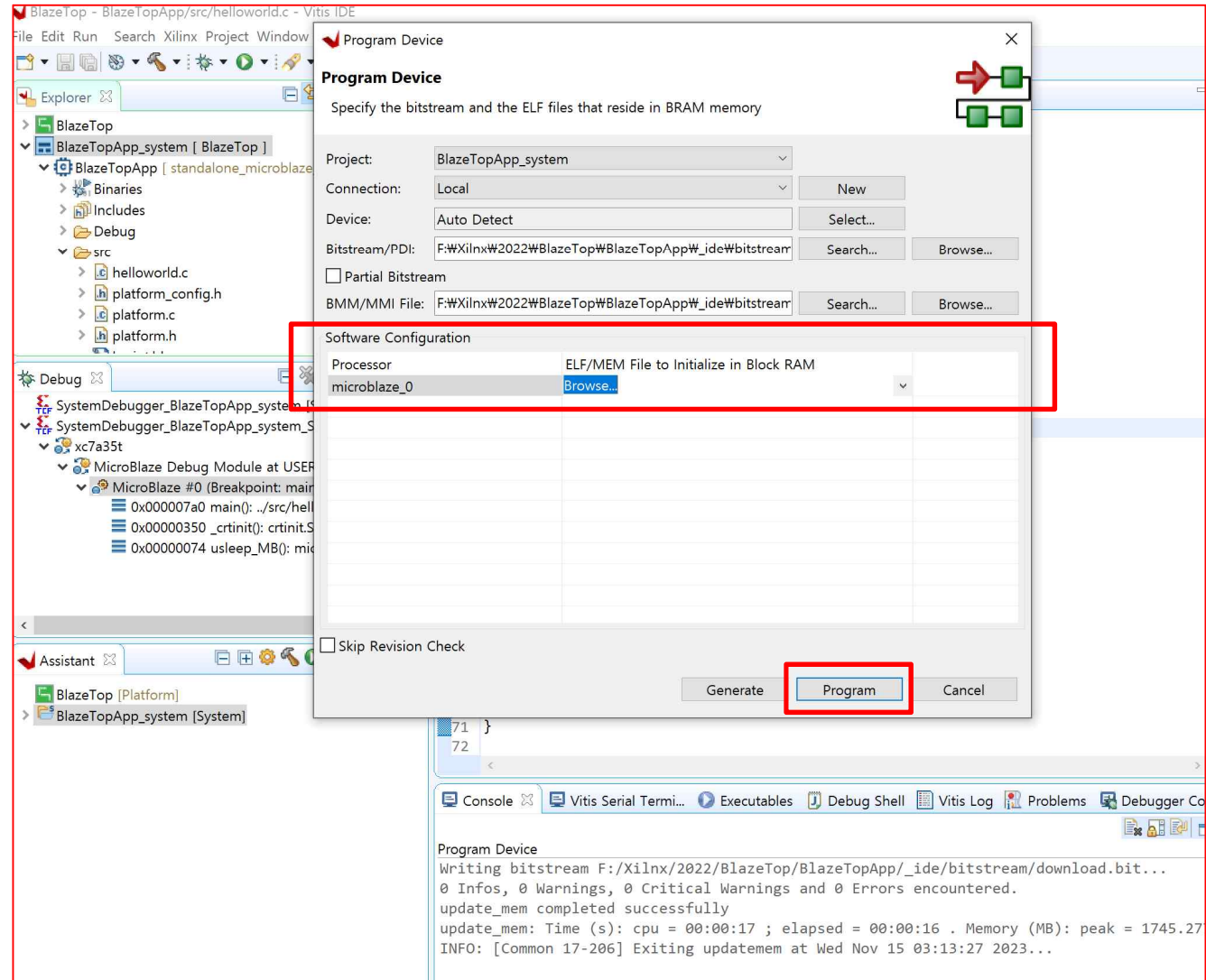


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

■ Program Device(3/4)

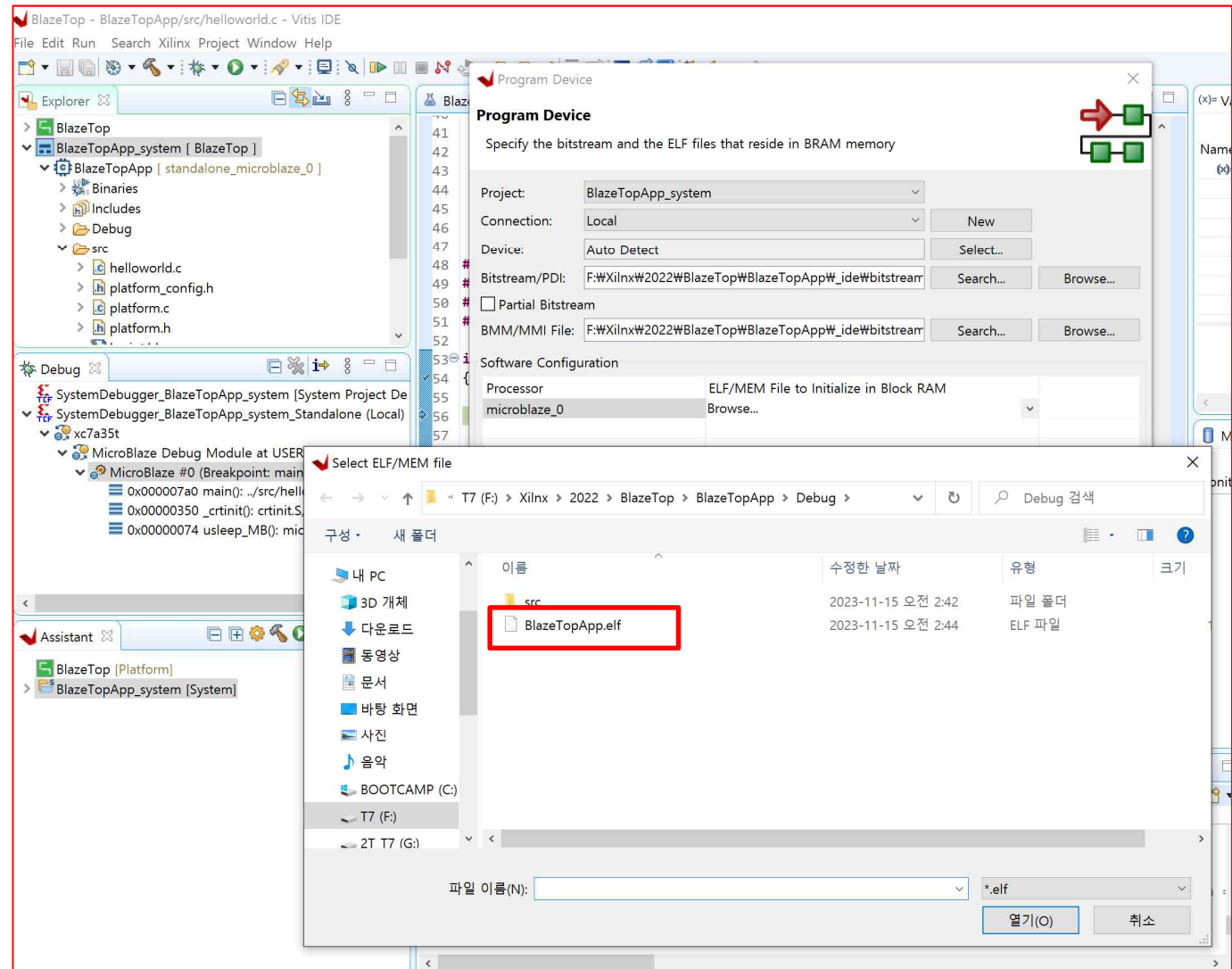


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation

■ Program Device(4/4)



LED_Counter Implementation

Microblaze
BlazeTop.xpr

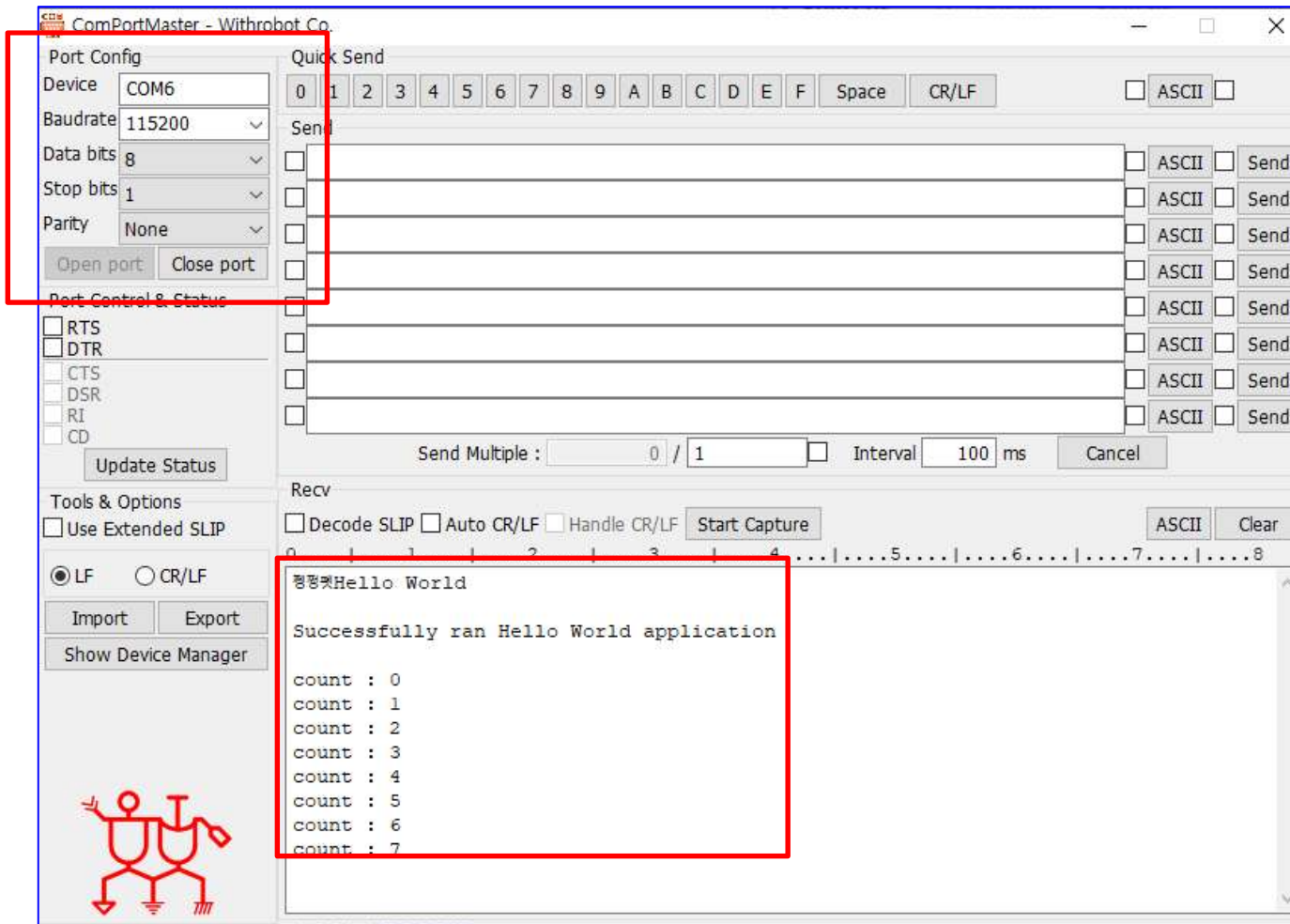
- *Create Project*
- *Create Block Design*
- *Add IP → MicroBlaze*
- *Add IP → AXI Uartlite*
- *Change Port Name and Interrupt Pin Connection*
- *Validate Design and Regenerate Layout*
- *Create HDL Wrapper*
- *User Logic Implementation → Led_On/Off_Logic → led_control*
- *User Logic Implementation → Led_On/Off_Logic → BlazeTop*
- *XDC Implementation*
- *Generate Bitstream*
- *Export Hardware*
- *SW Implementation → Launch Vitis IDE & Create Application Project*
- *SW Implementation → [helloworld.c] Code Implementation*
- *SW Implementation → Build Project , Debug and Program Device*
- ***Check The Result***

LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation → Check The Result

- Resume 아이콘()을 클릭하면 프로그램이 동작
- 터미널(ComPortMaster ...) 프로그램을 실행하고 ComPortMaster를 Open하면 메시지가 1초마다 출력

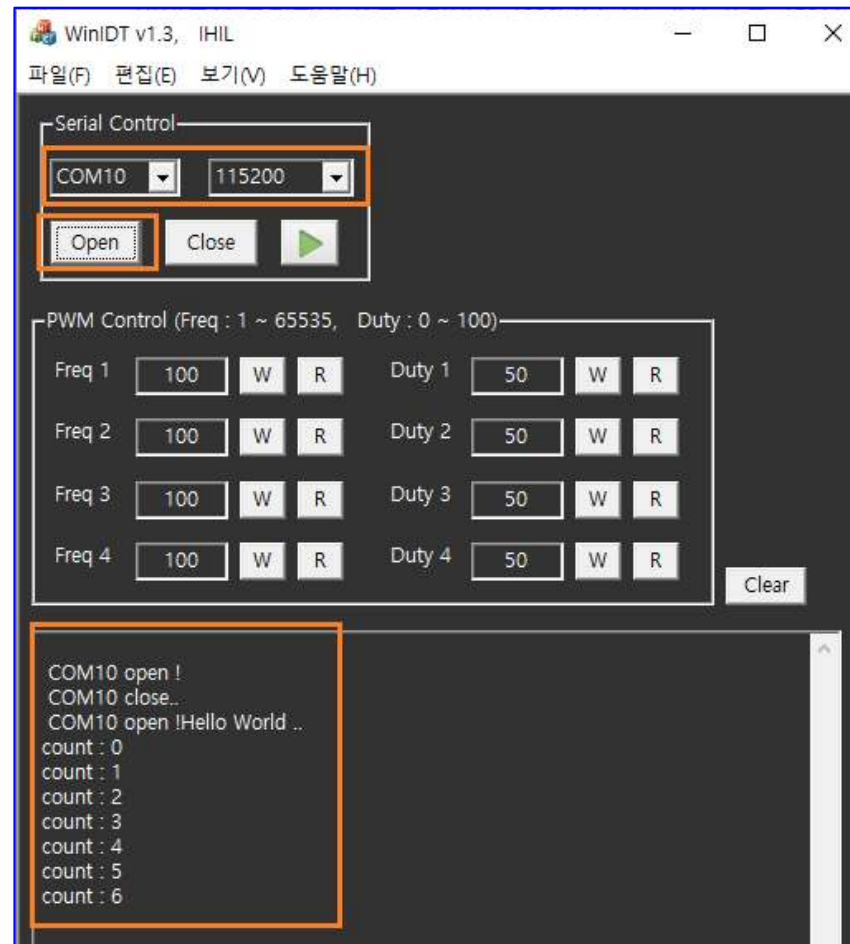


LED_Counter Implementation

Microblaze
BlazeTop.xpr

➤ SW Implementation → Check The Result

- Resume 아이콘()을 클릭하면 프로그램이 동작
- WinIDT 프로그램을 실행하고 Comport를 Open하면 메시지가 1초마다 출력

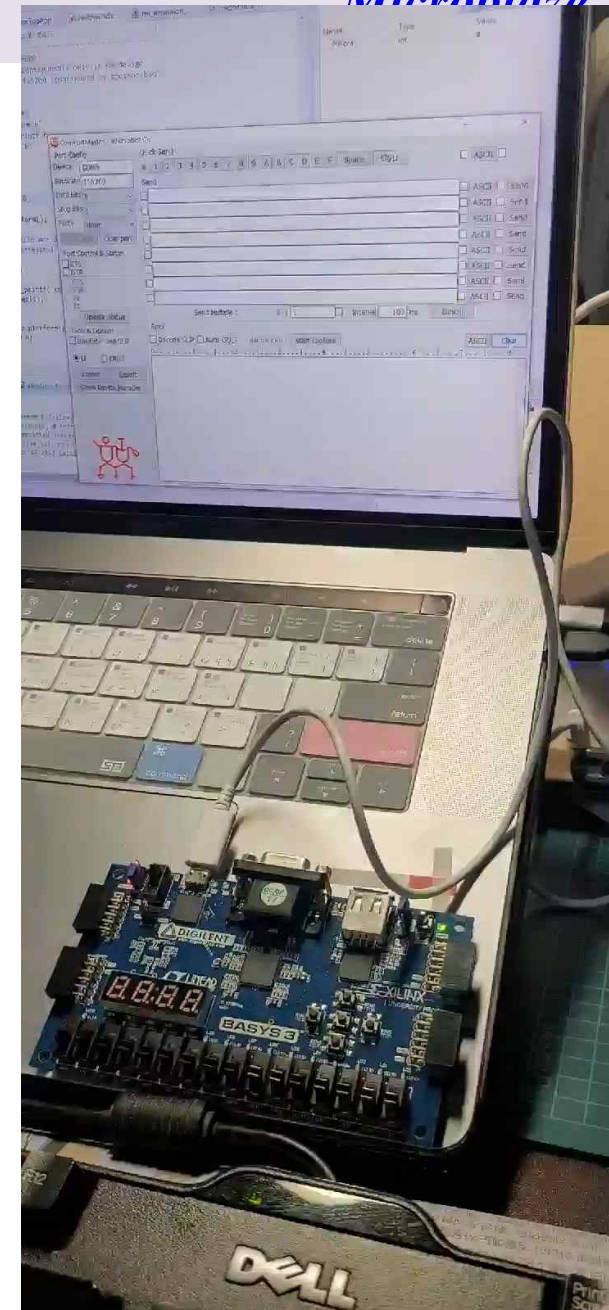


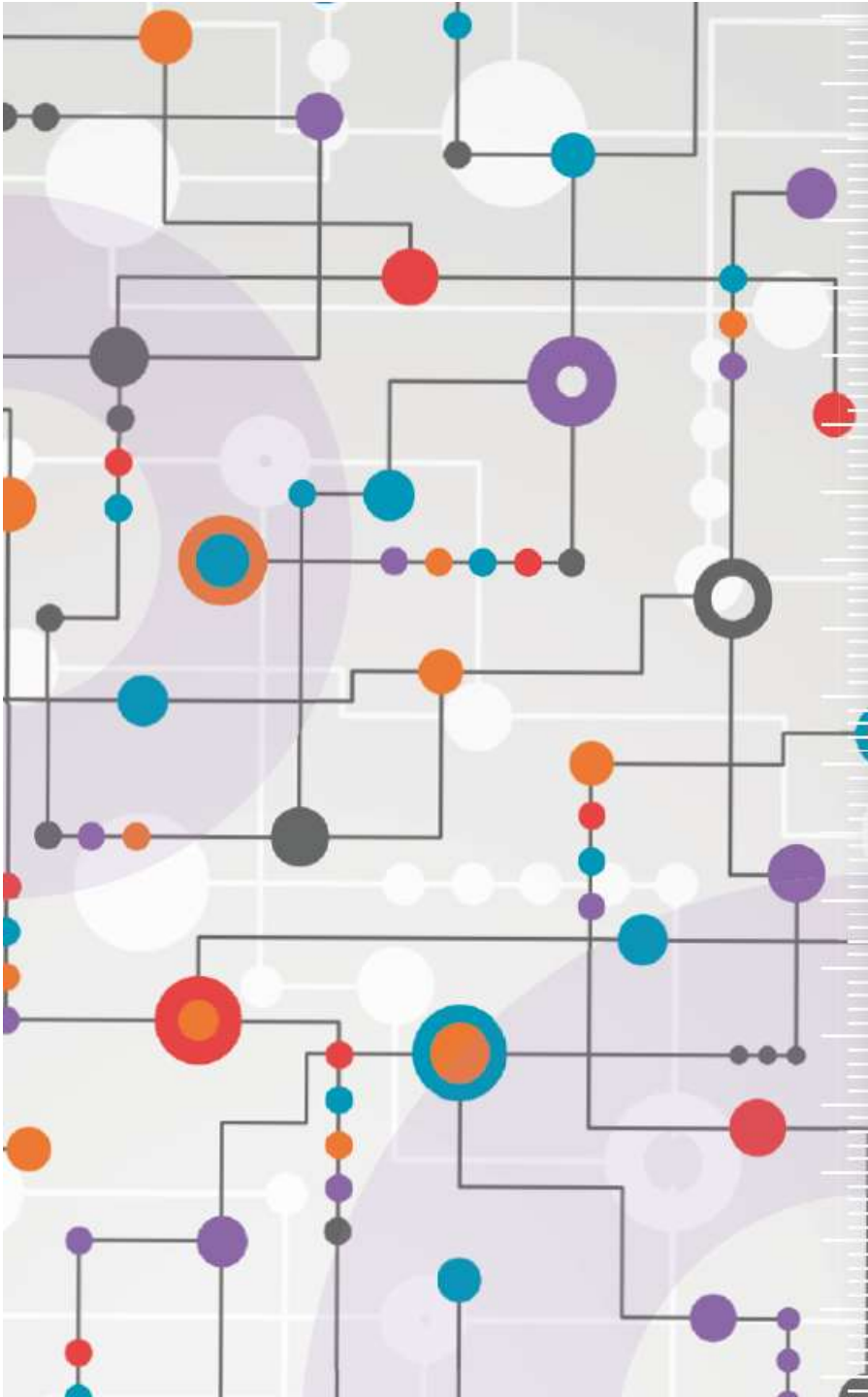
LED_Counter Implementation

Microblaze

➤ Check The Result

- LD0 ~ LD3 이 0.5초 간격으로 On/Off
- 기본적인 동작이 완료 ➔ *led_counter* 모듈에서 구현한 LED on/off 동작과 SW에서 구현한 1초마다 메시지를 표시하는 것이 완료





수고하셨습니다.